

Redes Neurais

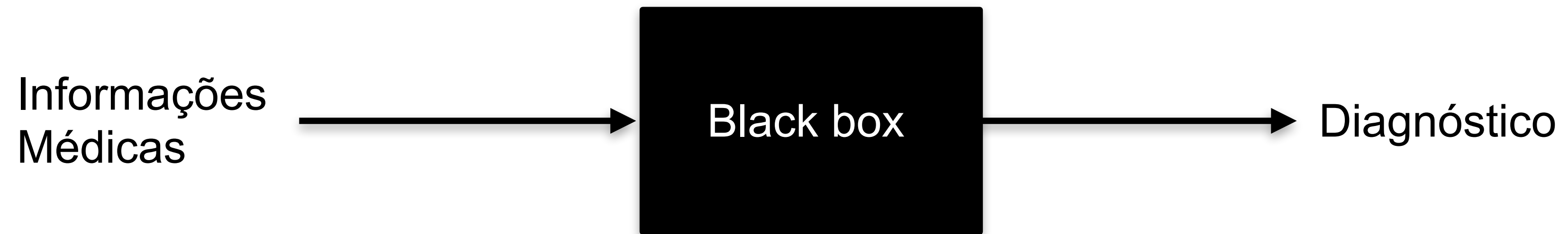
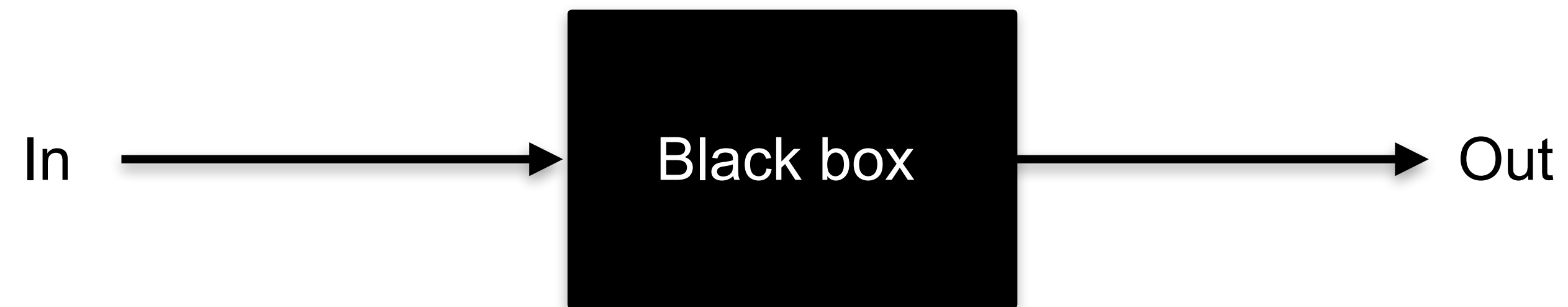
Francisco Rodrigues

Instituto de Ciências Matemáticas e de Computação

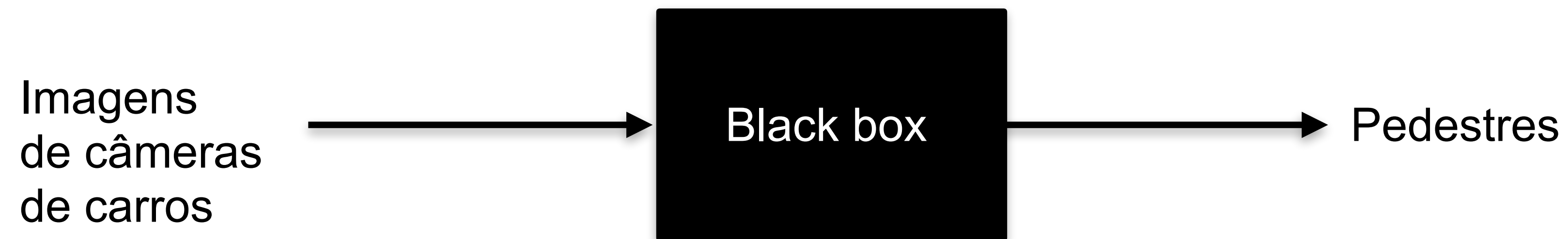
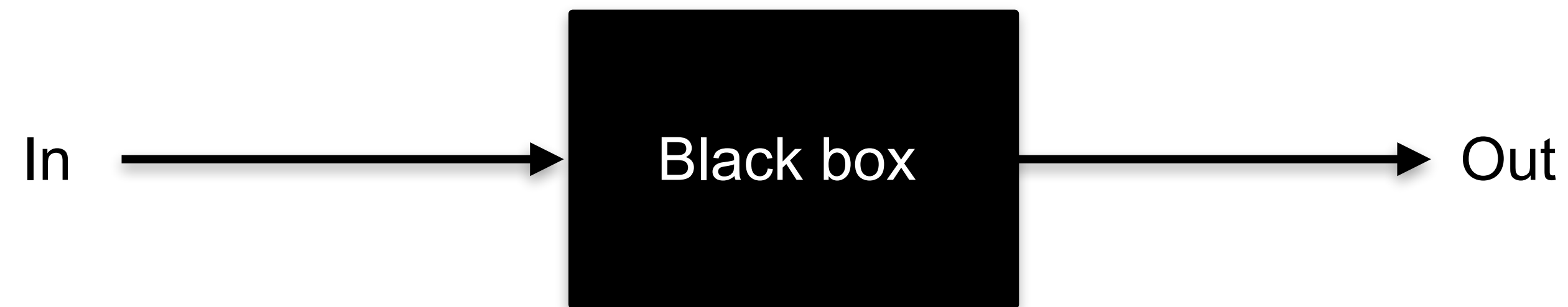
Universidade de São Paulo

francisco@icmc.usp.br

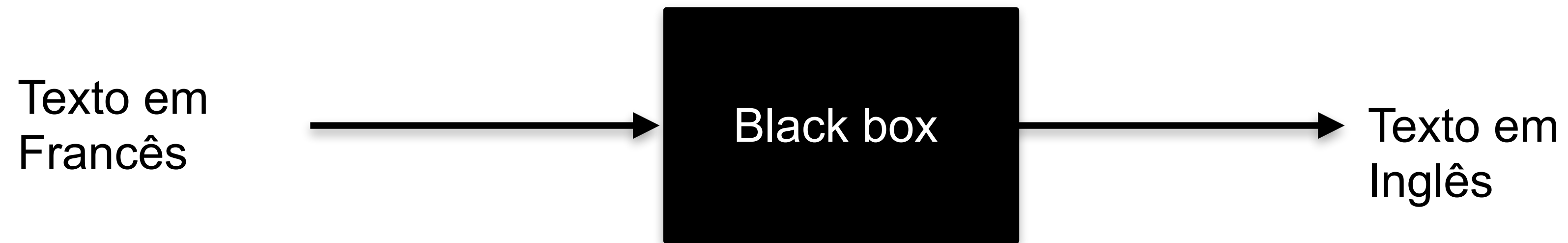
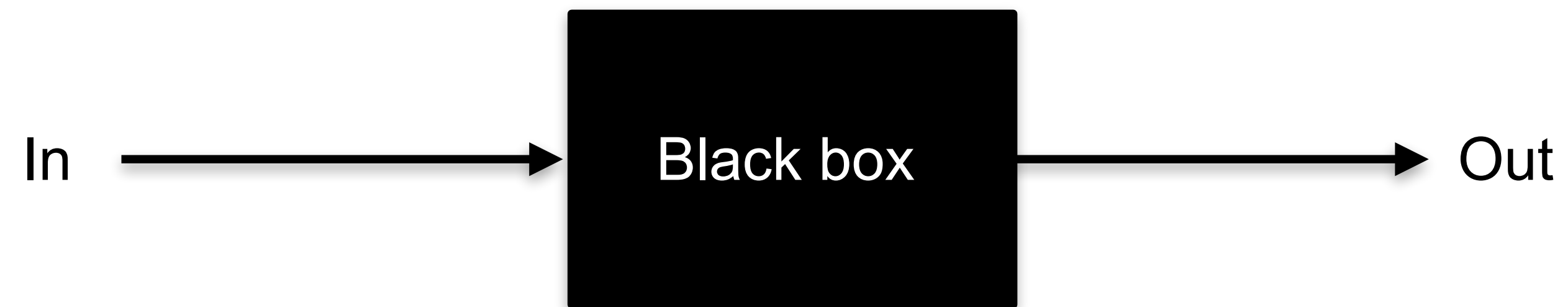
Redes neurais



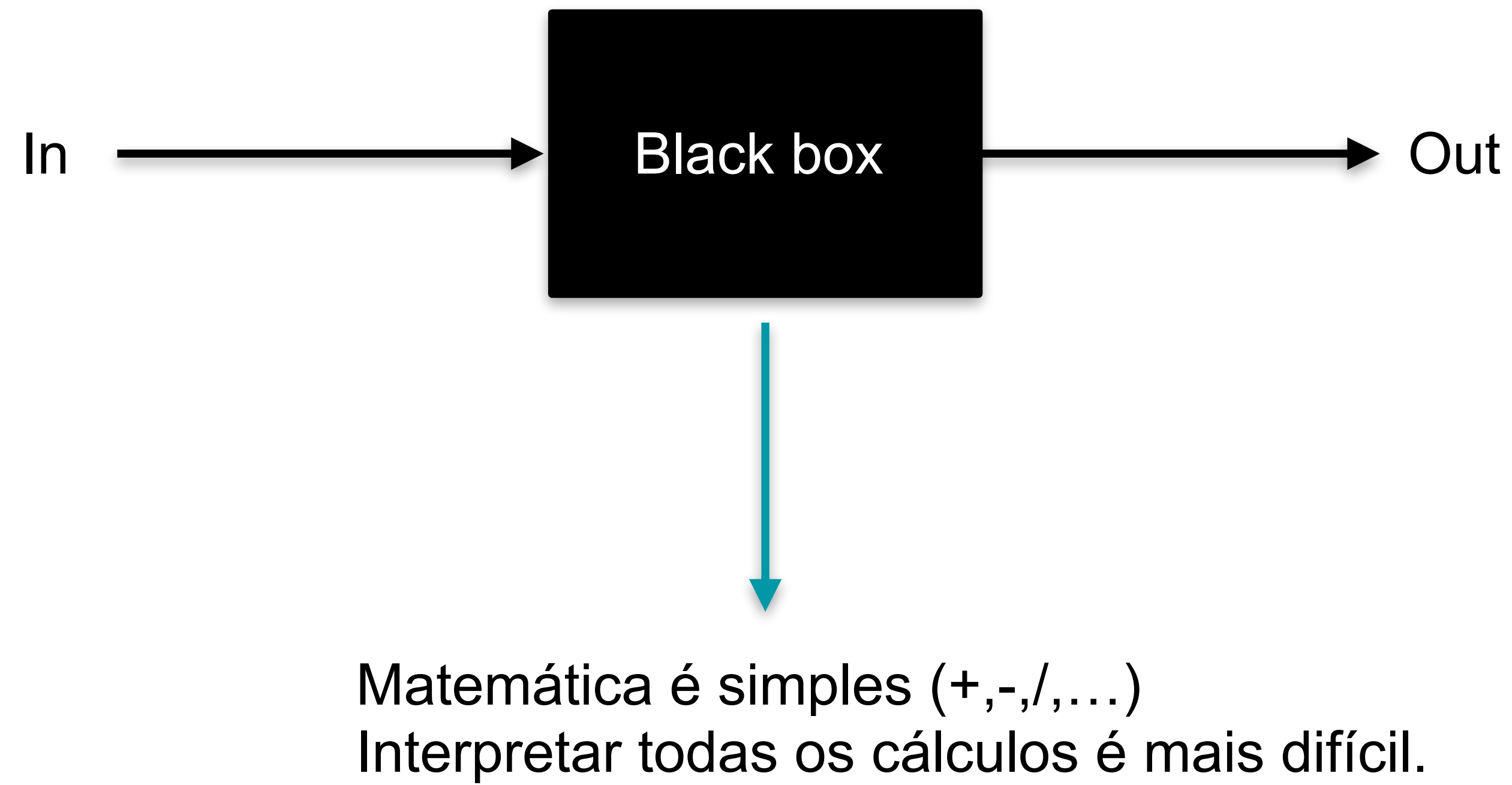
Redes neurais



Redes neurais

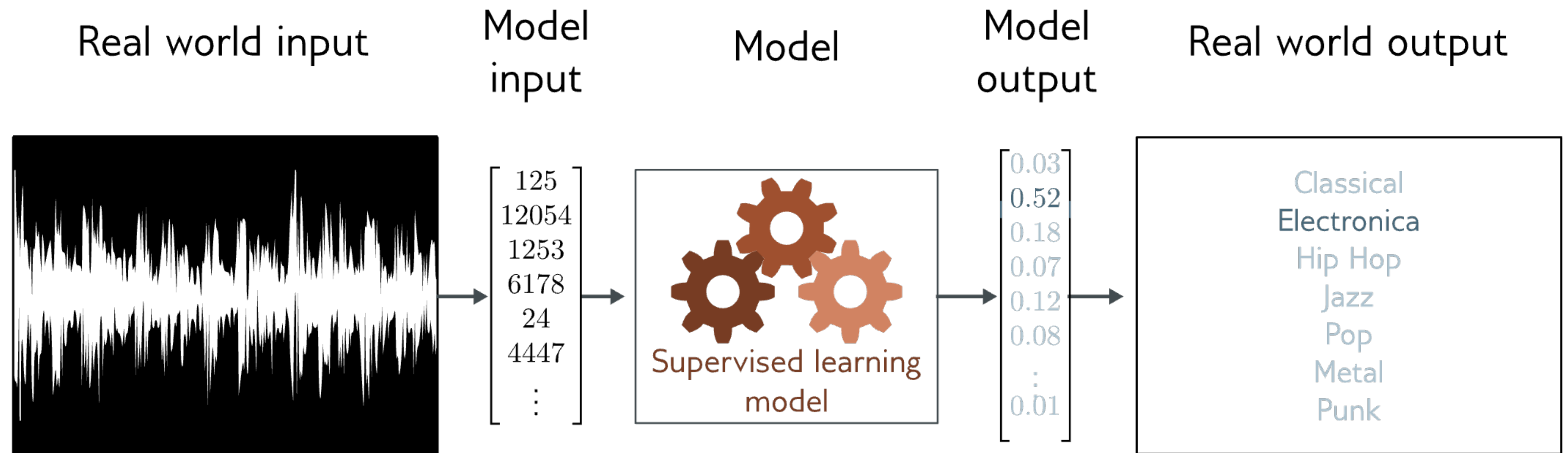


Redes neurais



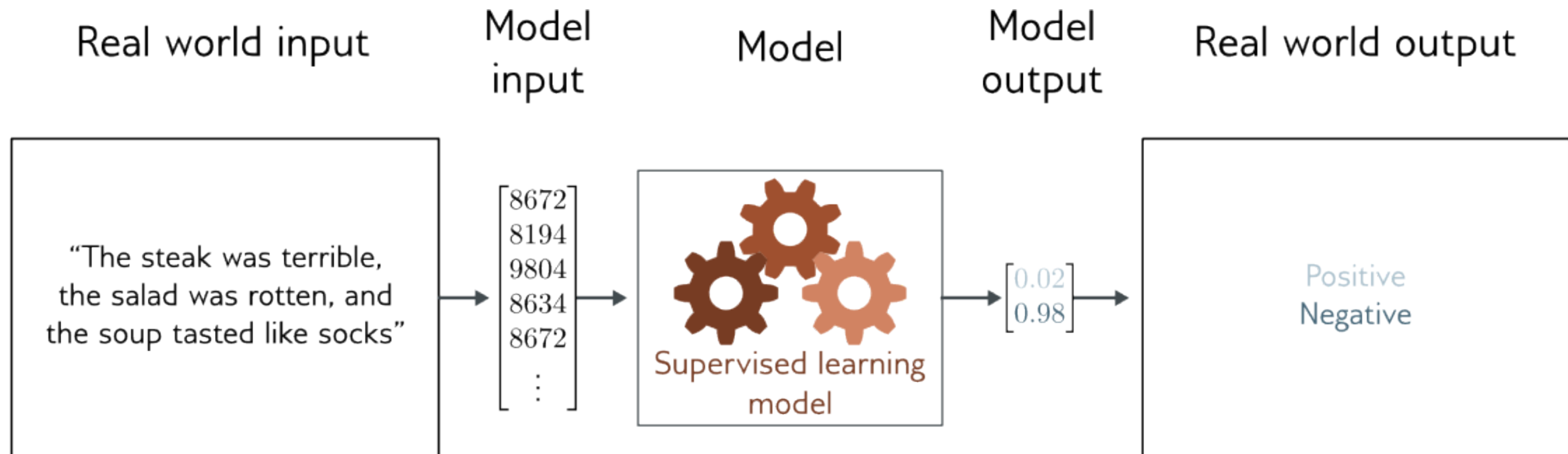
Redes neurais

Classificação



Redes neurais

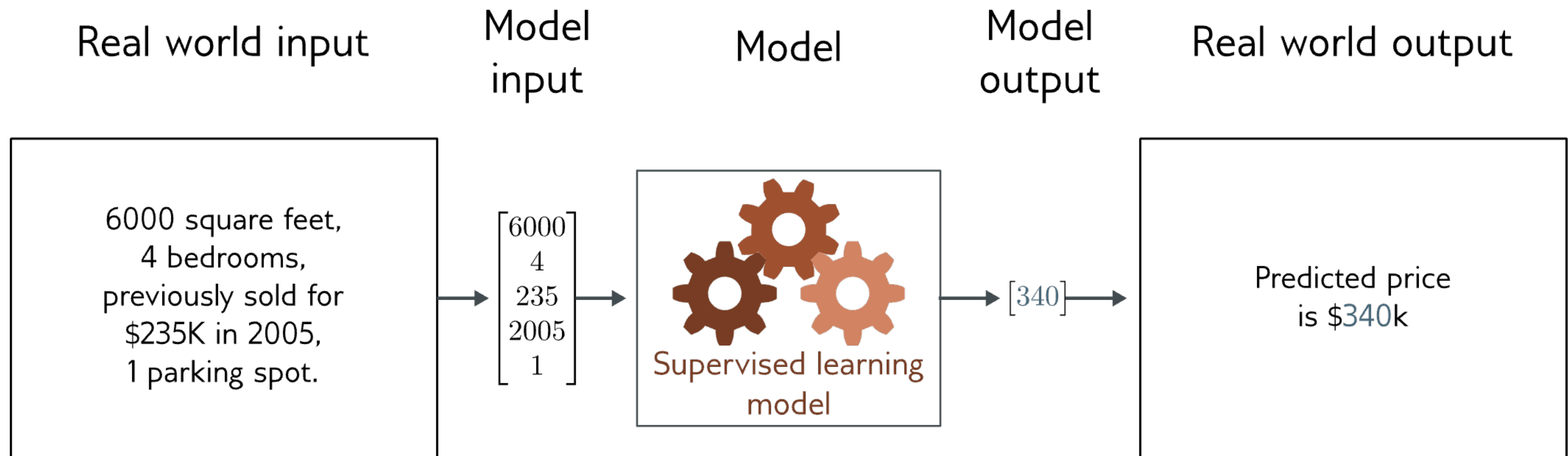
Classificação



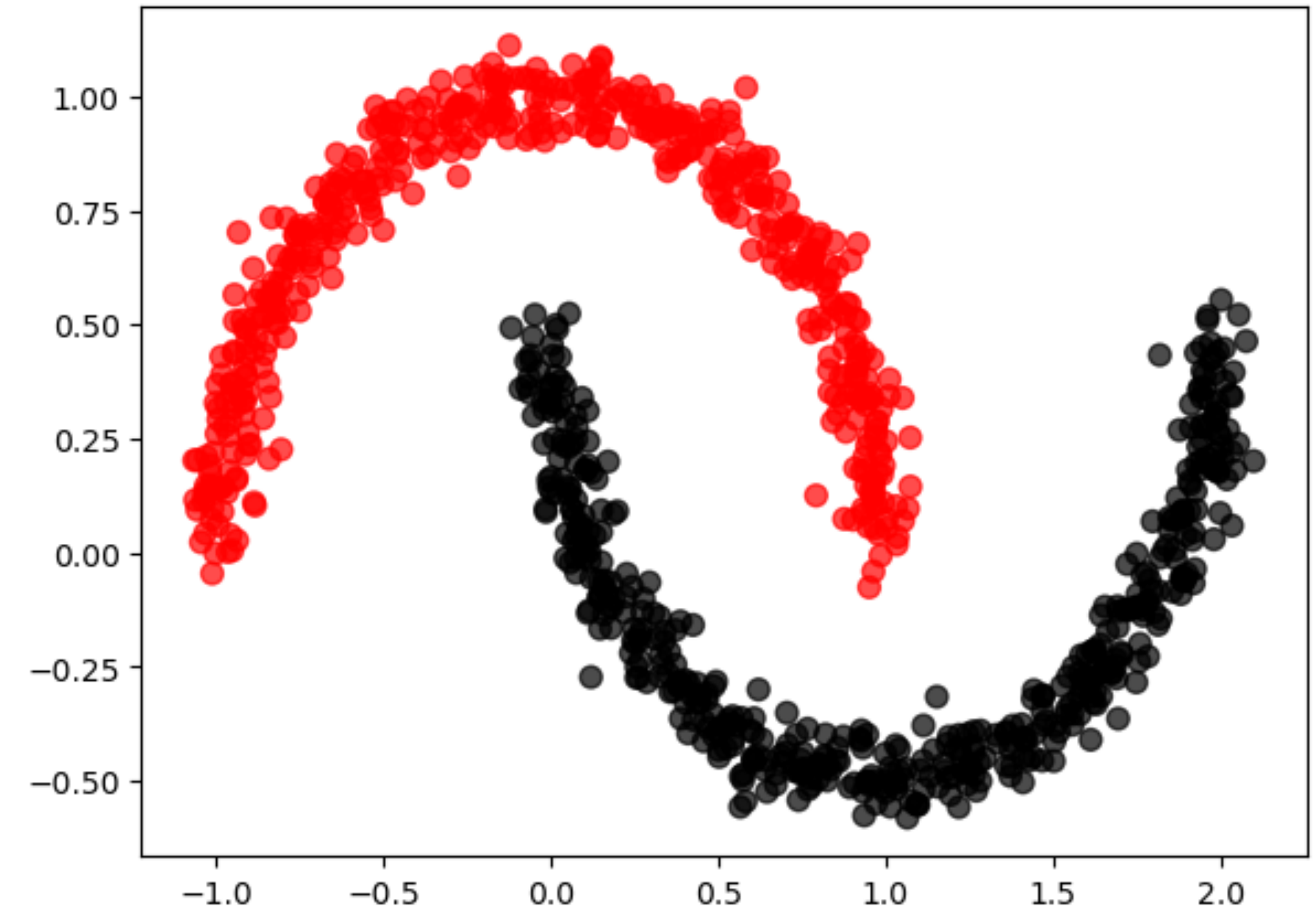
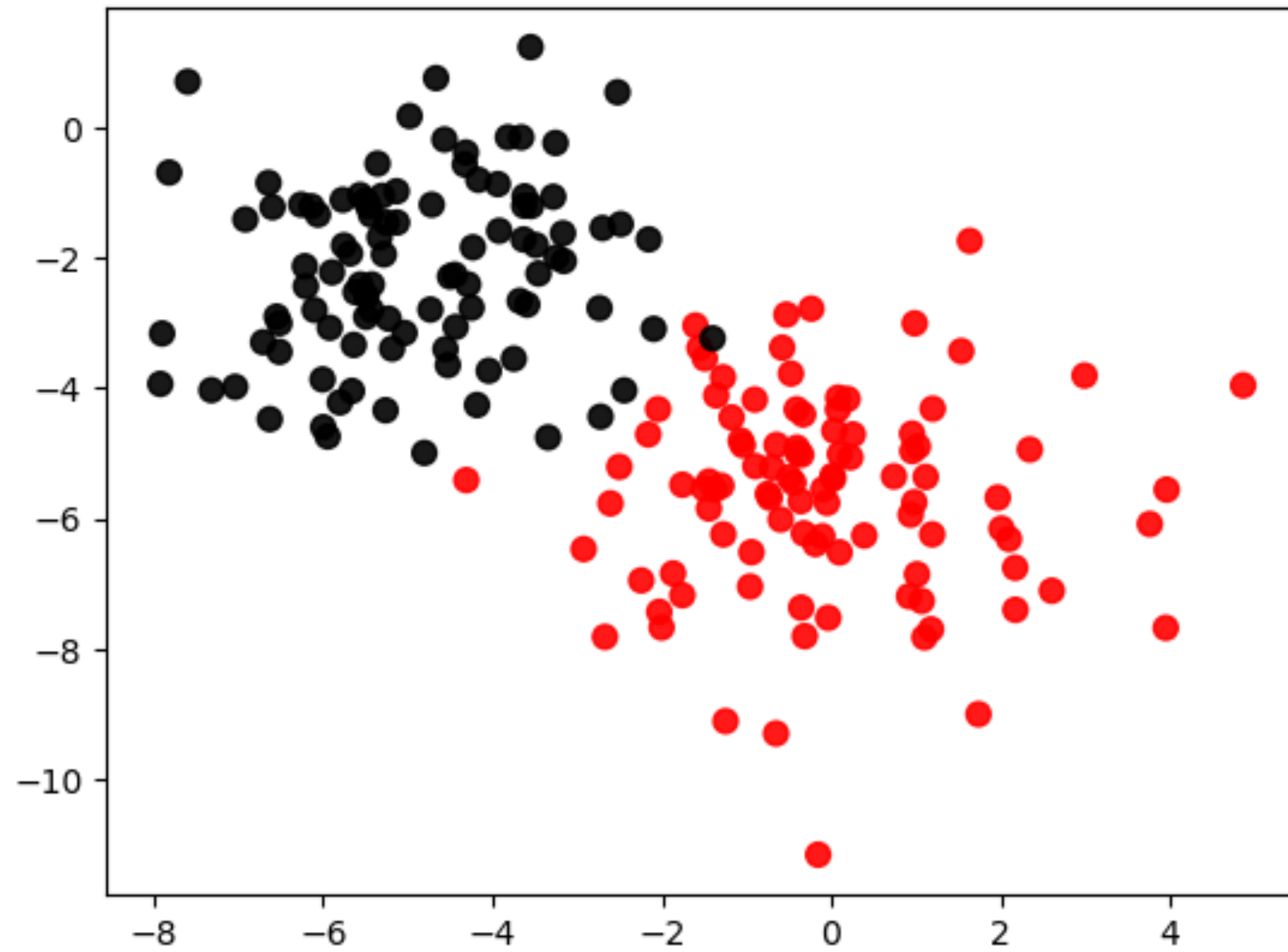
- Binary classification problem (two discrete classes)

Redes neurais

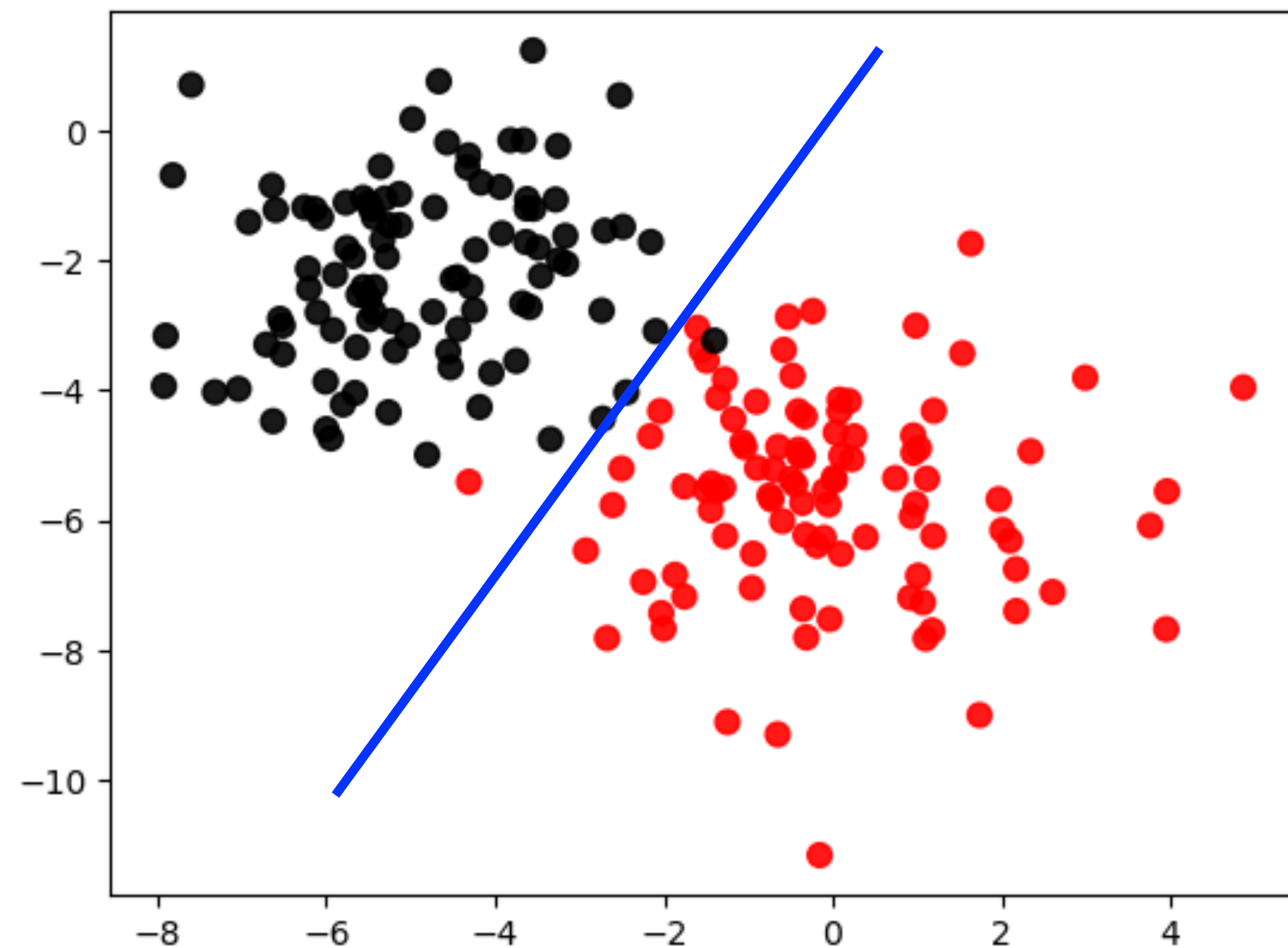
Regressão



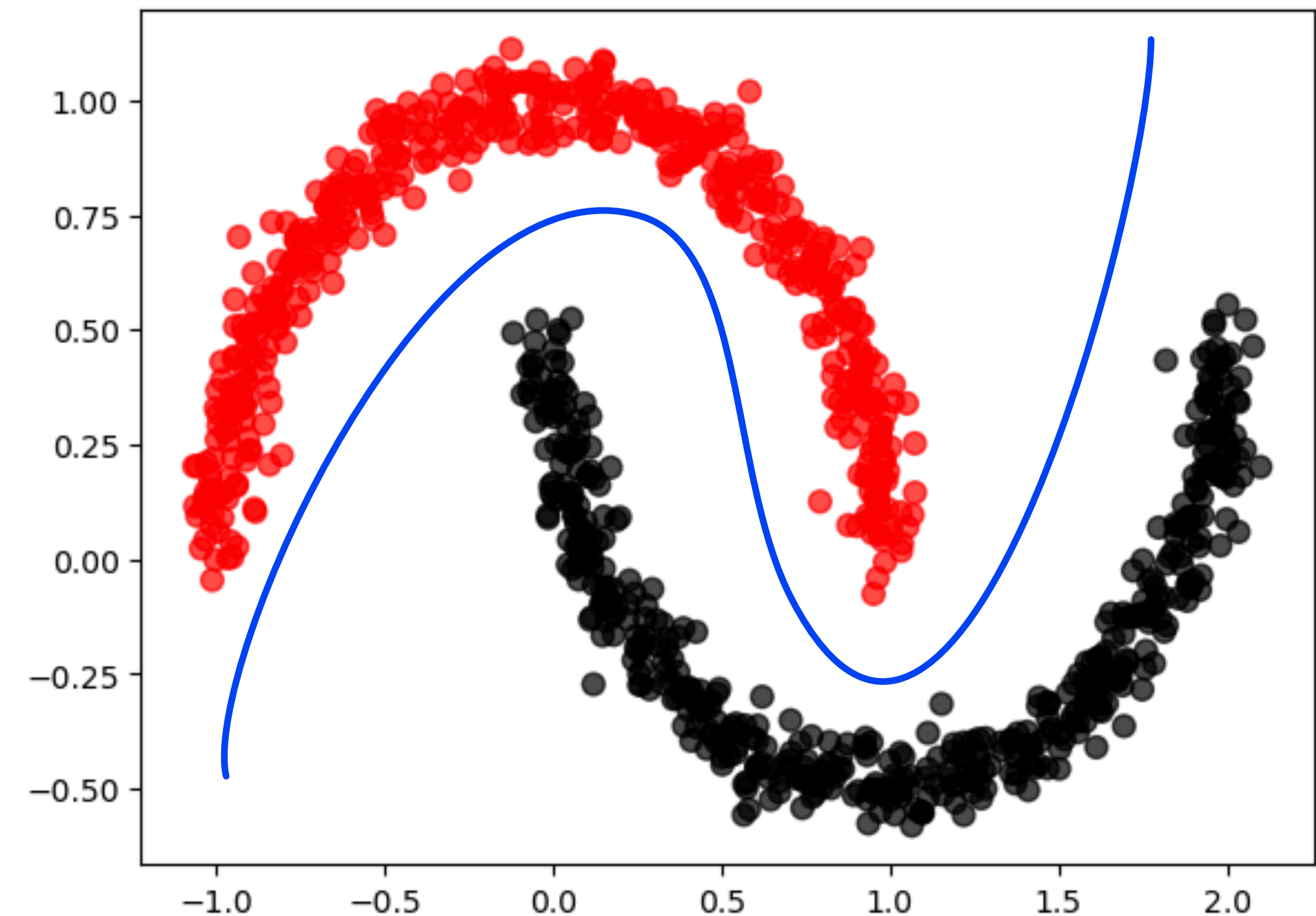
Redes neurais



Redes neurais

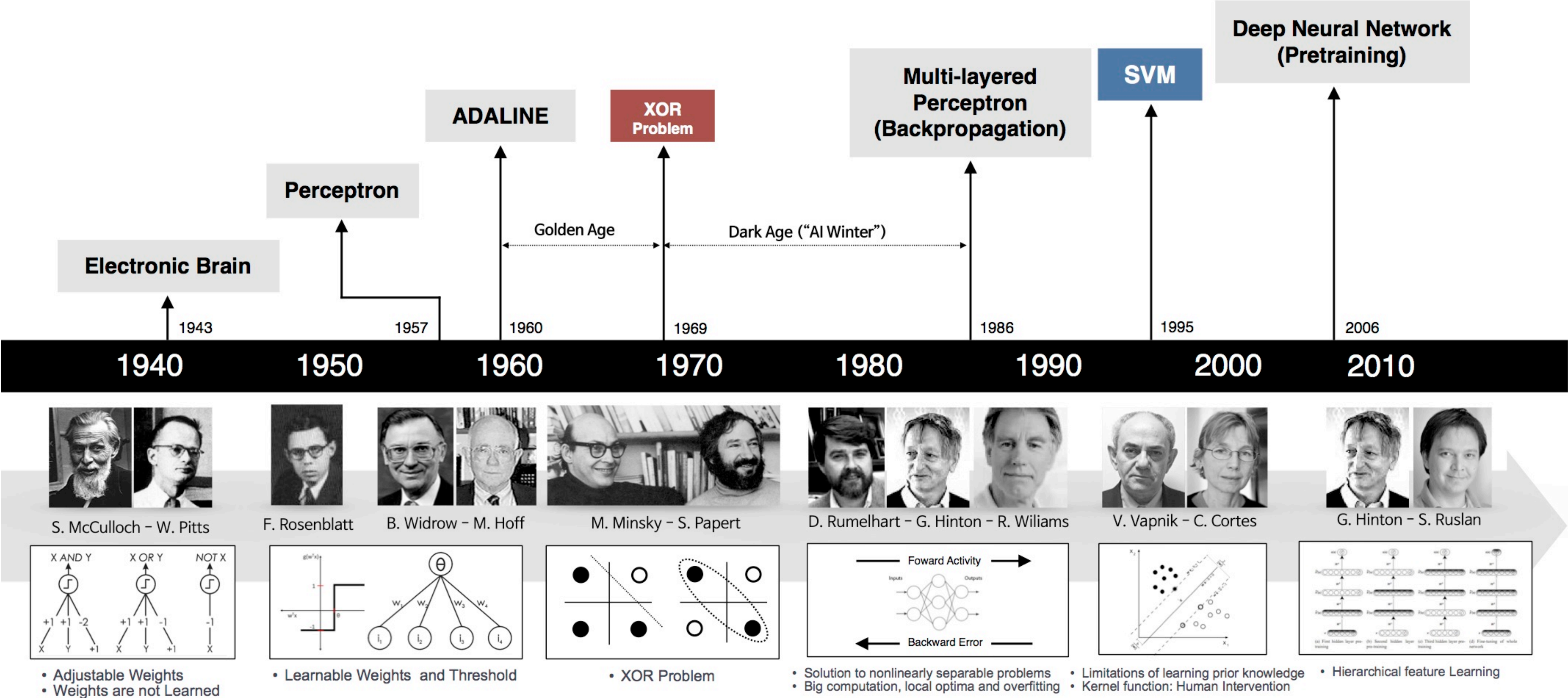


Solução linear

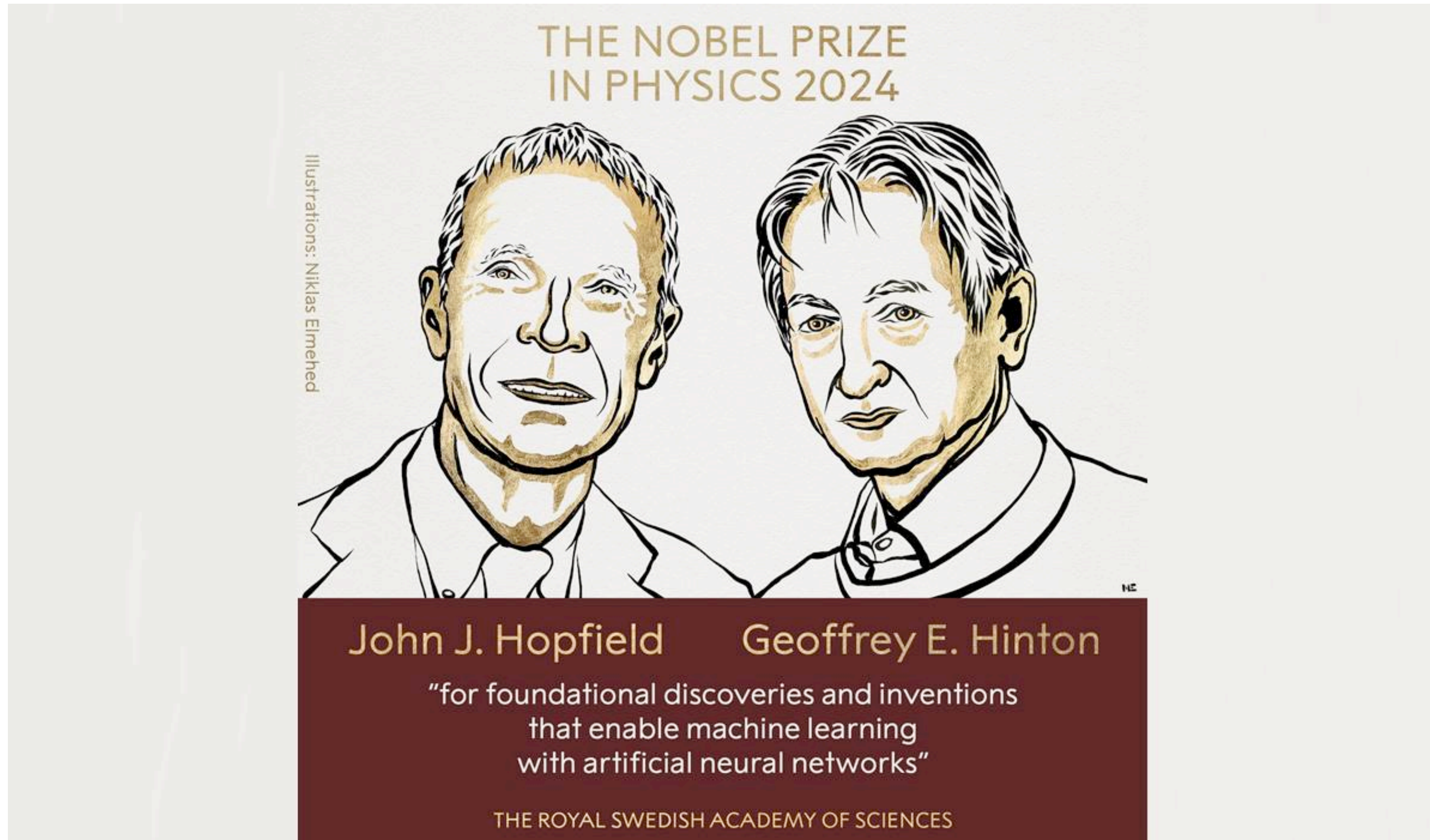


Solução não linear

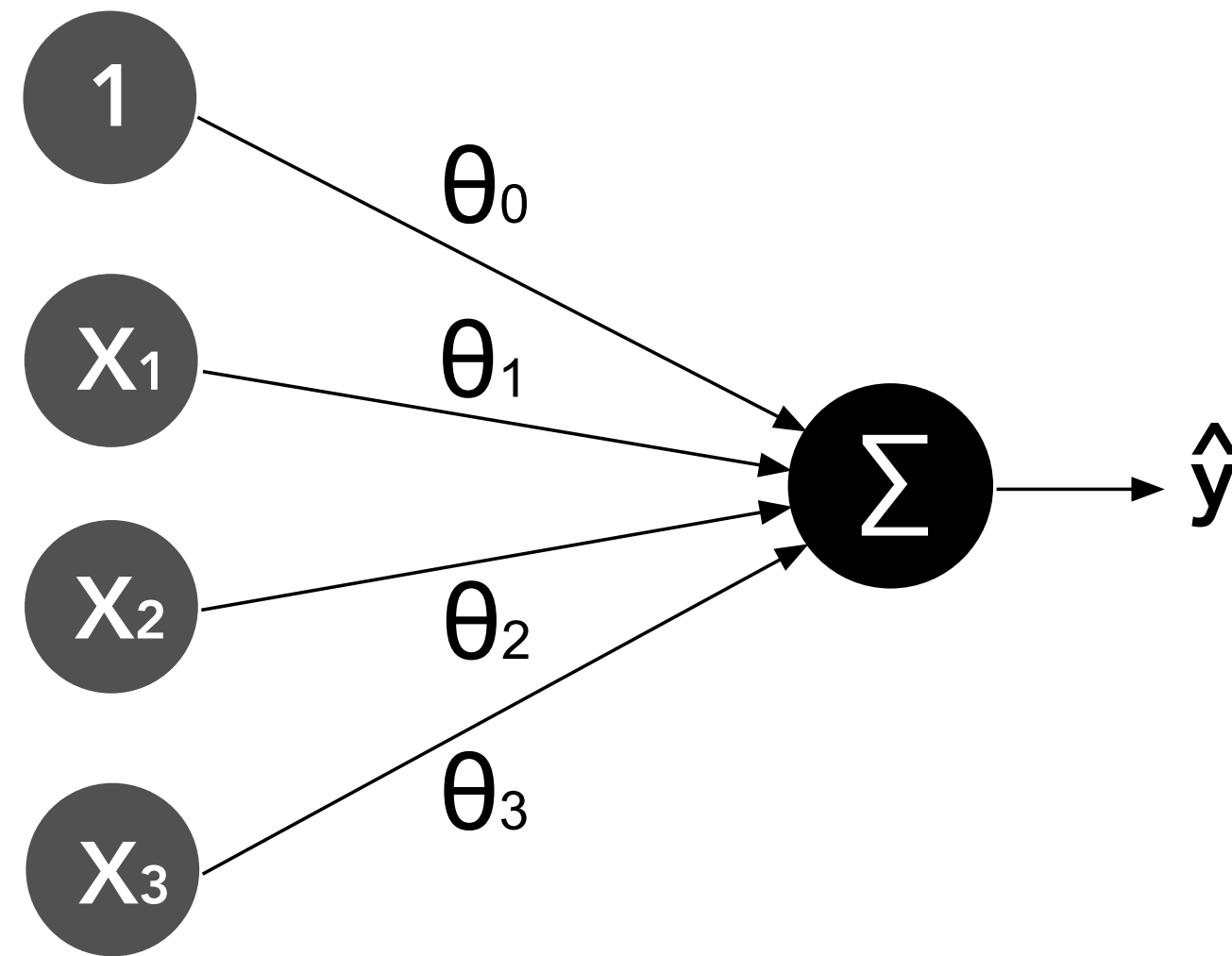
Redes neurais



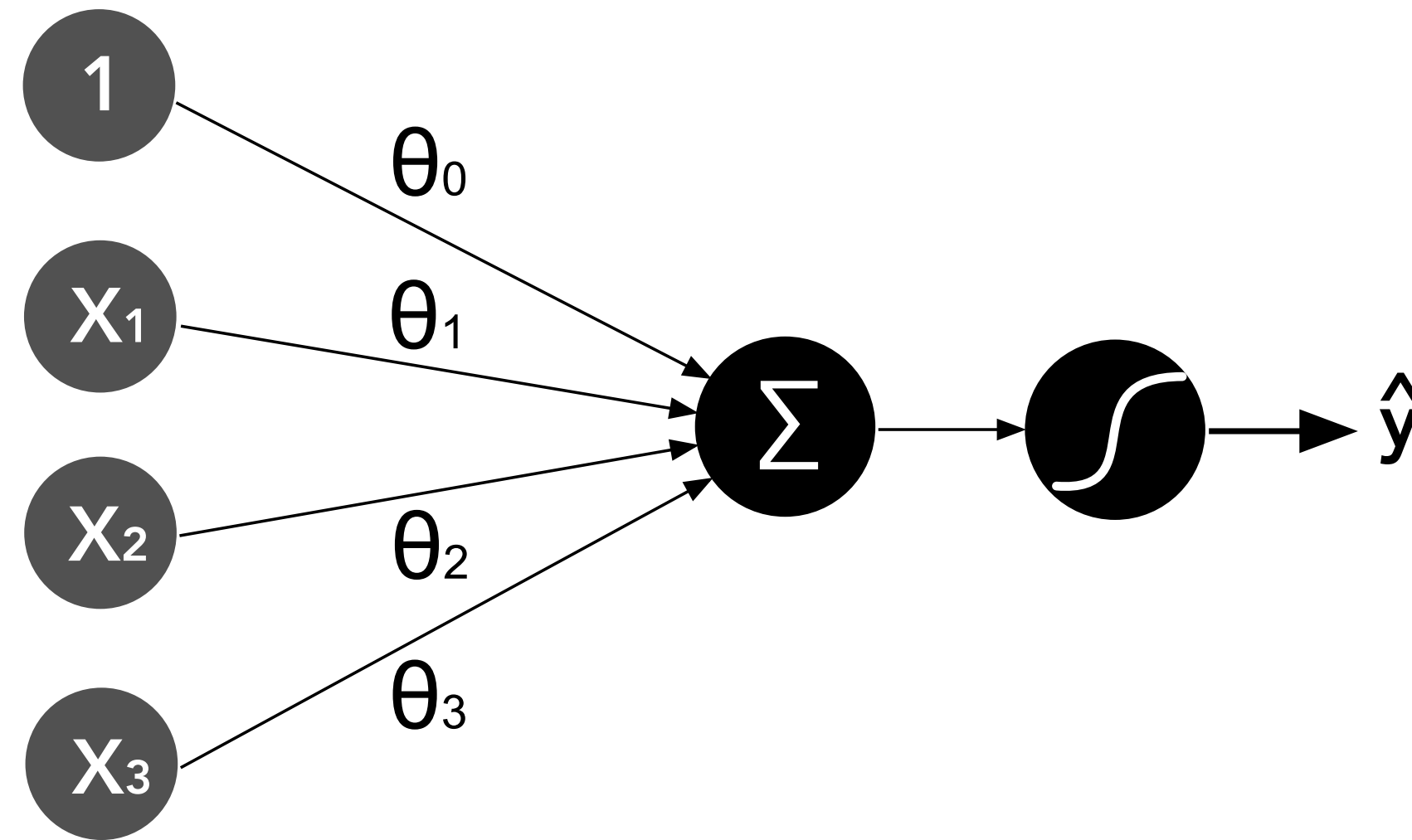
Redes neurais



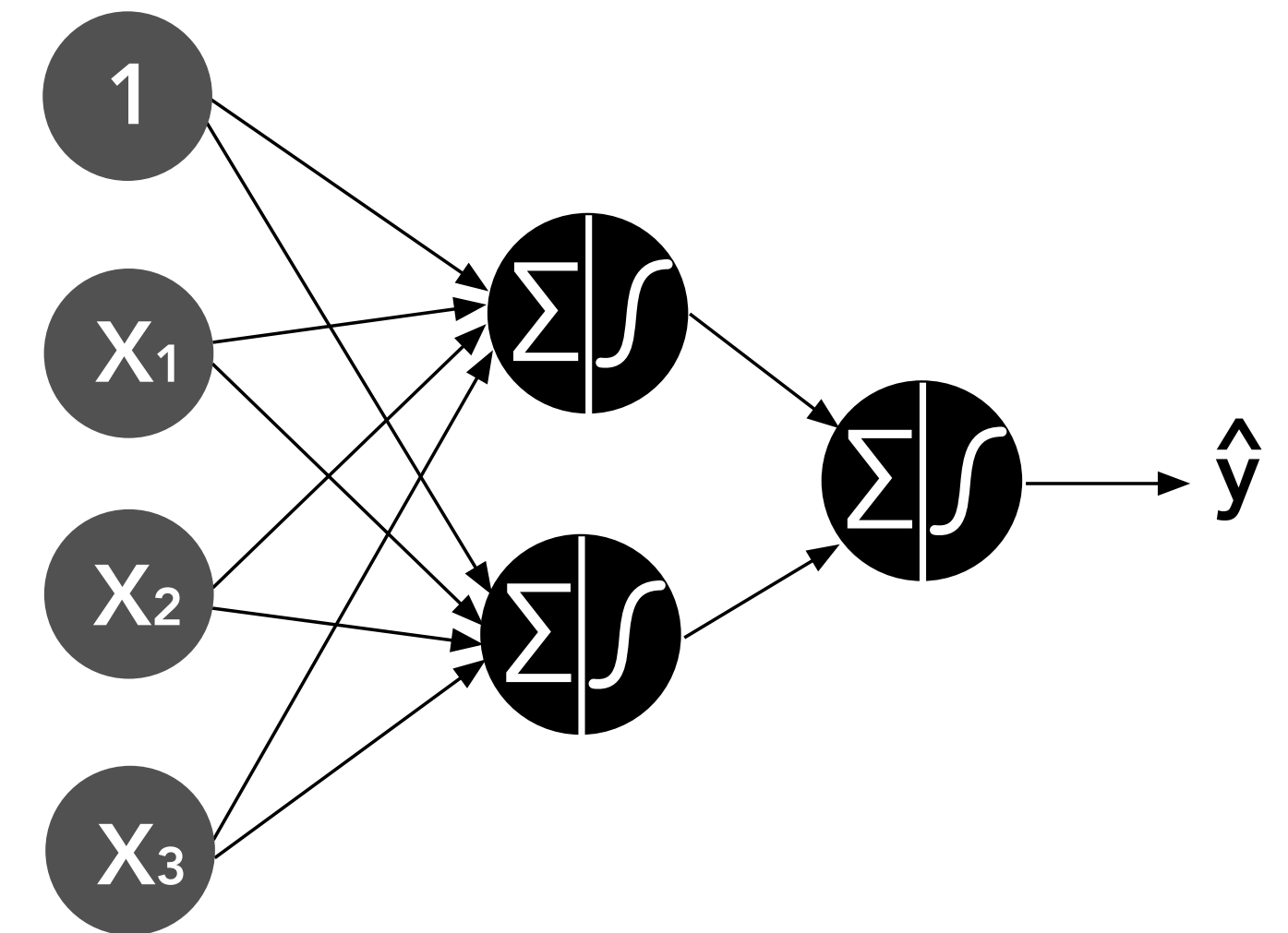
Redes neurais



Regressão Linear



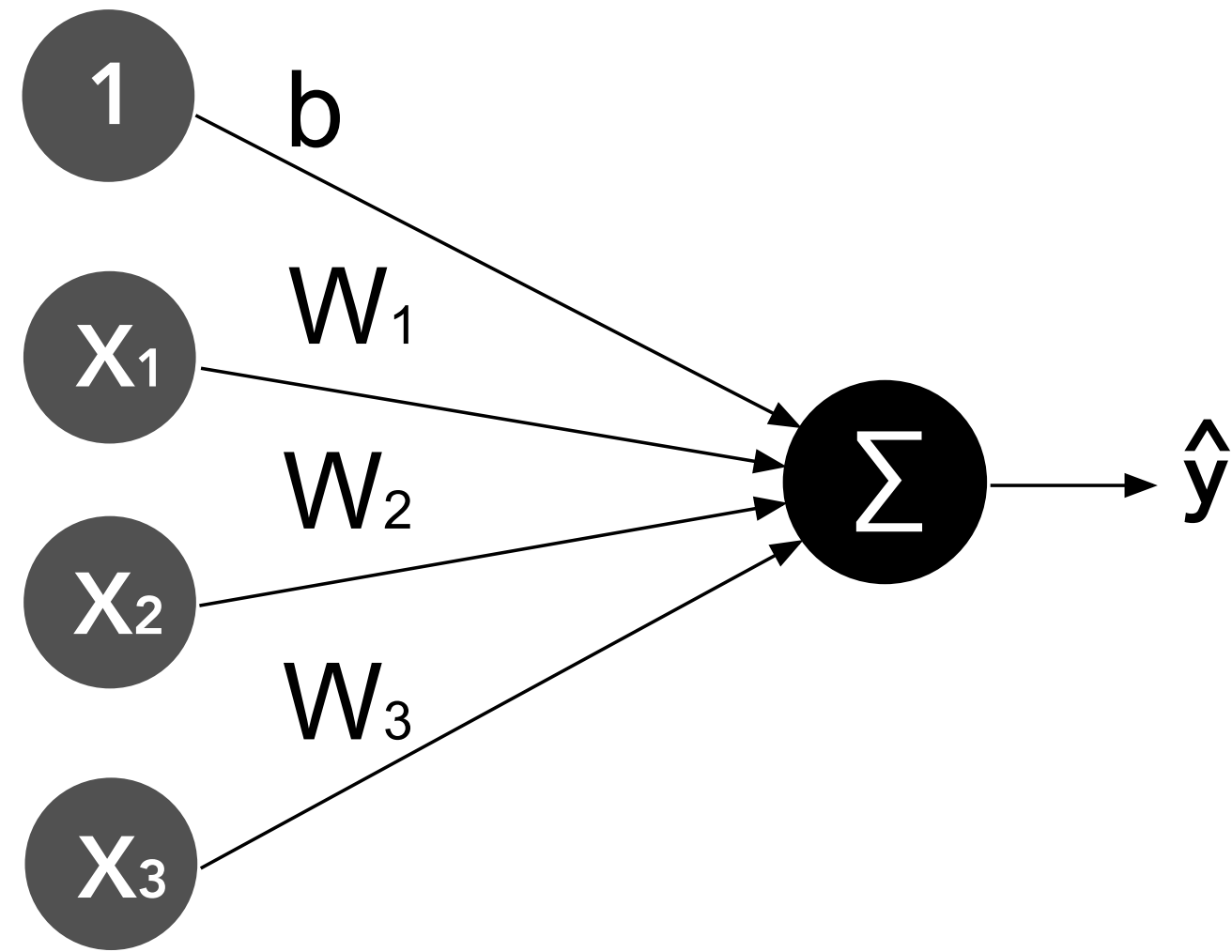
Regressão Logística



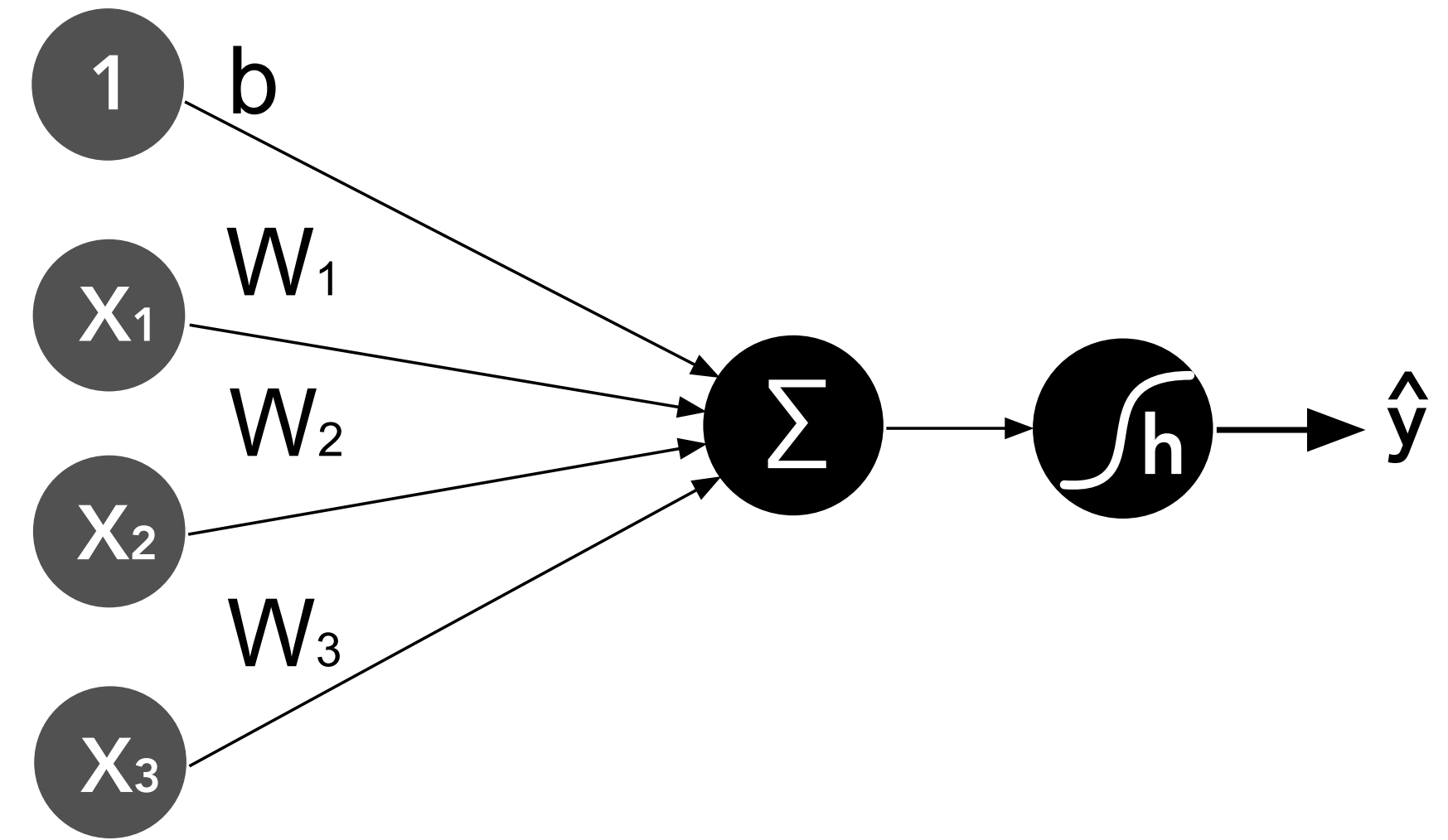
Rede Neural

Uma rede neural pode ser vista como uma série de classificadores de regressão logística empilhados uns sobre os outros.

Redes neurais



$$\hat{y} = W_1x_1 + W_2x_2 + \cdots + W_px_p + b.$$



$$\hat{y} = h(W_1x_1 + W_2x_2 + \cdots + W_px_p + b).$$

Regressão Logística

- Modelo de regressão linear:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_d X_d = \beta^T \mathbf{x}$$

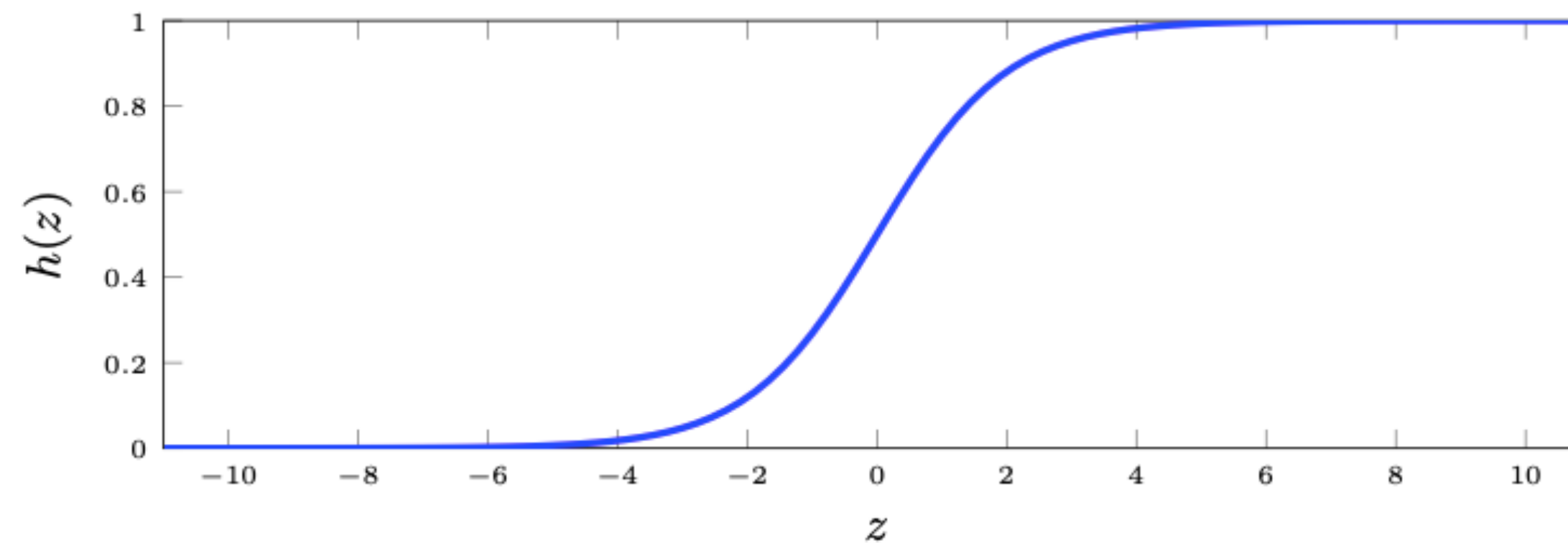
- onde $\mathbf{x}^T = [1, x_1, x_2, \dots, x_d]$
- Se considerarmos a saída y como um valor inteiro, podemos usar o modelo de regressão para realizarmos a classificação de dados.
- Vamos definir as probabilidade para o caso de duas classes:

$$p(y = 0 | \mathbf{X}) \text{ e } p(y = 1 | \mathbf{X})$$

Regressão Logística

- Vamos considerar a função logística:

$$h(z) = \frac{e^z}{1 + e^z}$$



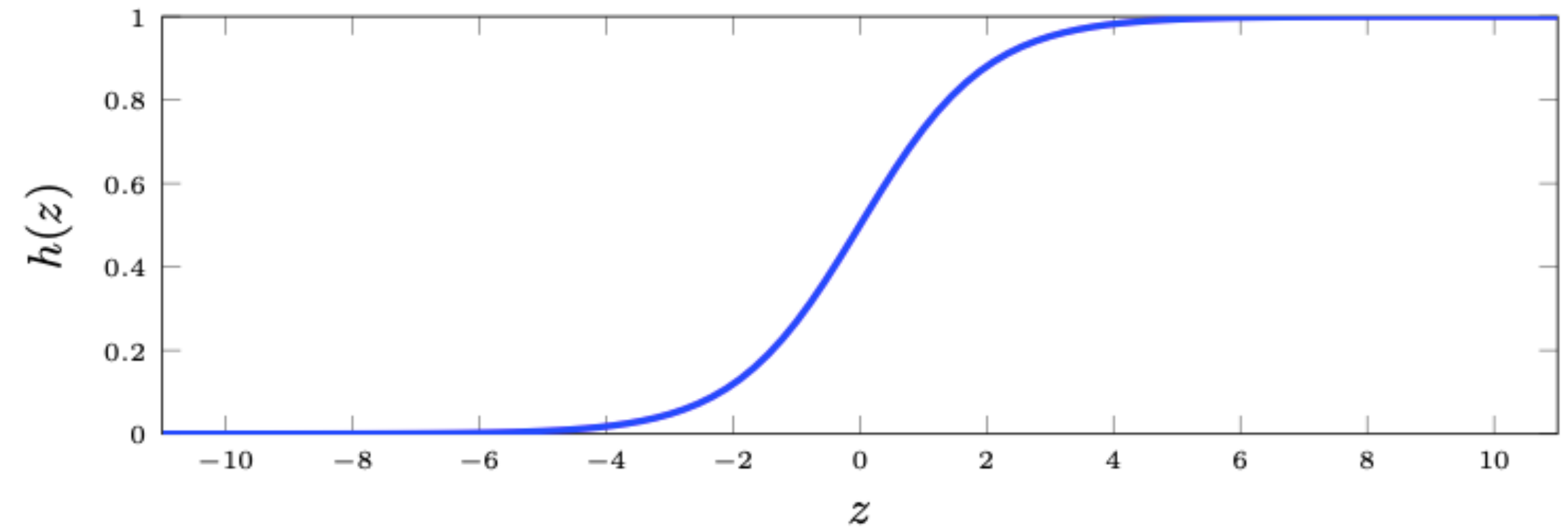
- Essa função retorna valores no intervalo $[0, 1]$.

Regressão Logística

- Usando a função logística:

$$p(y = 1 | \mathbf{x}) = \frac{e^{\beta^T \mathbf{x}}}{1 + e^{\beta^T \mathbf{x}}}$$

$$p(y = 0 | \mathbf{x}) = 1 - p(y = 1 | \mathbf{x}) = \frac{1}{1 + e^{\beta^T \mathbf{x}}}$$



- O aprendizado se resume a estimar o vetor de parâmetros β .
- Esse método é chamado regressão logística.

Regressão Logística

- Para estimar os parâmetros do modelo, usamos o conjunto de treinamento $T = \{(x_i, y_i)\}_{i=1}^N$.

- Usando estimação por máxima verossimilhança:

$$\hat{\beta} = \operatorname{argmax}_{\beta} p(\mathbf{y} | \mathbf{X}; \beta) .$$

- Assim, a máxima verossimilhança:

$$\ell(\beta) = p(\mathbf{y} | \mathbf{X}; \beta) = \prod_{i=1}^N p(y_i | x_i; \beta) = \prod_{i:y_i=1} p(y = 1 | \mathbf{x}_i; \beta) \prod_{i:y_i=0} p(y = 0 | \mathbf{x}_i; \beta)$$

$$\ell(\beta) = \prod_{i:y_i=1} \frac{e^{\beta^T \mathbf{x}_i}}{1 + e^{\beta^T \mathbf{x}_i}} \prod_{i:y_i=0} \frac{1}{1 + e^{\beta^T \mathbf{x}_i}}$$

Regressão Logística

- Para estimar os parâmetros, precisamos otimizar essa função em relação à beta.

$$\ell(\beta) = \prod_{i:y_i=1} \frac{e^{\beta^T \mathbf{x}_i}}{1 + e^{\beta^T \mathbf{x}_i}} \prod_{i:y_i=0} \frac{1}{1 + e^{\beta^T \mathbf{x}_i}}$$

- Para facilitar os cálculos, podemos a função logaritmo, que é uma função monotônica e logo, o máximo é o mesmo que o da verossimilhança:

$$\log \ell(\beta) = \sum_{i:y_i=1} \left(\beta^T \mathbf{x}_i - \log(1 + e^{\beta^T \mathbf{x}_i}) \right) - \sum_{i:y_i=0} \log(1 + e^{\beta^T \mathbf{x}_i})$$

Regressão Logística

- Calculando a derivada e igualando a zero:

$$\nabla_{\beta} \log \ell(\beta) = \sum_{i=1}^n \mathbf{x}_i \left(y_i - \frac{e^{\beta^T \mathbf{x}_i}}{1 + e^{\beta^T \mathbf{x}_i}} \right) = 0$$

- Como essa equação é não-linear, devemos resolvê-la numericamente, por exemplo, usando o método de Newton–Raphson.
- Com isso, determinamos os parâmetros do modelo e realizamos a classificação em uma das duas classes.

Regressão Logística

- A superfície de decisão pode ser calculada usando:

$$p(y = 1 | \mathbf{x}) = p(y = 0 | \mathbf{x})$$

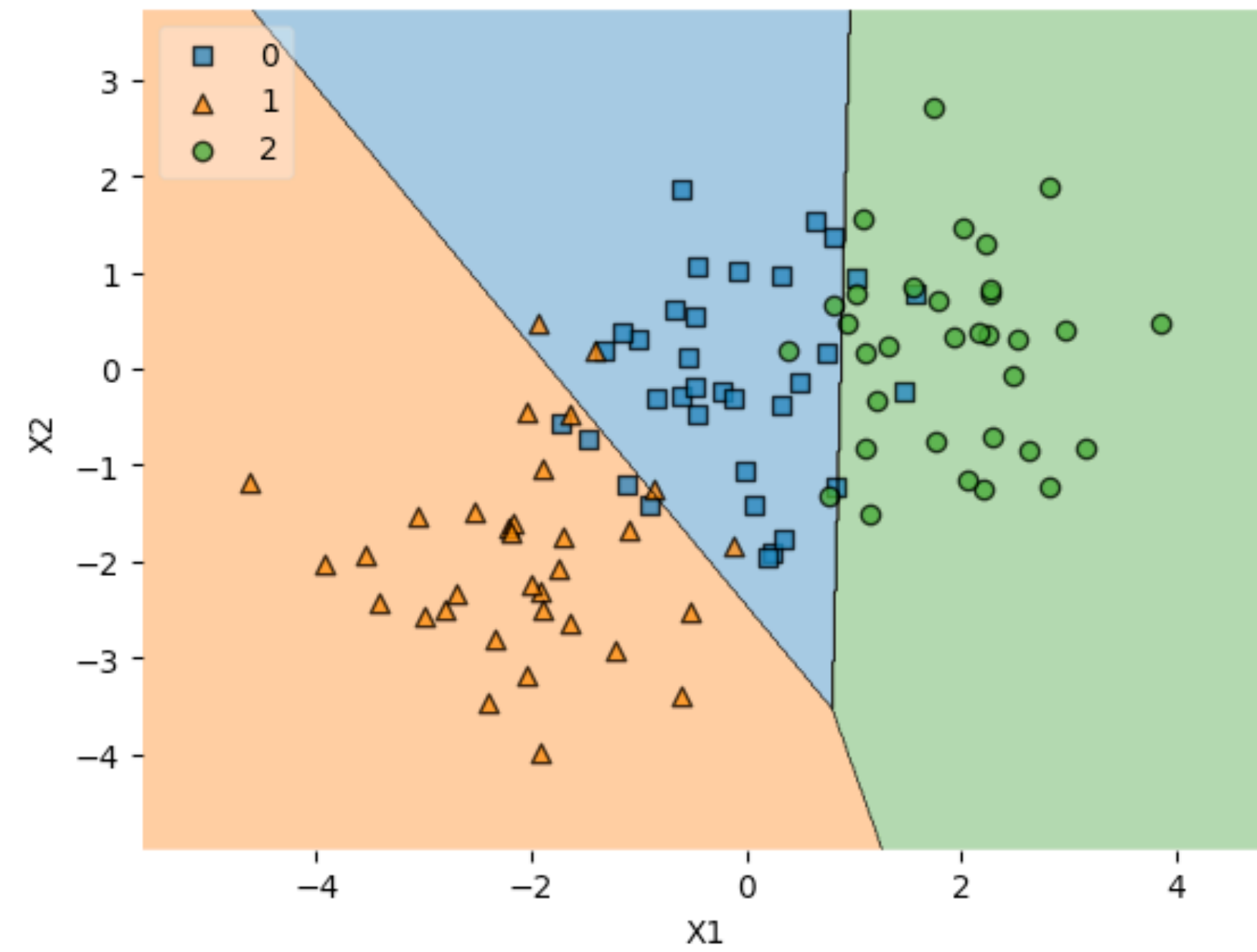
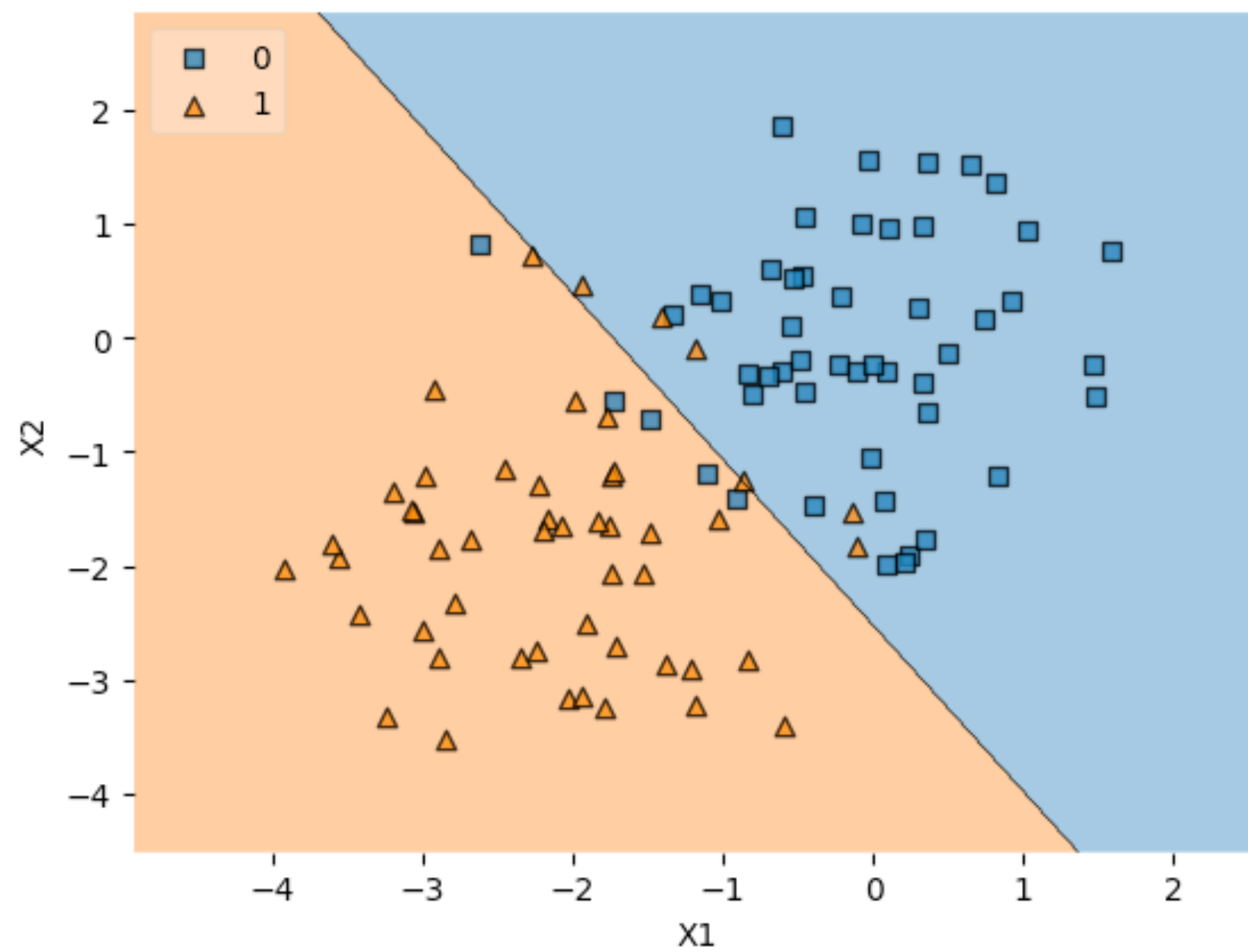
$$\frac{e^{\beta^T \mathbf{x}}}{1 + e^{\beta^T \mathbf{x}}} = \frac{1}{1 + e^{\beta^T \mathbf{x}}}$$

- Ou seja, basta resolvermos:

$$\beta^T \mathbf{x} = 0$$

- A solução são hiperplanos. Logo, a superfície de separação são hiperplanos (lineares).

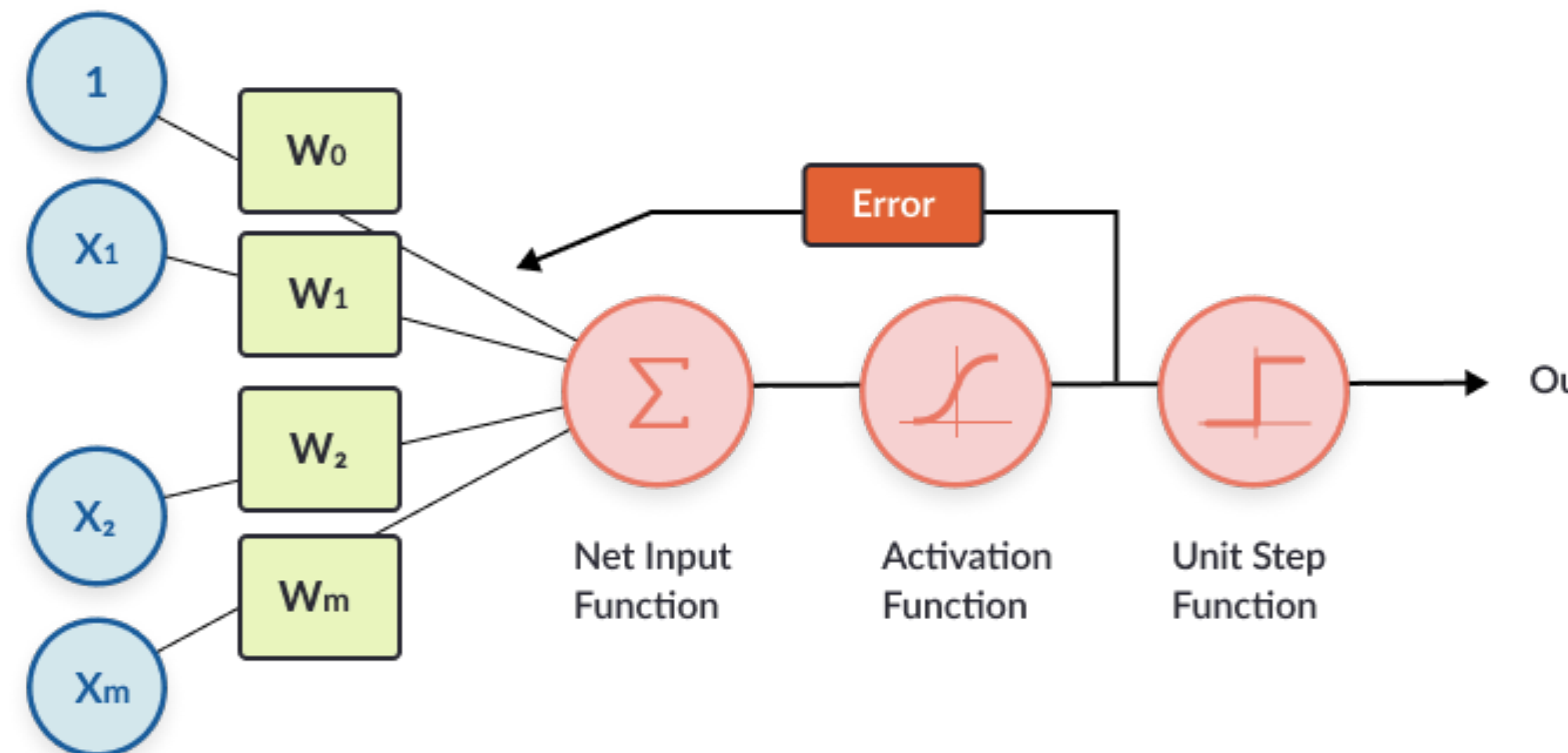
Regressão Logística



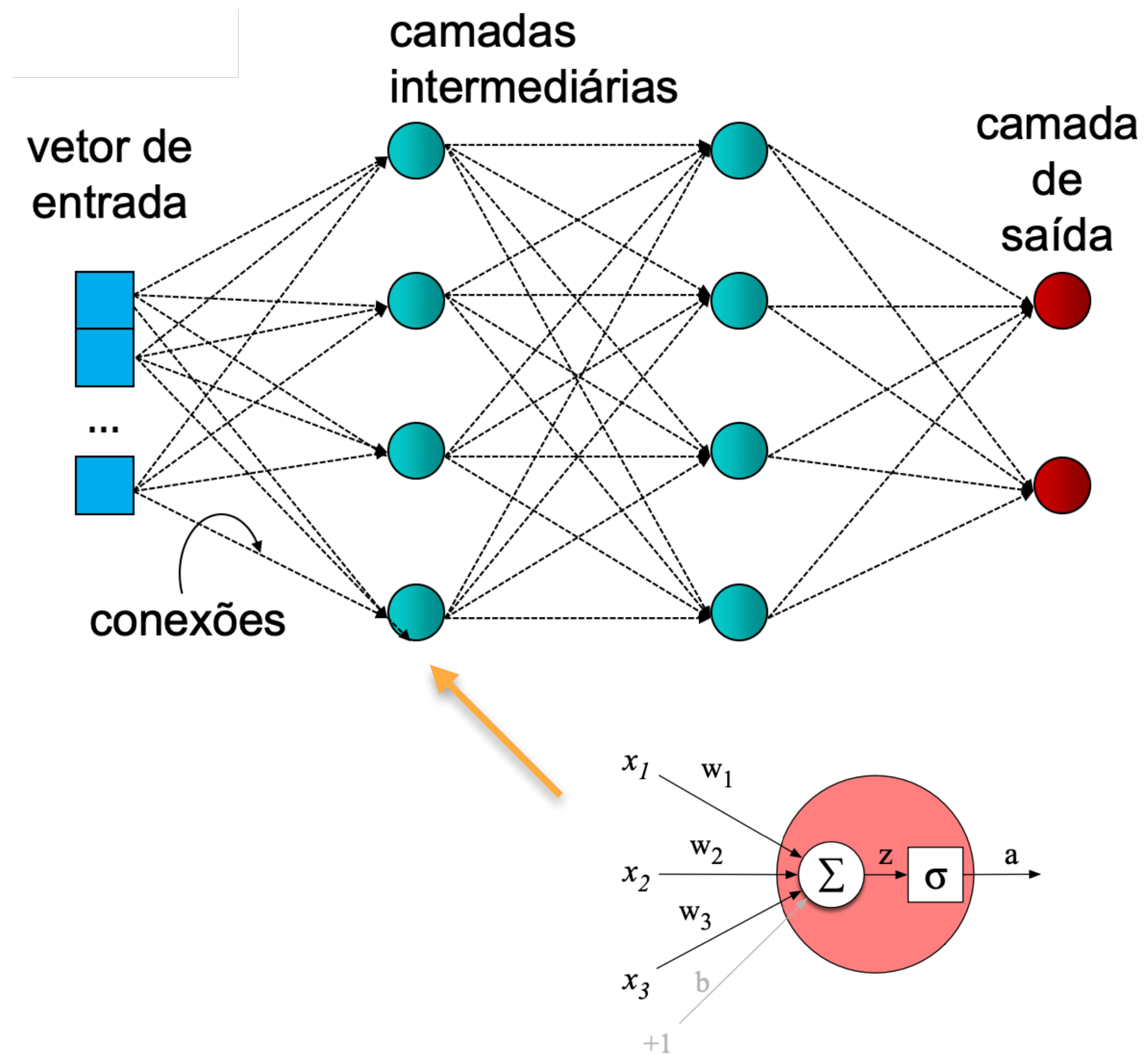
Regressão Logística

$$Z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_d X_d = \beta^T \mathbf{x}$$

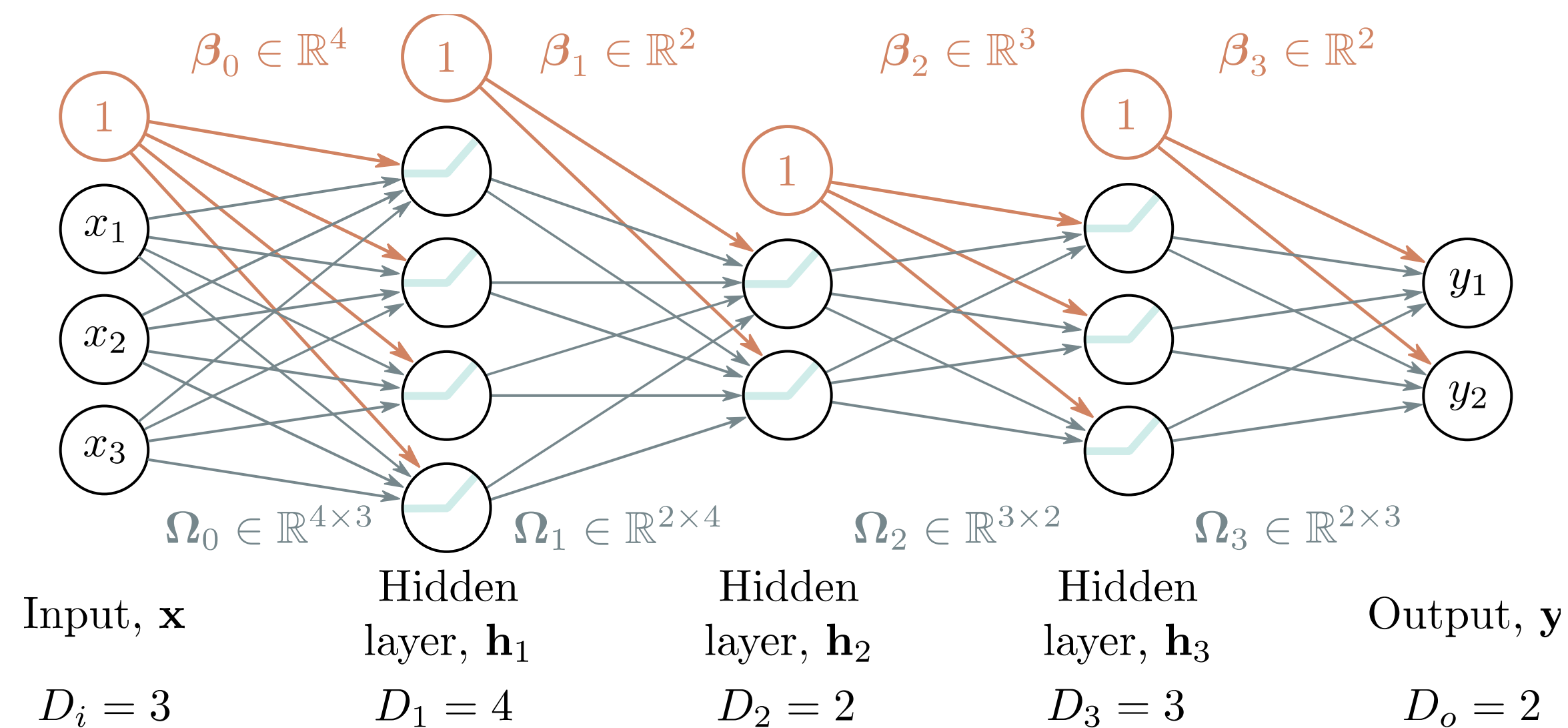
$$p(y = 0 | \mathbf{x}) = \frac{1}{1 + e^{\beta^T \mathbf{x}}} \quad p(y = 1 | \mathbf{x}) = \frac{e^{\beta^T \mathbf{x}}}{1 + e^{\beta^T \mathbf{x}}}$$



Redes neurais



Redes neurais

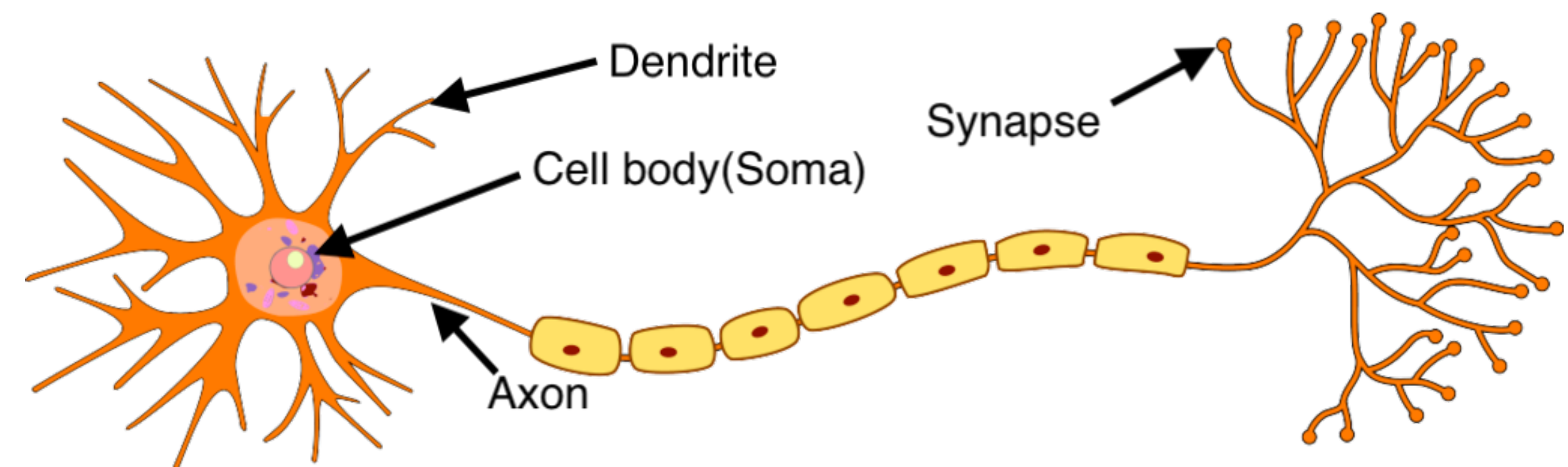
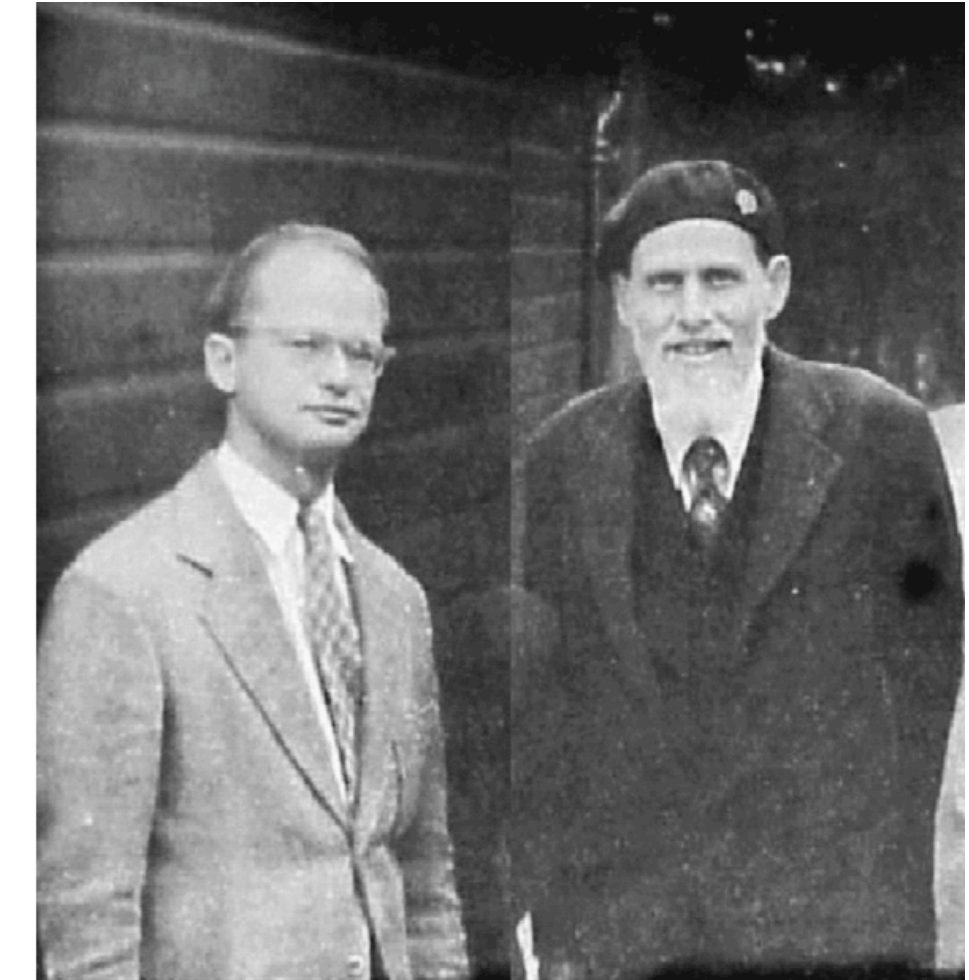
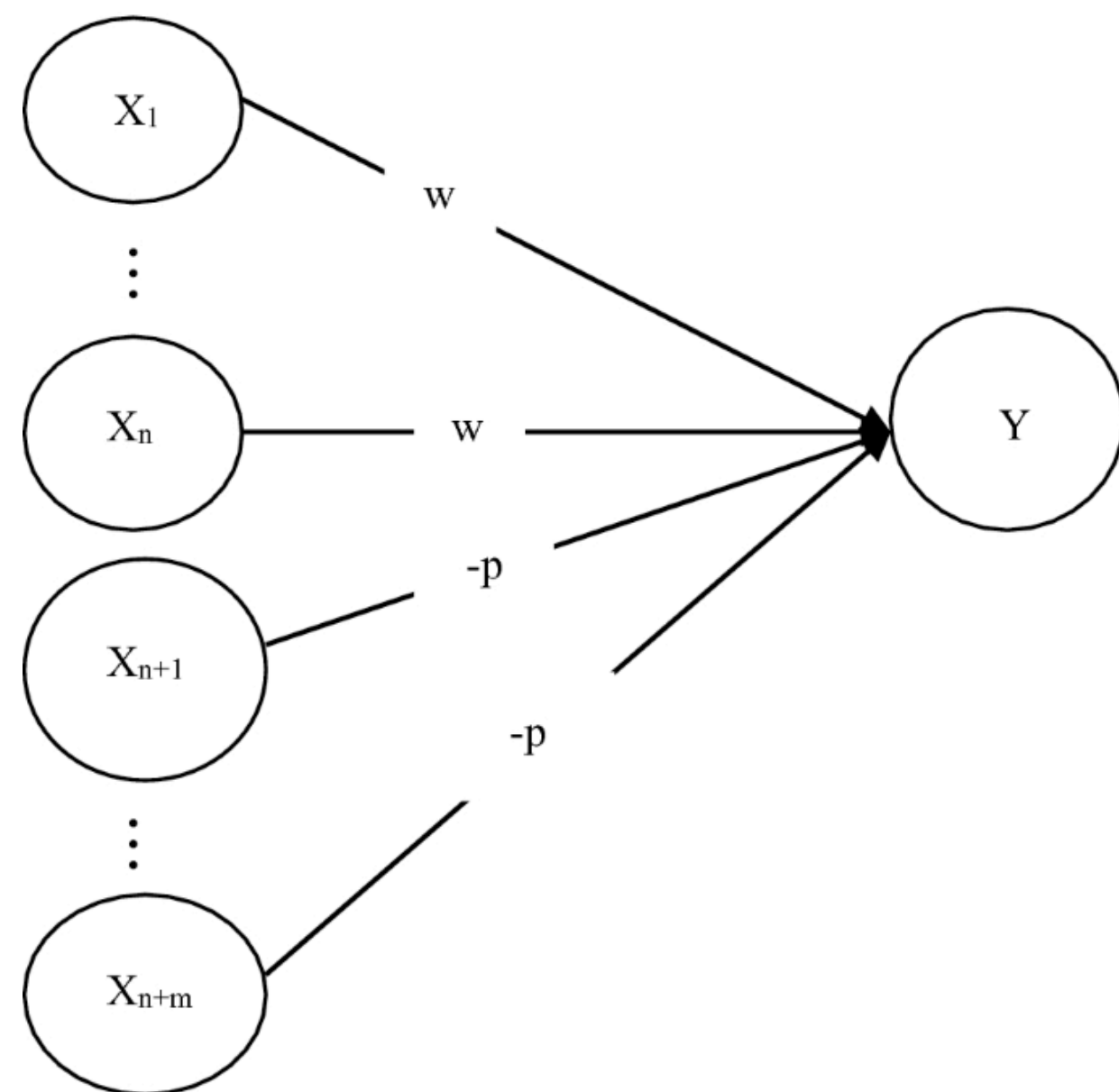


$$y' = \phi'_0 + \phi'_1 a[\psi_{10} + \psi_{11} a[\theta_{10} + \theta_{11} x] + \psi_{12} a[\theta_{20} + \theta_{21} x] + \psi_{13} a[\theta_{30} + \theta_{31} x]] \\ + \phi'_2 a[\psi_{20} + \psi_{21} a[\theta_{10} + \theta_{11} x] + \psi_{22} a[\theta_{20} + \theta_{21} x] + \psi_{23} a[\theta_{30} + \theta_{31} x]] \\ + \phi'_3 a[\psi_{30} + \psi_{31} a[\theta_{10} + \theta_{11} x] + \psi_{32} a[\theta_{20} + \theta_{21} x] + \psi_{33} a[\theta_{30} + \theta_{31} x]]$$

- Uma rede neural é, essencialmente, apenas uma equação.
- No entanto, é uma equação gigantesca!

Redes neurais

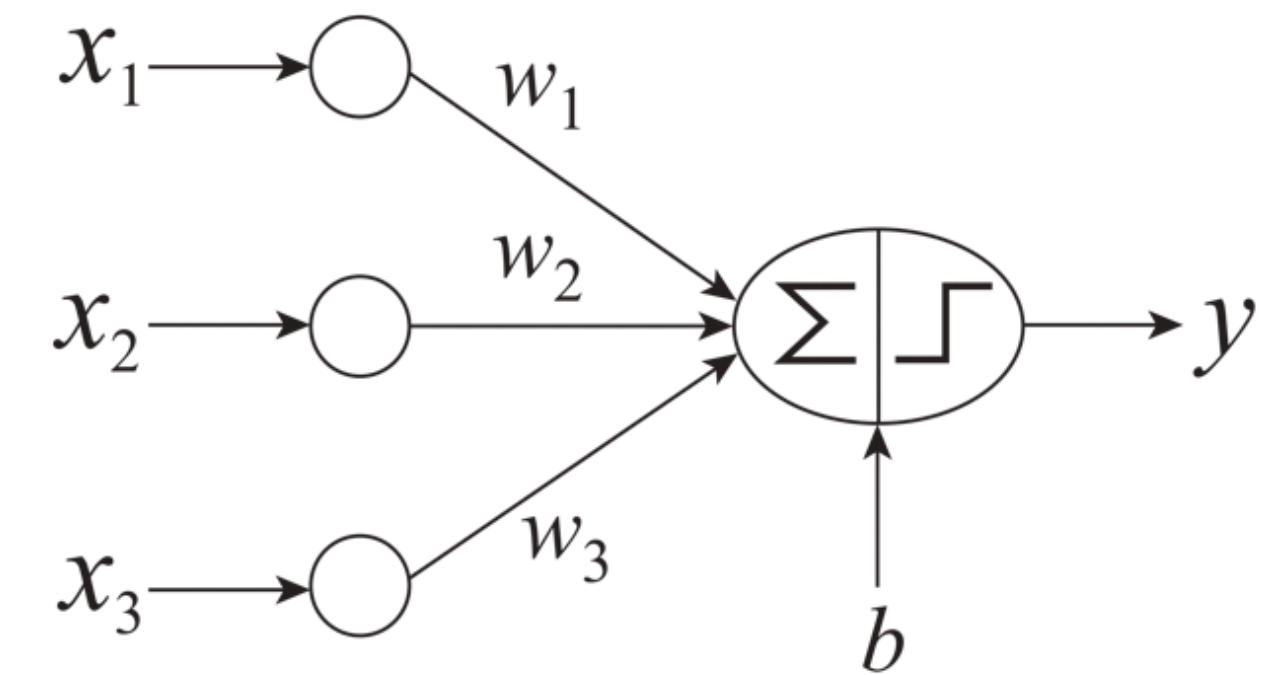
Perceptron (1943, Warren McCulloch e Walter Pitts)



Redes neurais

Perceptron (1943, Warren McCulloch e Walter Pitts)

- McCulloch e Pitts mostraram que a combinação de vários neurônios artificiais em sistemas neurais produz um elevado poder computacional, uma vez que qualquer função que pudesse ser representada por uma combinação de funções lógicas poderia ser modelada por uma rede formada por esses neurônios.
- Relacionado à regressão logística (a função de ativação é um sinal em vez de sigmoide).



$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases}$$

$$\tilde{\mathbf{w}} = (\mathbf{w}^T \ b)^T \quad \tilde{\mathbf{x}} = (\mathbf{x}^T \ 1)^T$$

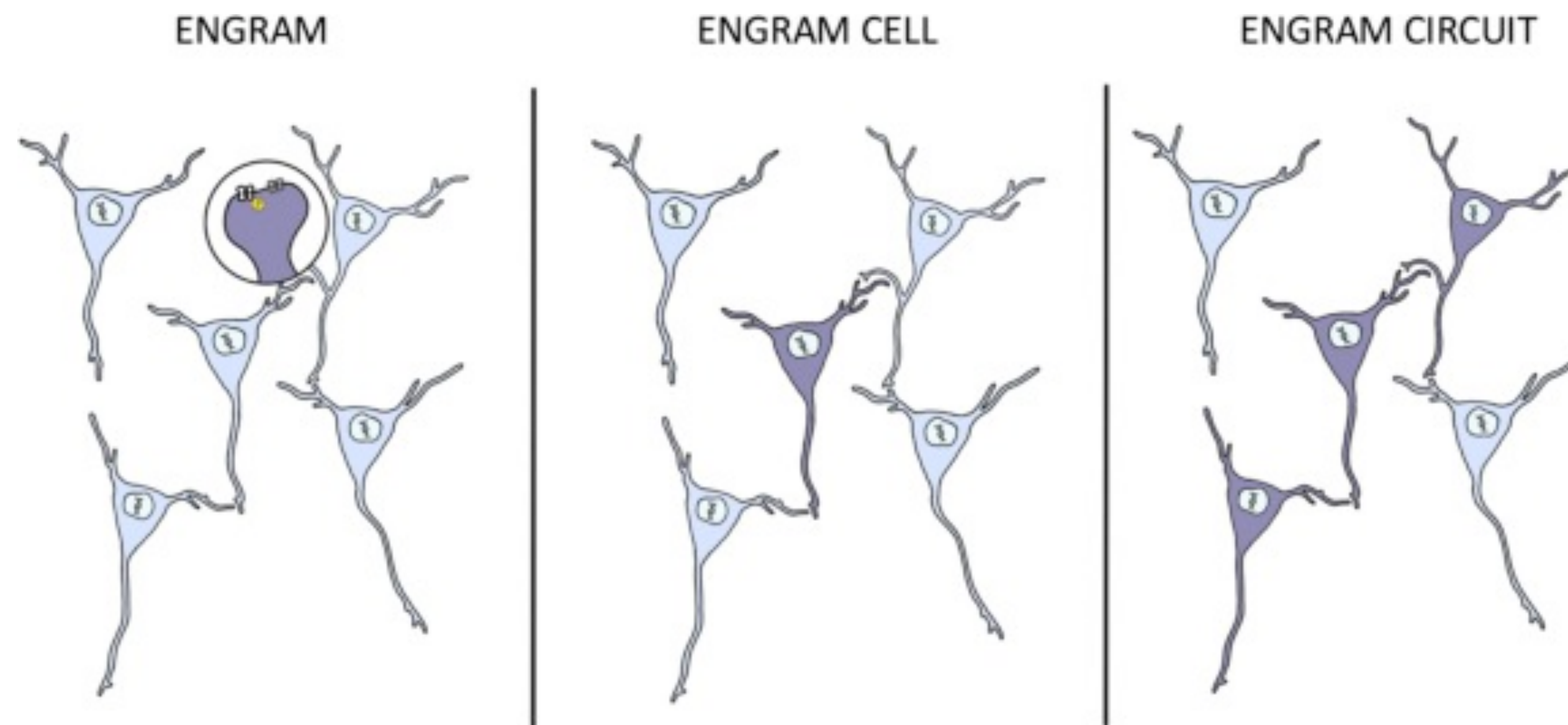
$$\hat{y} = \text{sign}(\tilde{\mathbf{w}}^T \tilde{\mathbf{x}})$$

Activation Function

Redes neurais

Regra de Hebb (1949, Donald Hebb)

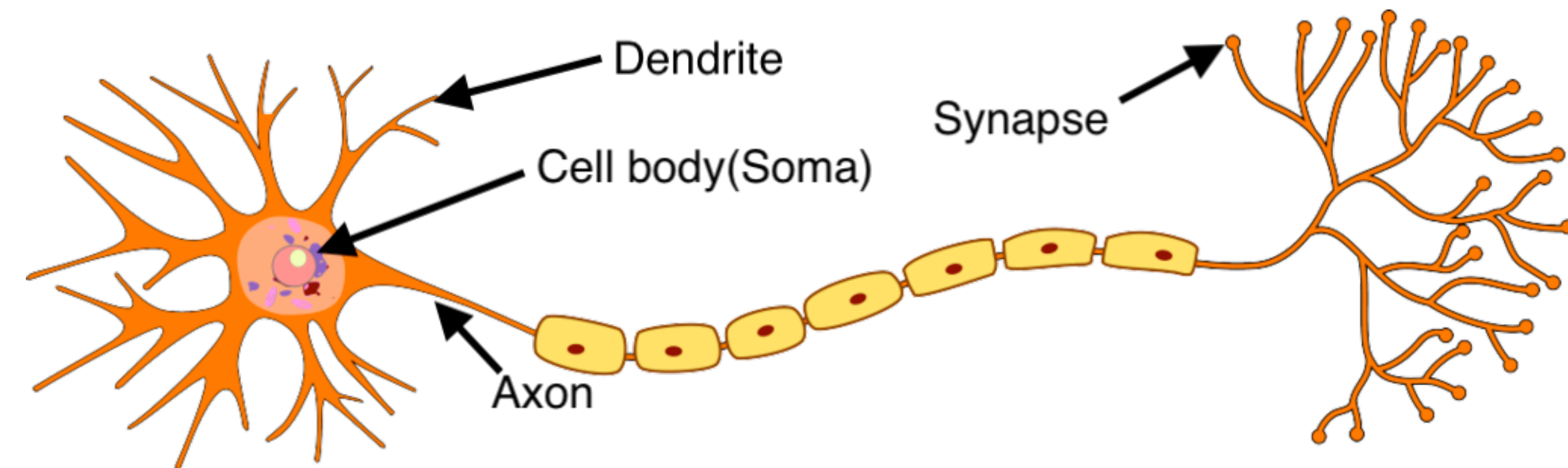
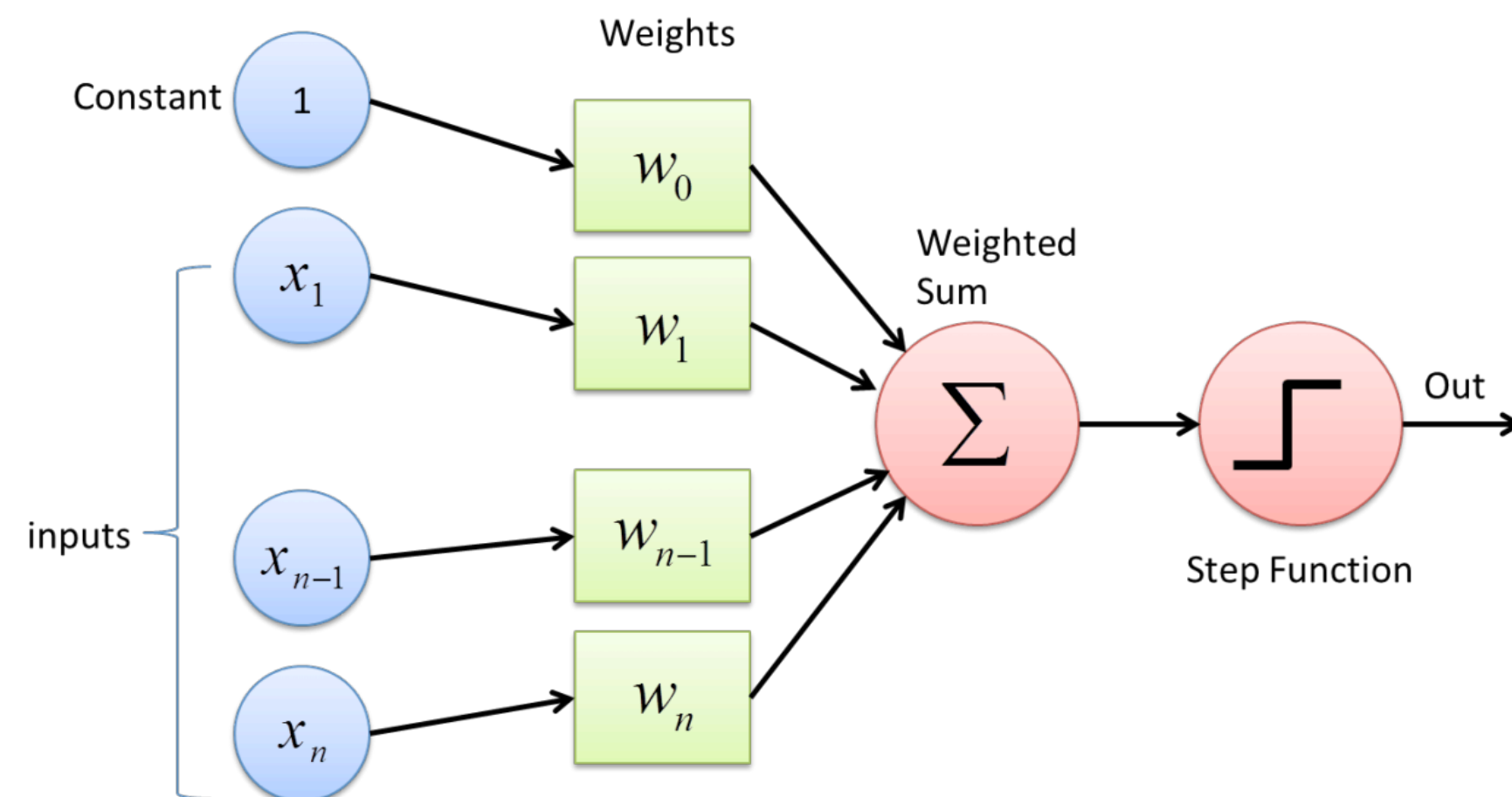
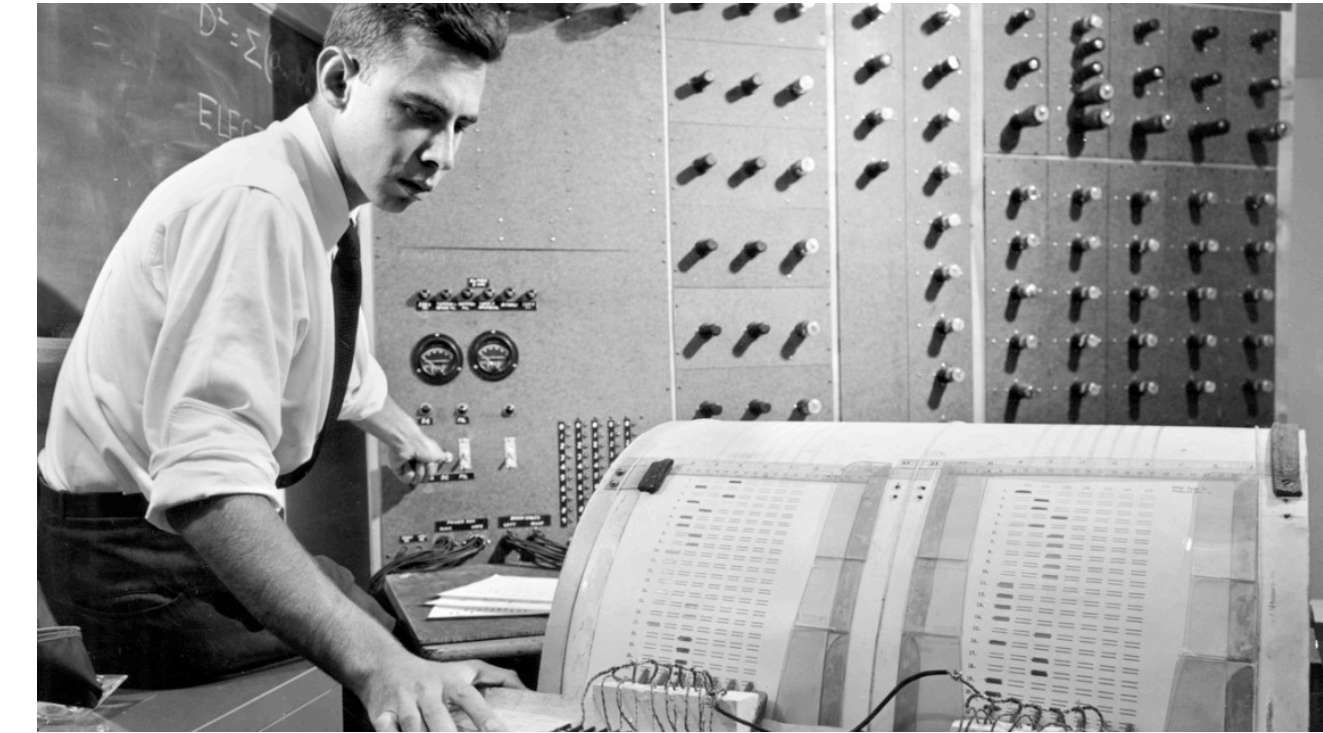
- Neurônios que disparam juntos, se conectam juntos.



- **Um engrama é uma unidade de informação cognitiva impressa em uma substância física**, teorizada como o meio pelo qual as memórias são armazenadas como mudanças biofísicas ou bioquímicas no cérebro ou outro tecido biológico , em resposta a estímulos externos.

Redes neurais

Perceptron (1957, Frank Rosenblatt)



- Perceptron: único neurônio.
- O perceptron é considerado um dos primeiros algoritmos de aprendizado supervisionado e serviu como base para o desenvolvimento das redes neurais artificiais modernas.
- Ele foi implementado inicialmente em hardware, em uma máquina chamada Mark I Perceptron.

Redes neurais

- **Algoritmo:**
 - Entrada: Um conjunto de objetos de treinamento
 - Saída: Rede perceptron com valores dos pesos ajustados
- Inicializar pesos da rede com valores baixos

Repita:

Para cada objeto $\mathbf{x}_i$, do conjunto de treinamento faça

Calcular valor da saída produzida pelo neurônio: $f(\mathbf{x}_i)$

Calcular: Erro = $y_i - f(\mathbf{x}_i)$

Se Erro > 0, **então:**

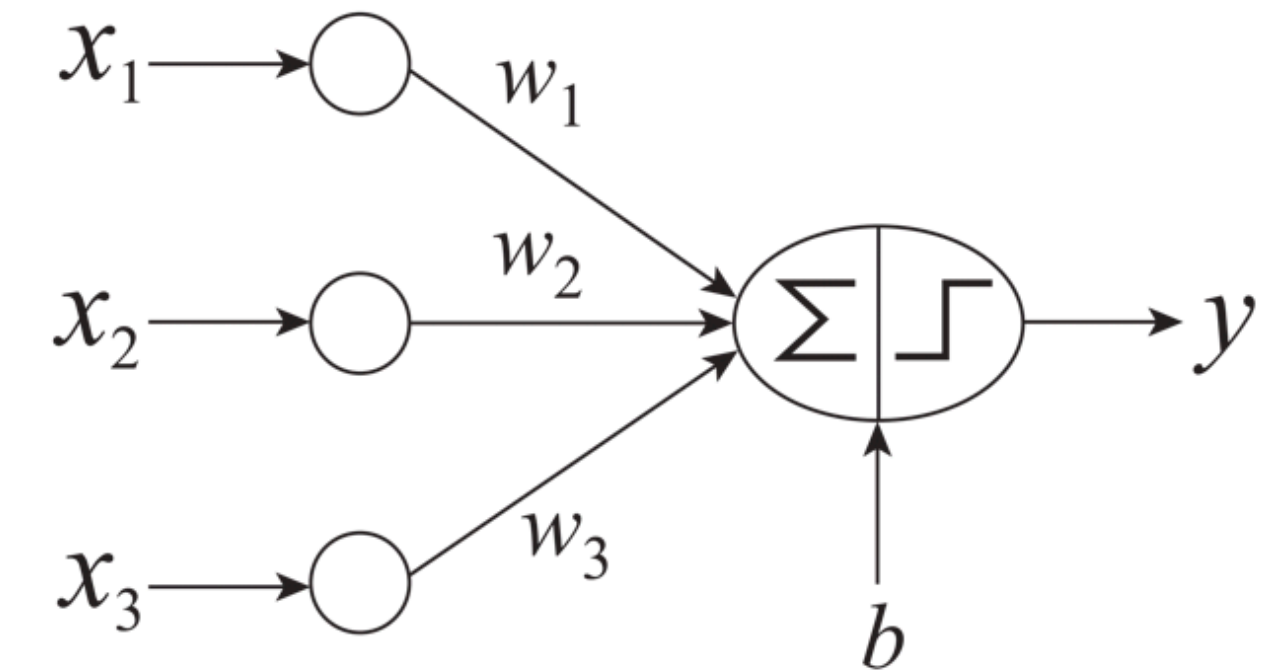
Ajustar pesos do neurônio utilizando:

$$w_j(t + 1) = w_j(t) + \eta x_i^j (y_i - \hat{f}(\mathbf{x}_i))$$

Fim

Fim

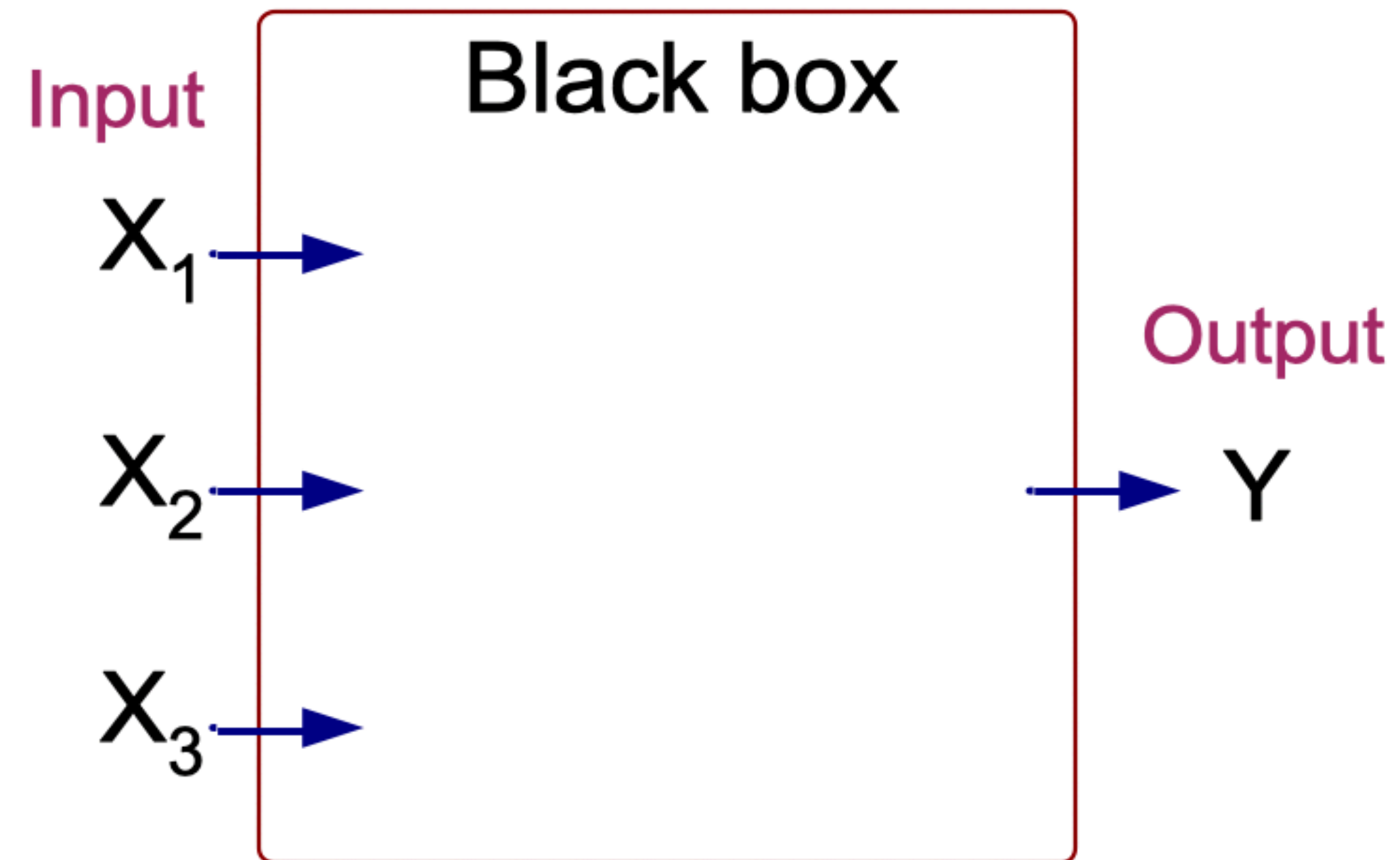
Até Erro = 0.



$$y = \begin{cases} 1, & \text{if } \mathbf{w}^T \mathbf{x} + b > 0. \\ -1, & \text{otherwise.} \end{cases}$$

Redes neurais

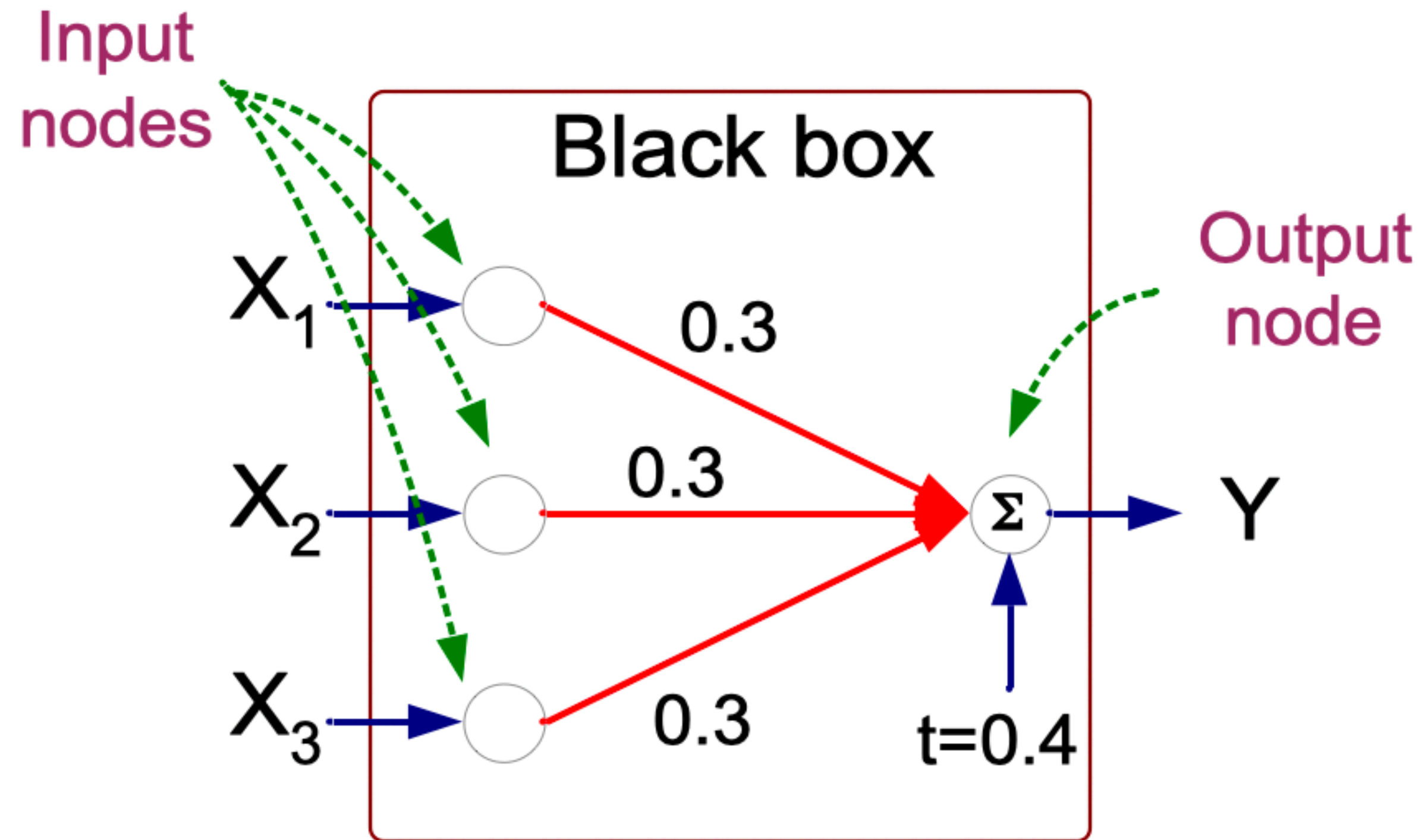
X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



A saída Y é 1 se pelo menos duas das três entradas forem iguais a 1.

Redes neurais

X_1	X_2	X_3	Y
1	0	0	-1
1	0	1	1
1	1	0	1
1	1	1	1
0	0	1	-1
0	1	0	-1
0	1	1	1
0	0	0	-1



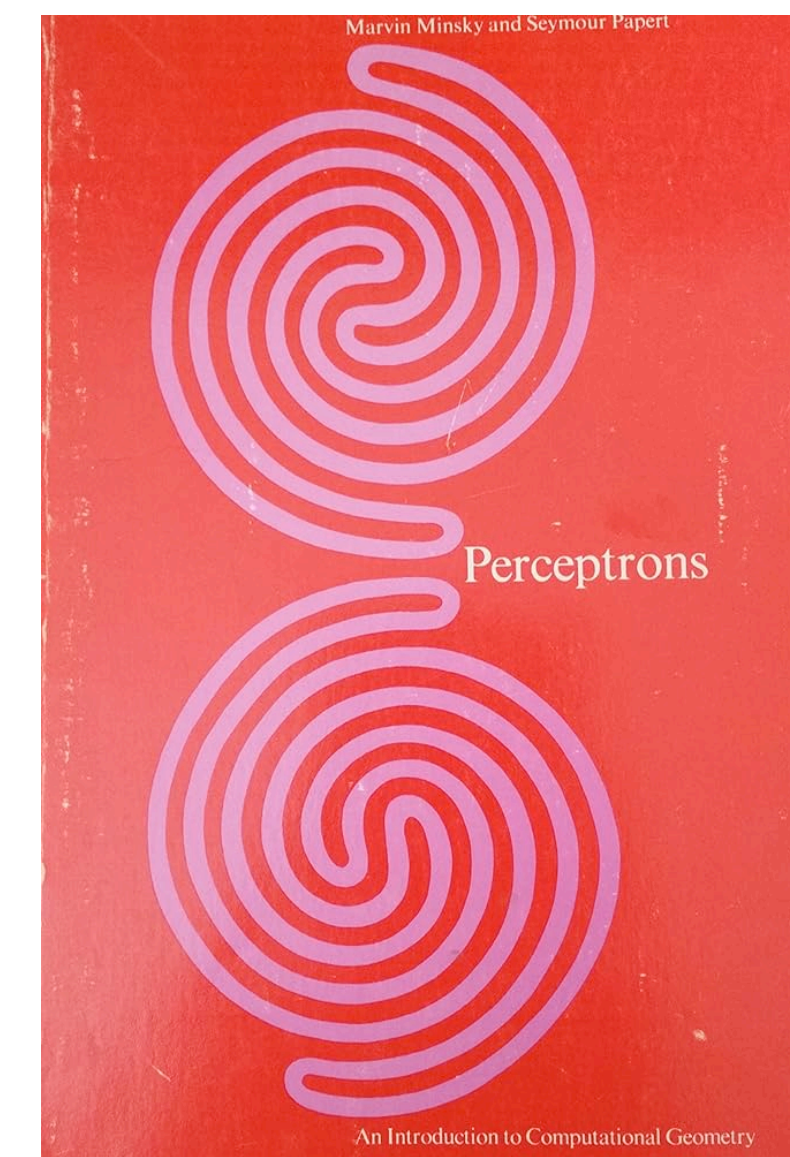
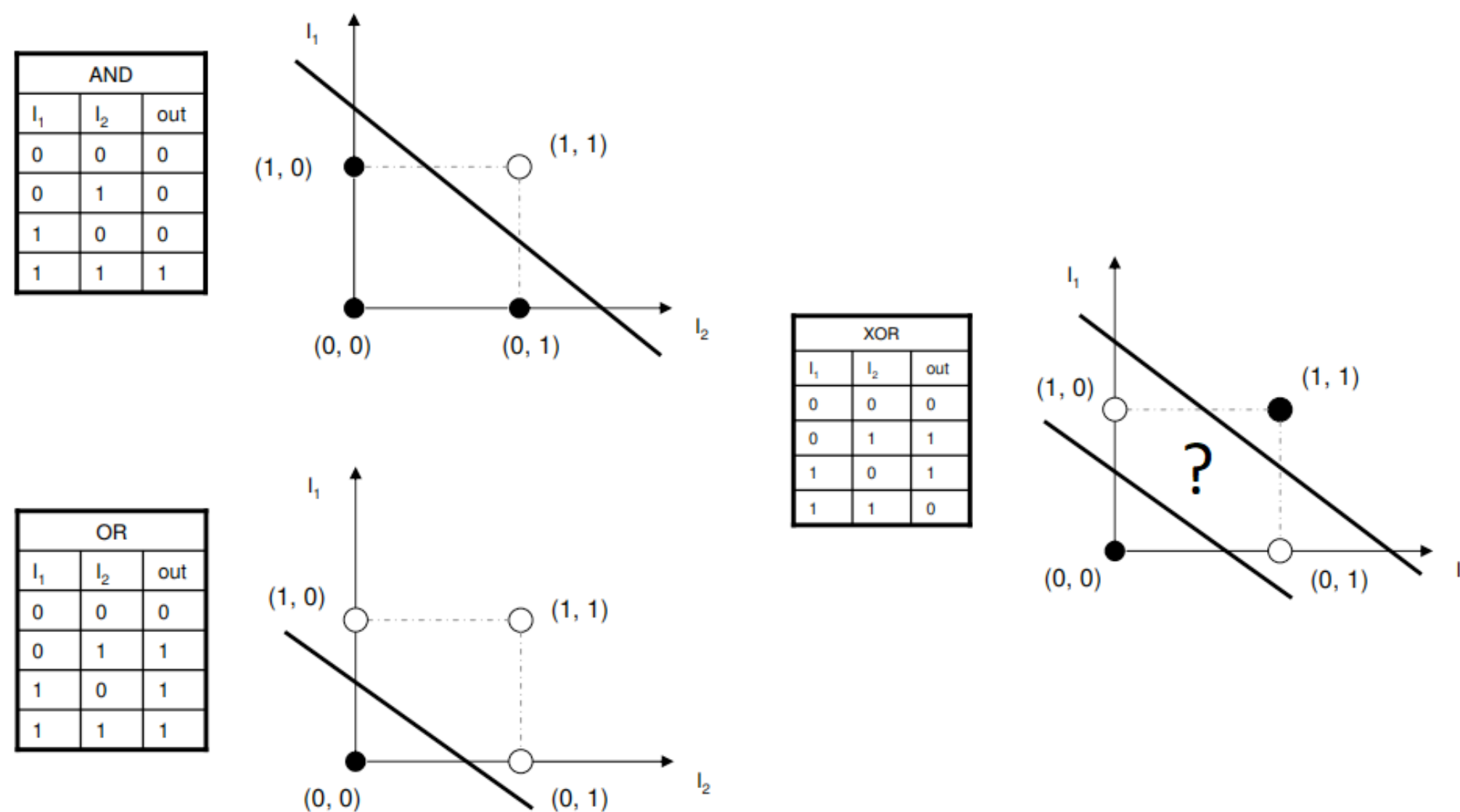
$$Y = \text{sign}(0.3 X_1 + 0.3 X_2 + 0.3 X_3 - 0.4)$$

$$\text{where } \text{sign}(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Redes neurais

Inverno das redes neurais (AI Winter)

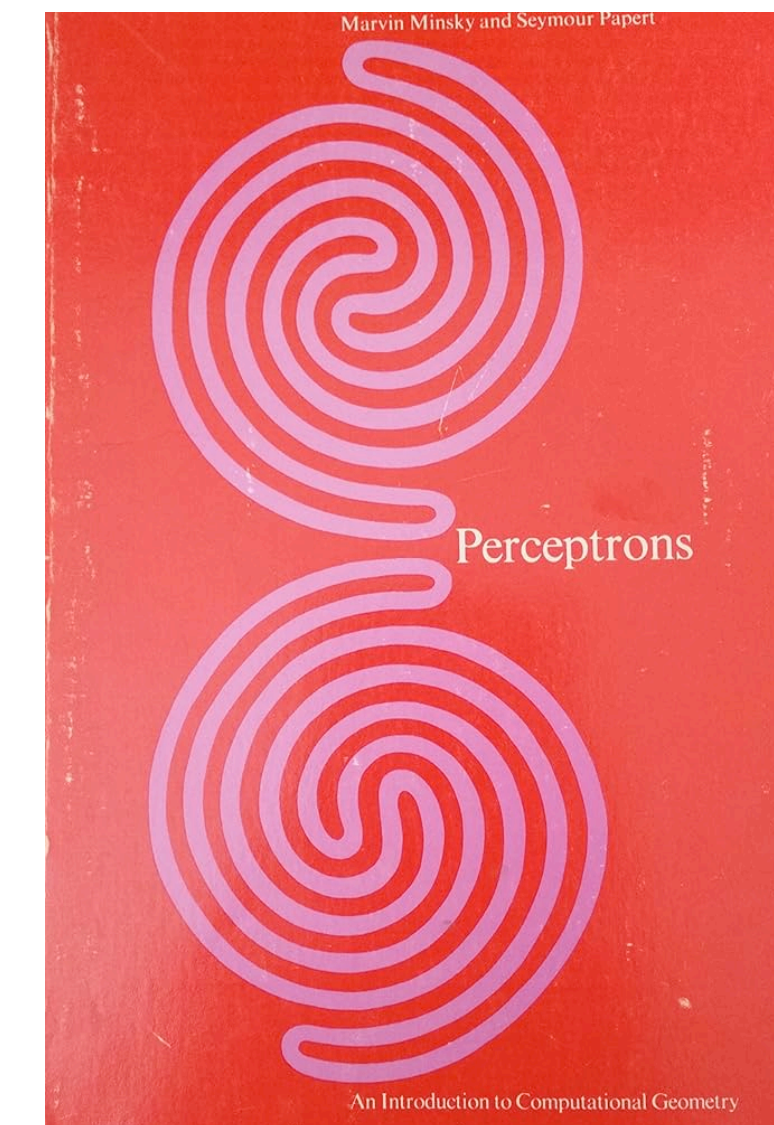
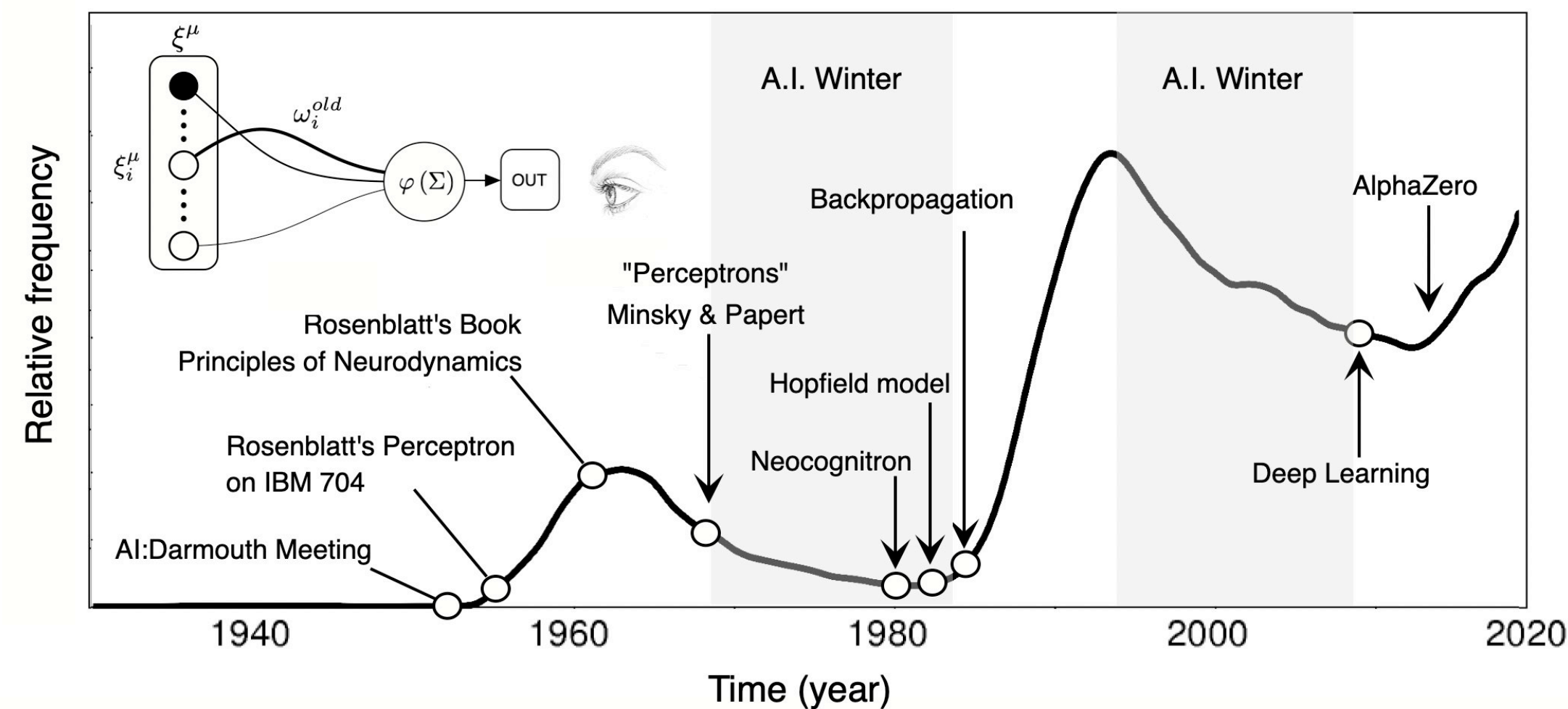
- Em 1969, Marvin Minsky e Seymour Papert, dois pesquisadores influentes do MIT, publicaram o livro *Perceptrons*, no qual demonstraram matematicamente que o perceptron não conseguia resolver problemas simples que envolvem não linearidade, como o problema do XOR (função "ou exclusivo").



Redes neurais

Inverno das redes neurais (AI Winter)

- Eles argumentaram que, mesmo com múltiplas camadas, não havia garantia de que essas redes pudessem ser treinadas de forma eficiente.
- Embora os autores tenham ressaltado que redes com múltiplas camadas poderiam ser uma solução (mas ainda mal compreendidas na época), muitos interpretaram o livro como uma prova definitiva de que redes neurais não funcionariam.
- Isso levou ao abandono da área por muitos pesquisadores e financiadores.



Redes neurais

Backpropagation (1986, Rumelhart, Hinton e Williams)

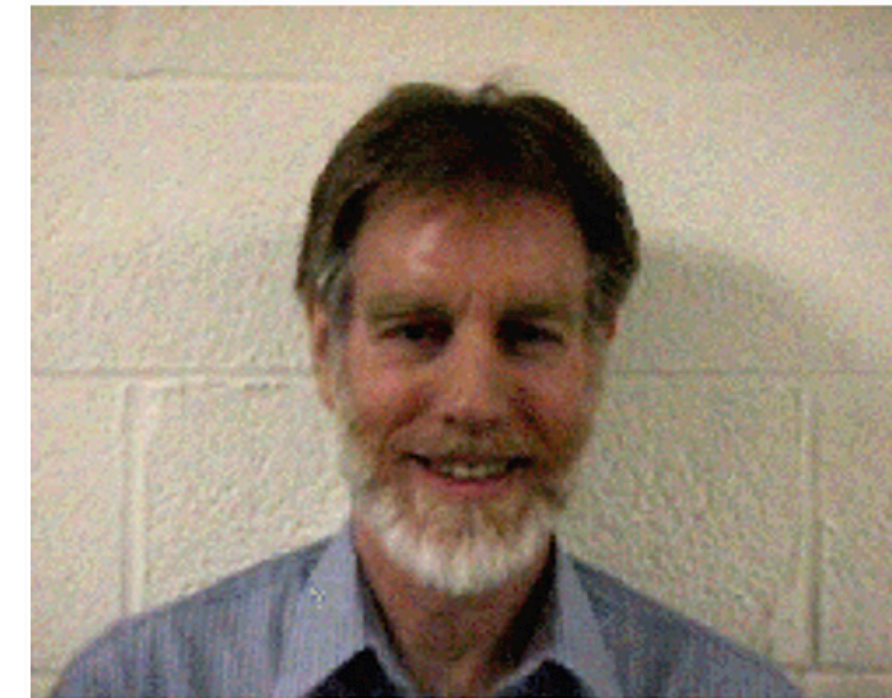
- O backpropagation (ou retropropagação do erro) é um dos algoritmos mais fundamentais e influentes da história das redes neurais artificiais.
- É um algoritmo de otimização baseado em gradiente descendente. Seu objetivo é ajustar os pesos da rede neural de modo que a função de erro (ou perda) seja minimizada.



David Rumelhart



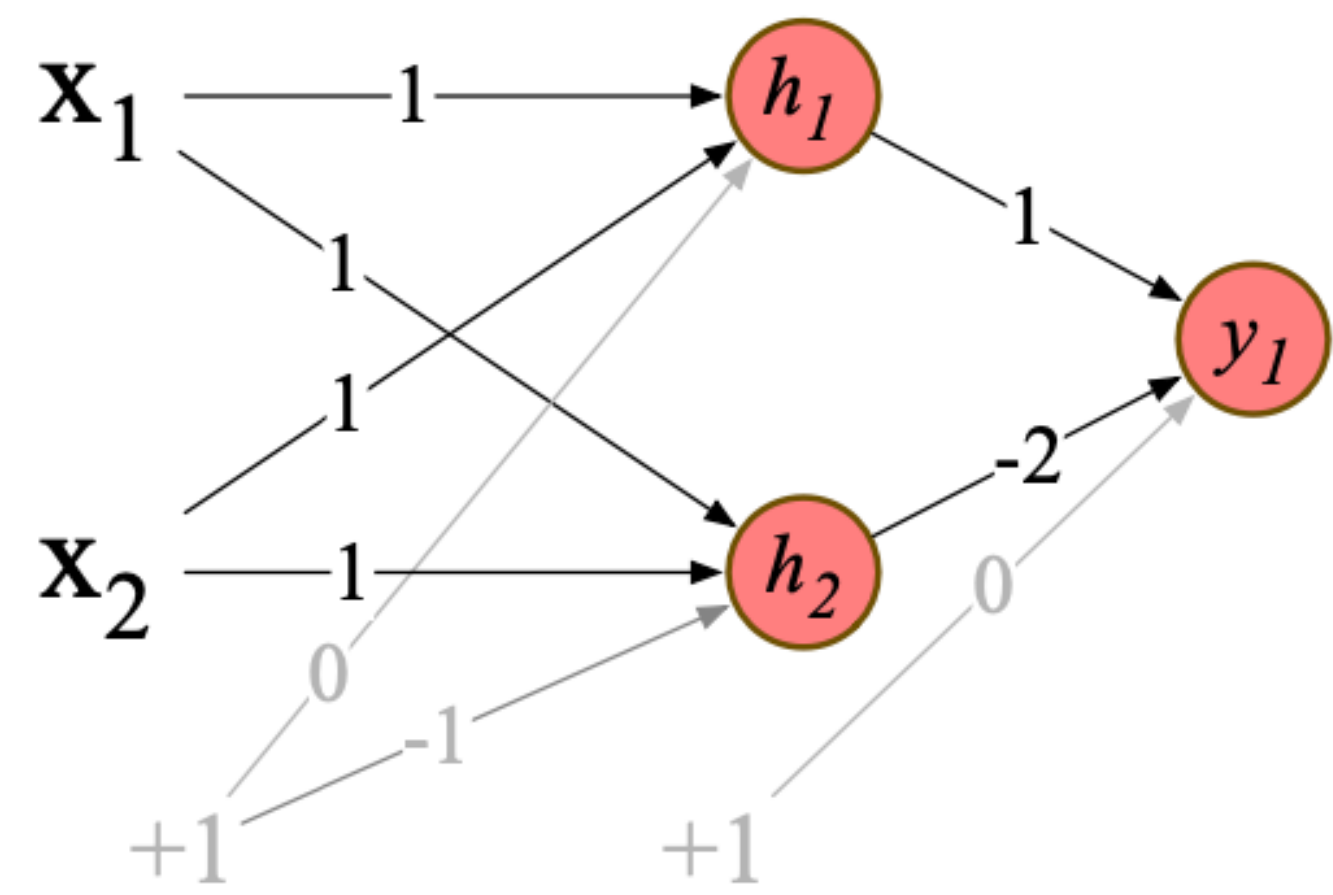
Geoffrey Hinton



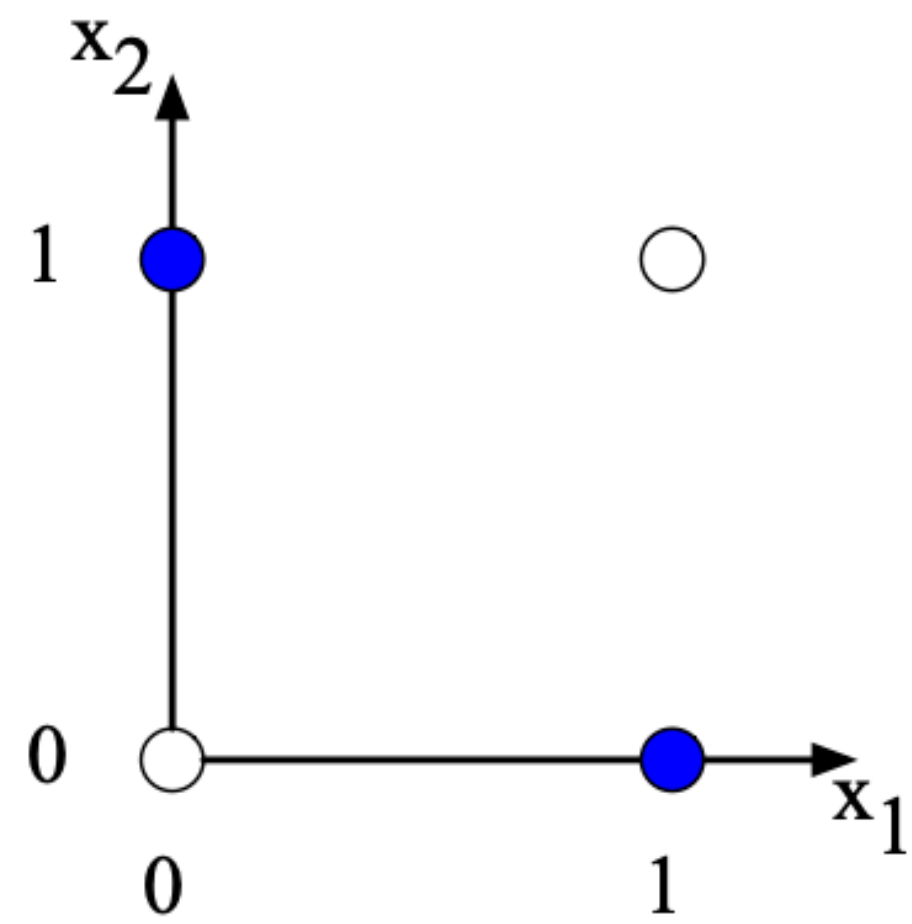
Ronald Williams

Redes neurais

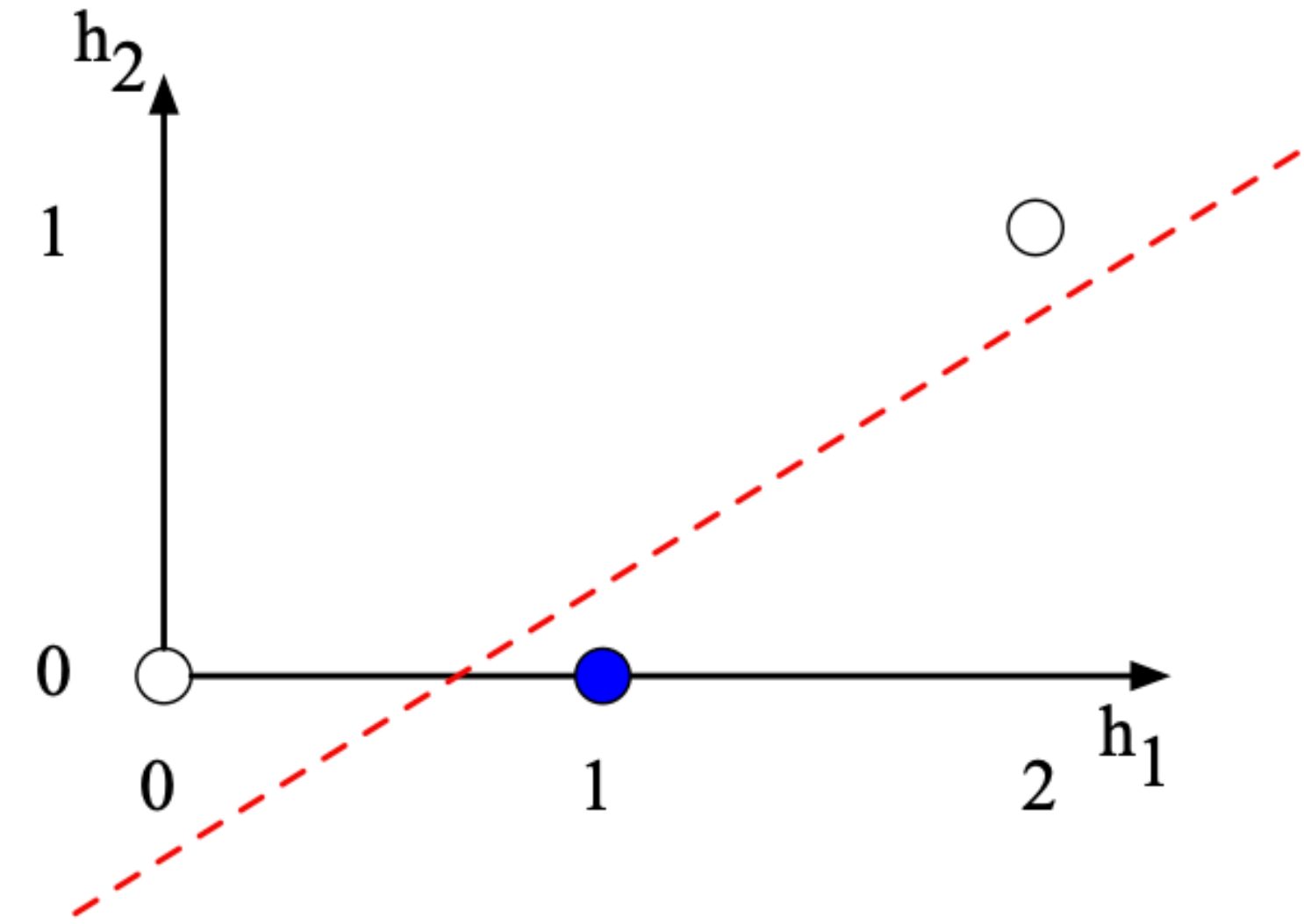
Solução do XOR: Goodfellow et al. (2016)



XOR		
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



a) The original x space



b) The new (linearly separable) h space

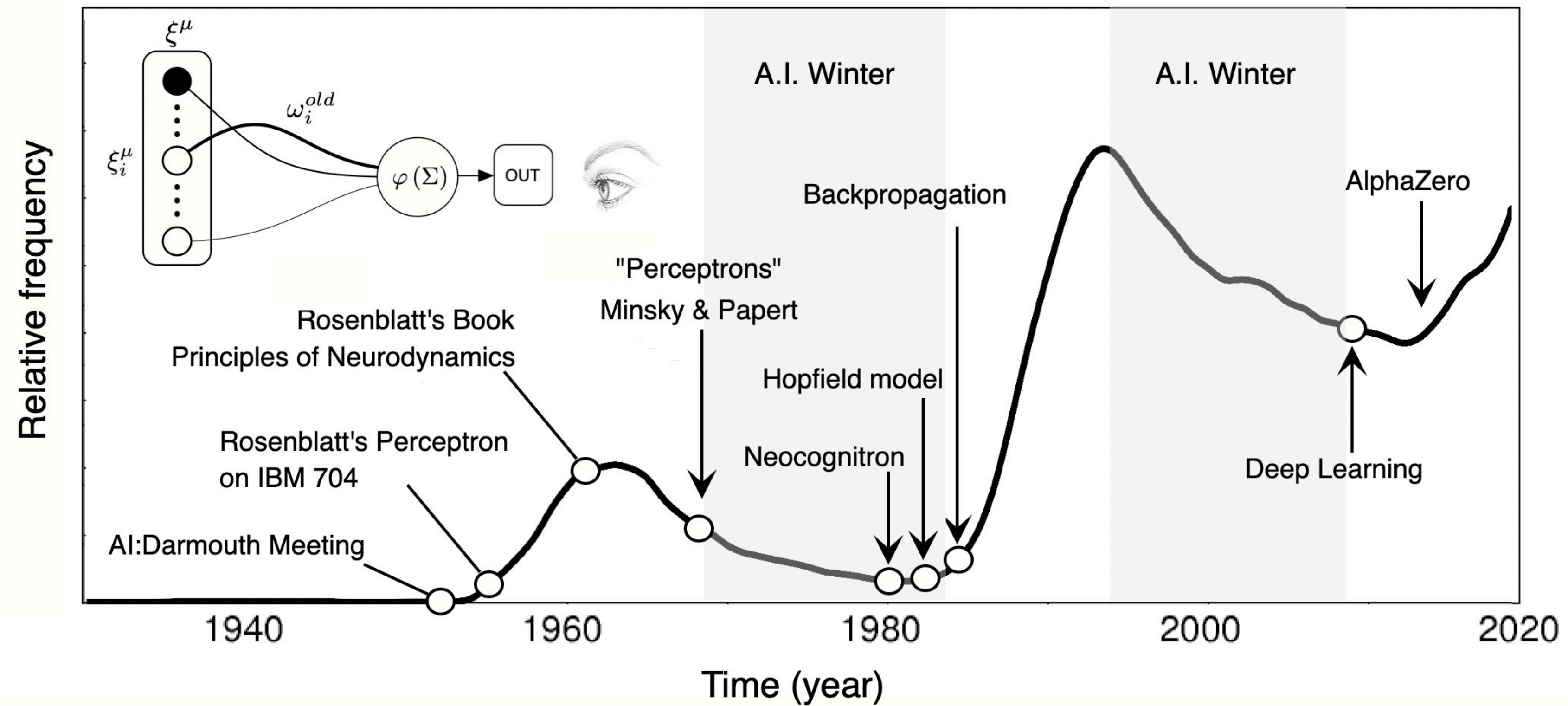
Redes neurais

Backpropagation (1986, Rumelhart, Hinton e Williams)

- **Forward pass (propagação direta):**
 - Os dados de entrada passam pelas camadas da rede.
 - A saída da rede é comparada com a saída esperada (rótulo).
 - Calcula-se um erro (por exemplo, erro quadrático médio ou entropia cruzada).
- **Backward pass (retropropagação):**
 - O erro é propagado de volta da camada de saída para as camadas anteriores.
 - Calcula-se o gradiente do erro com relação a cada peso da rede, usando a regra da cadeia do cálculo diferencial.
 - Esses gradientes são então usados para atualizar os pesos.
- **Atualização dos pesos:**
 - Usando o gradiente descendente (ou variantes como Adam), os pesos são ajustados:
$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \cdot \frac{\partial E}{\partial w_{ij}}$$
 - $w_{ij}^{(t)}$: peso atual da conexão do neurônio j para o neurônio i na atualização t.
 - η : taxa de aprendizado

Redes neurais

Backpropagation (1986, Rumelhart, Hinton e Williams)



Redes neurais

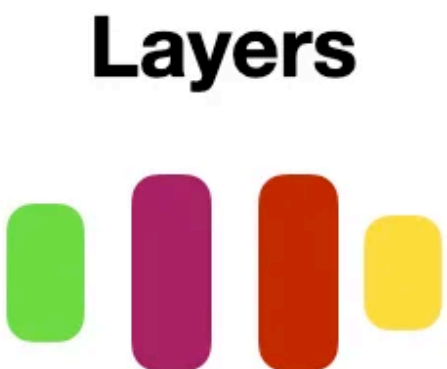
Componentes e arquiteturas



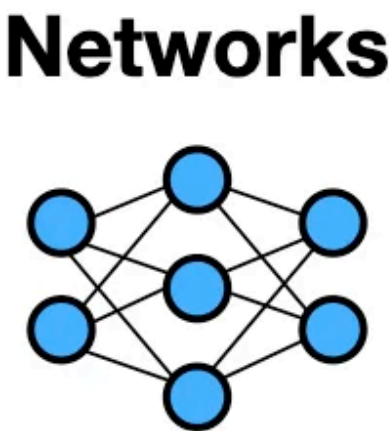
Name	Description
Vanilla	Basic unit computing weighted sum + activation; used in FFNNs and CNNs
LSTM	Advanced neuron with memory and gates for long-term dependencies in sequences



Name	Description
ReLU	$\max(0, x)$; fast, widely used in deep networks
Sigmoid	S-shaped, output in (0, 1); used in binary classification
Tanh	Like sigmoid but centered at 0; output in (-1, 1)
Softmax	Outputs a probability distribution; used in final layer of multi-class classification

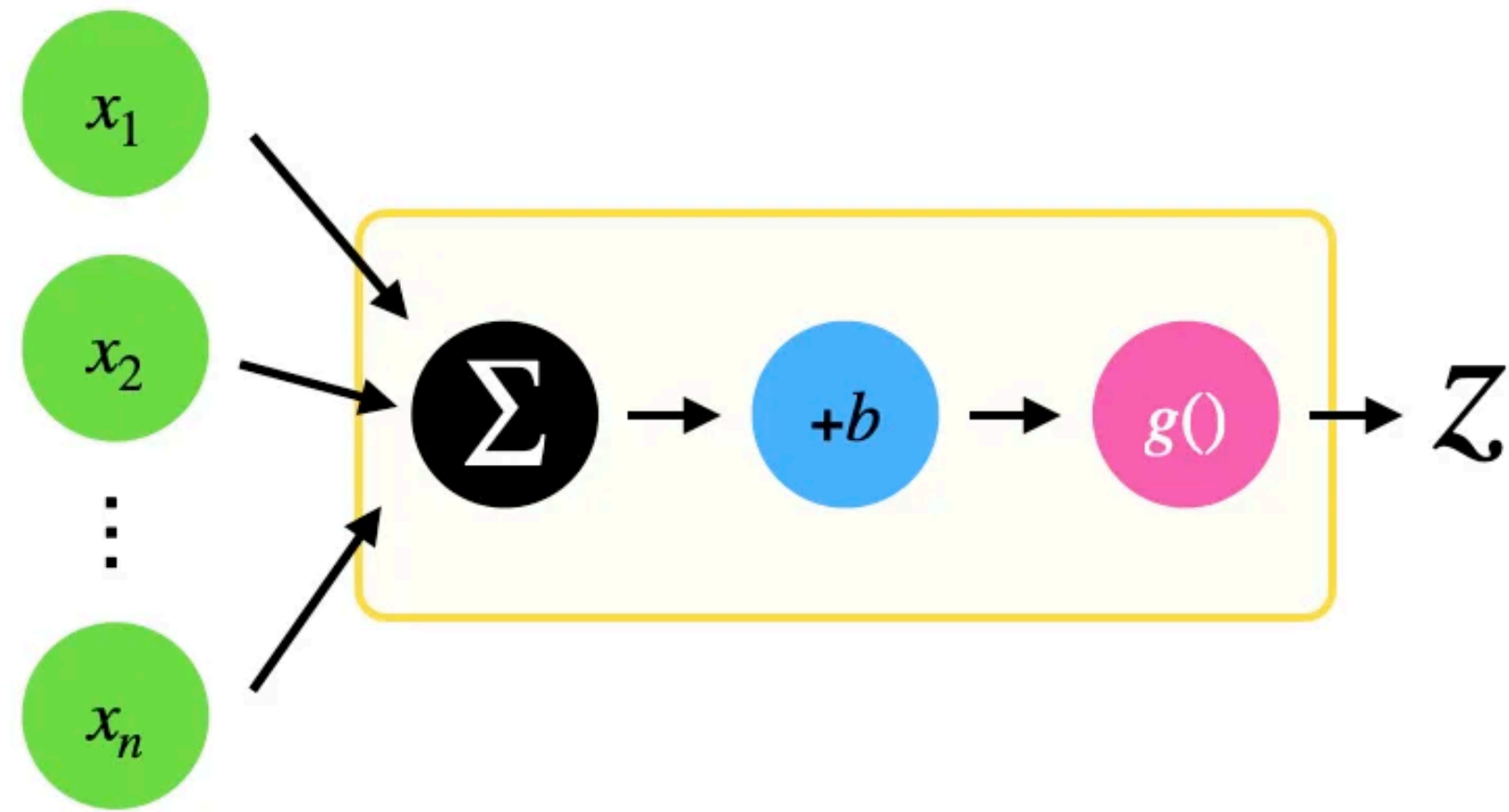


Name	Description
Fully Connected	Standard layer where each neuron connects to all inputs
Recurrent	Maintains memory across timesteps; used in RNNs, LSTMs
Convolutional	Extracts spatial features using filters; used in image data
Attention	Computes weighted importance of different inputs; key in Transformers
Pooling	Downsamples spatial data; used in CNNs to reduce size and noise
Normalization	Stabilizes training by normalizing activations; includes BatchNorm, LayerNorm
Dropout	Randomly deactivates neurons during training to prevent overfitting



Name	Description
Feedforward (FFNN)	Basic architecture with no loops; used for static input-output tasks
RNN	Handles sequential data using recurrence; remembers previous inputs
CNN	Uses convolutions to process grid-like data such as images
Transformer	Uses self-attention to model sequences without recurrence; state-of-the-art in NLP & beyond

Redes neurais



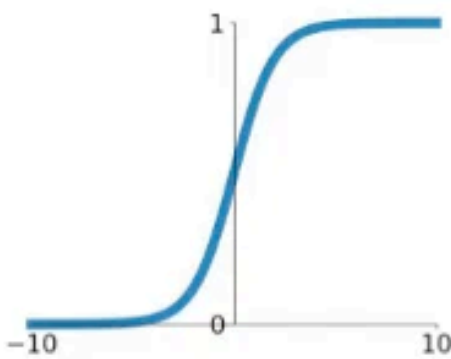
$$z = g\left(\sum w_i x_i + b\right)$$

The equation is annotated with colored boxes and labels: z is in a yellow box labeled 'Output'; g is in a pink box labeled 'Activation'; w_i and b are in blue boxes labeled 'Parameters'; x_i is in a green box labeled 'Inputs'. Arrows point from the labels to their respective components in the equation.

Funções de ativação

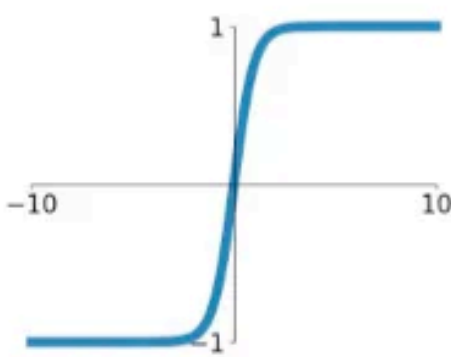
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



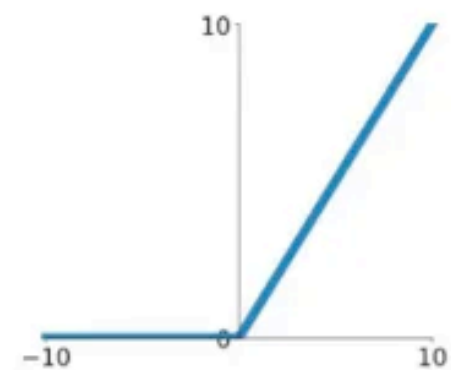
tanh

$\tanh(x)$



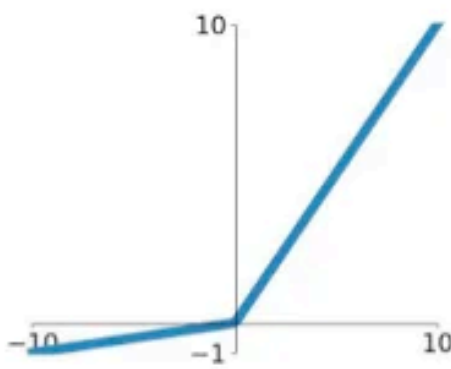
ReLU

$\max(0, x)$



Leaky ReLU

$\max(0.1x, x)$

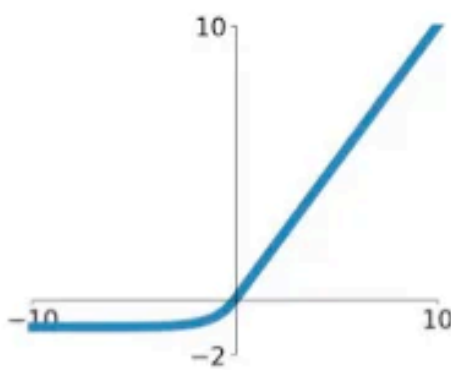


Maxout

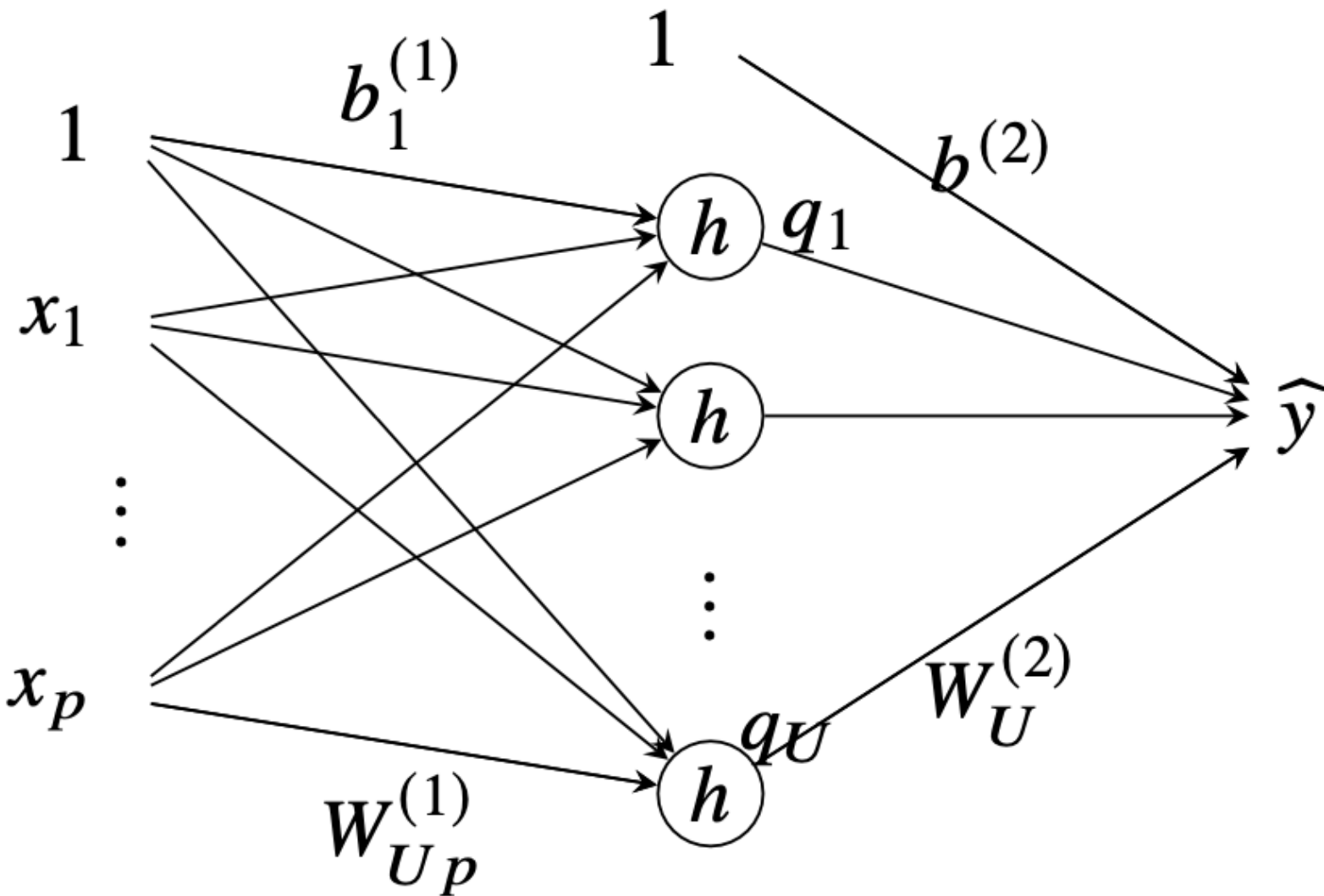
$\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



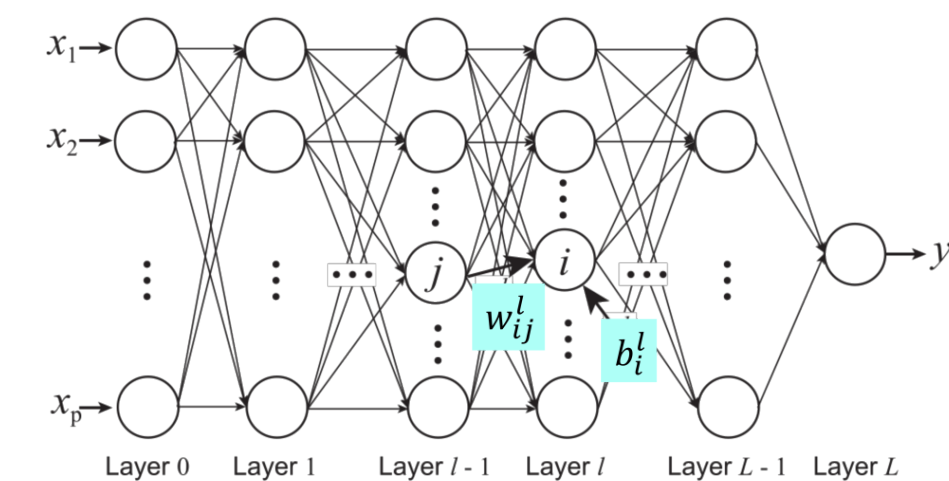
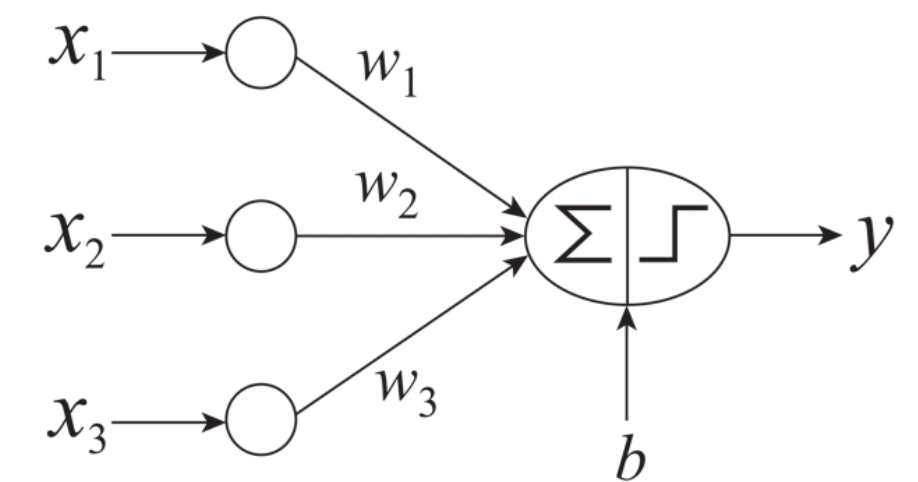
Input variables Hidden units Output



Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks
<https://arxiv.org/abs/2402.09092>

Redes neurais

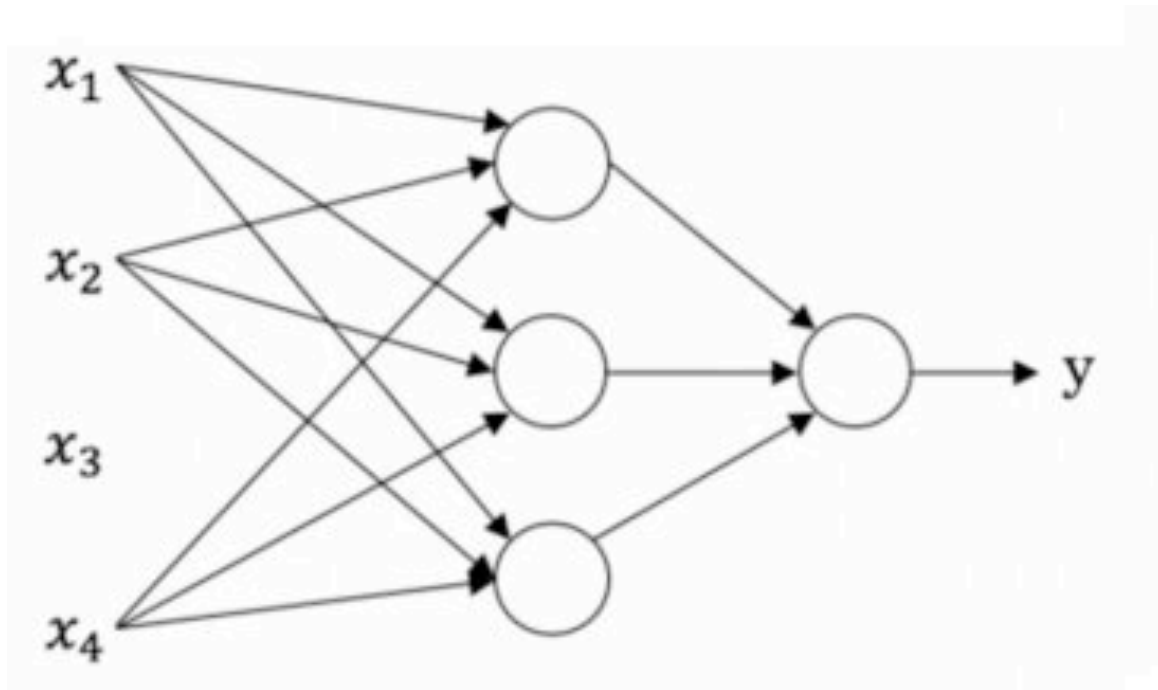
- Uma RNA é caracterizada por dois aspectos básicos:
 - **Arquitetura:** está relacionada ao tipo e número de unidades de processamento e à forma como os neurônios estão conectados.
 - Cada terminal de entrada do neurônio, simulando os dendritos, recebe um valor.
 - Os valores recebidos são ponderados e combinados por uma função matemática, equivalendo ao processamento realizado pelo soma.
 - A saída de um neurônio é definida por meio da aplicação de uma função de ativação à entrada total.
 - Os neurônios podem estar dispostos em uma ou mais camadas.
 - **Aprendizado:** diz respeito às regras utilizadas para o ajuste dos pesos da rede e que informação é utilizada pelas regras.



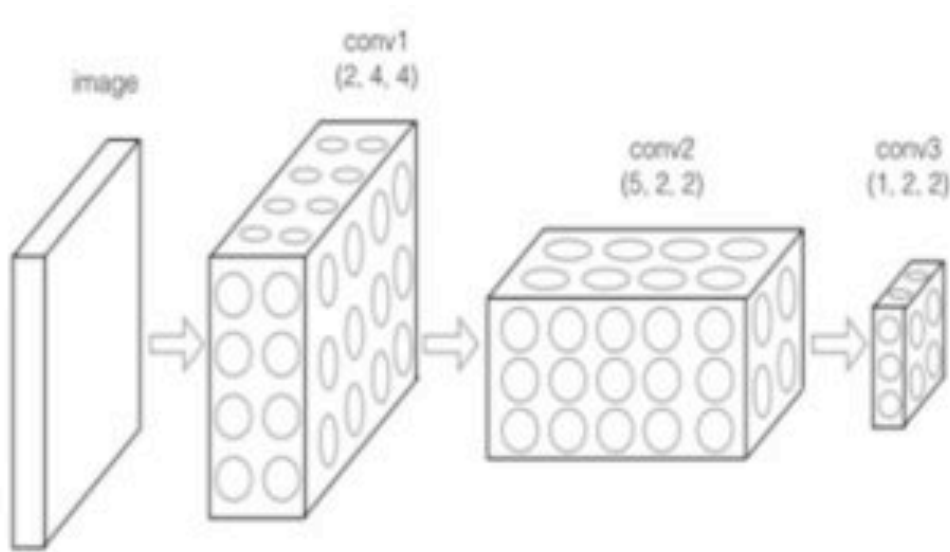
Redes neurais

- Rede Neural Multicamadas

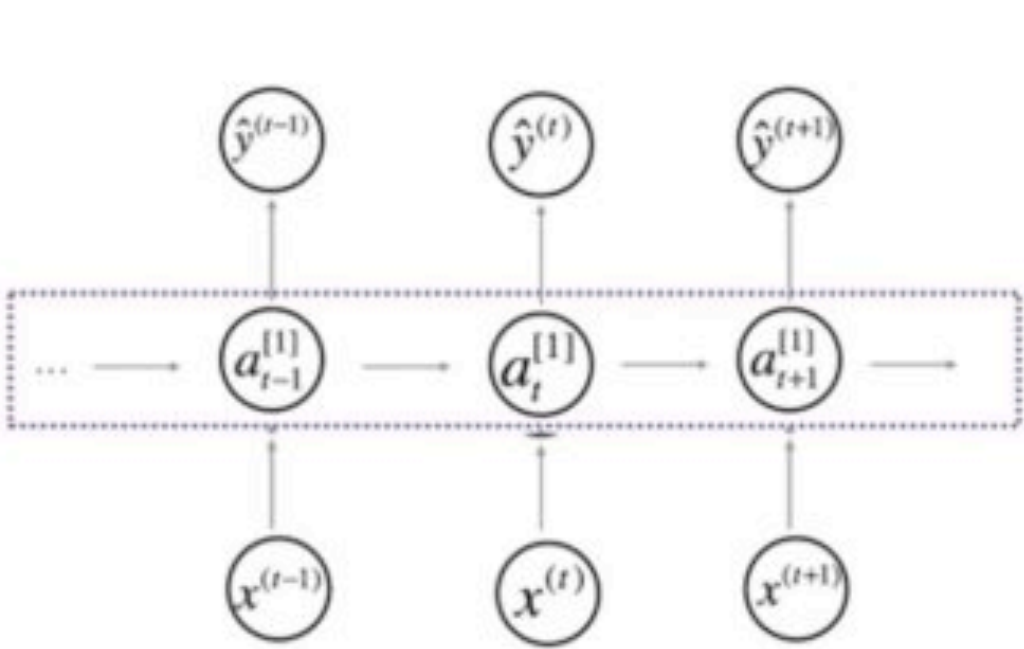
O número de camadas, o número de neurônios em cada camada, o grau de conectividade e a presença ou não de conexões de retropropagação definem a topologia de uma RNA.



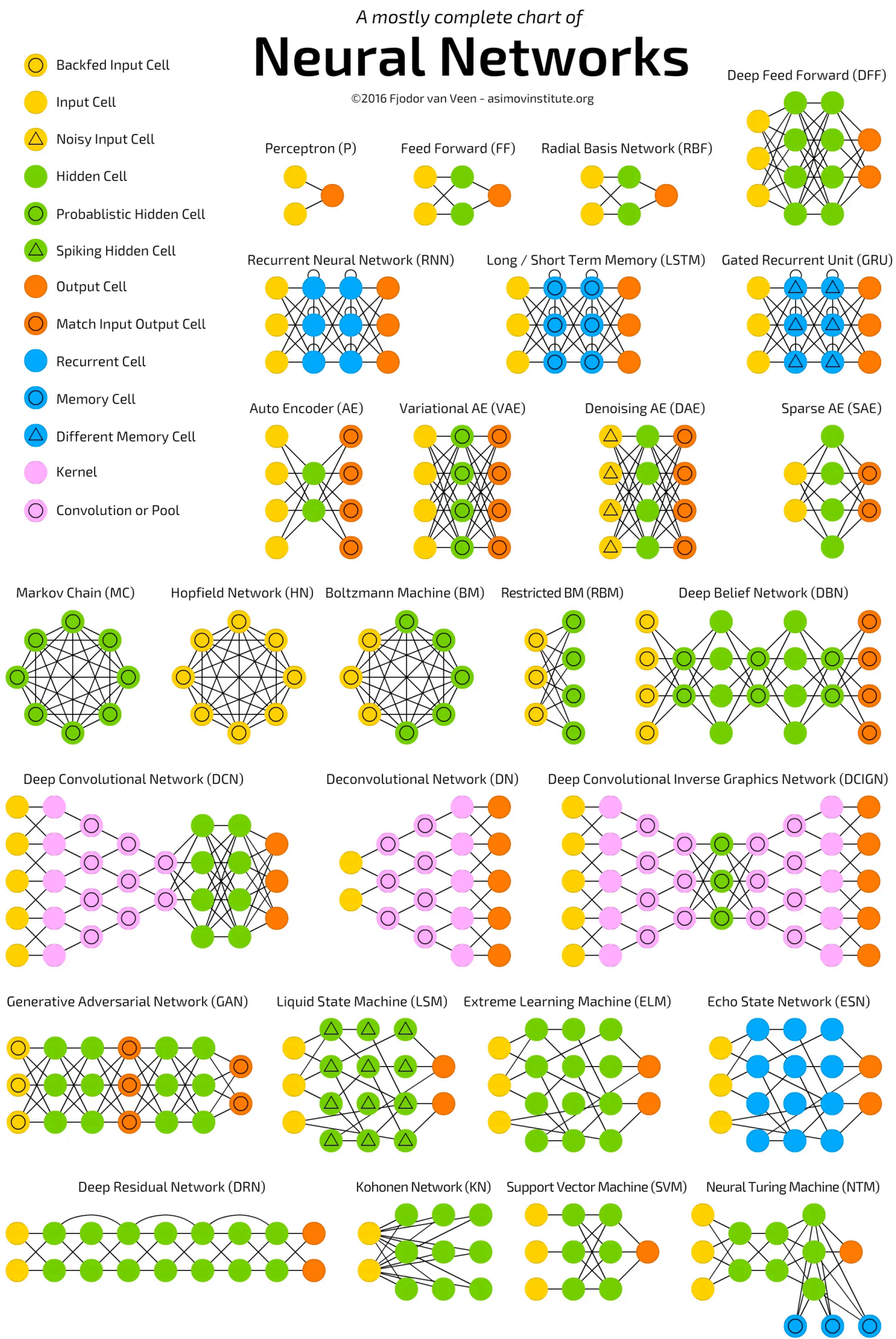
Standard NN



Convolutional NN

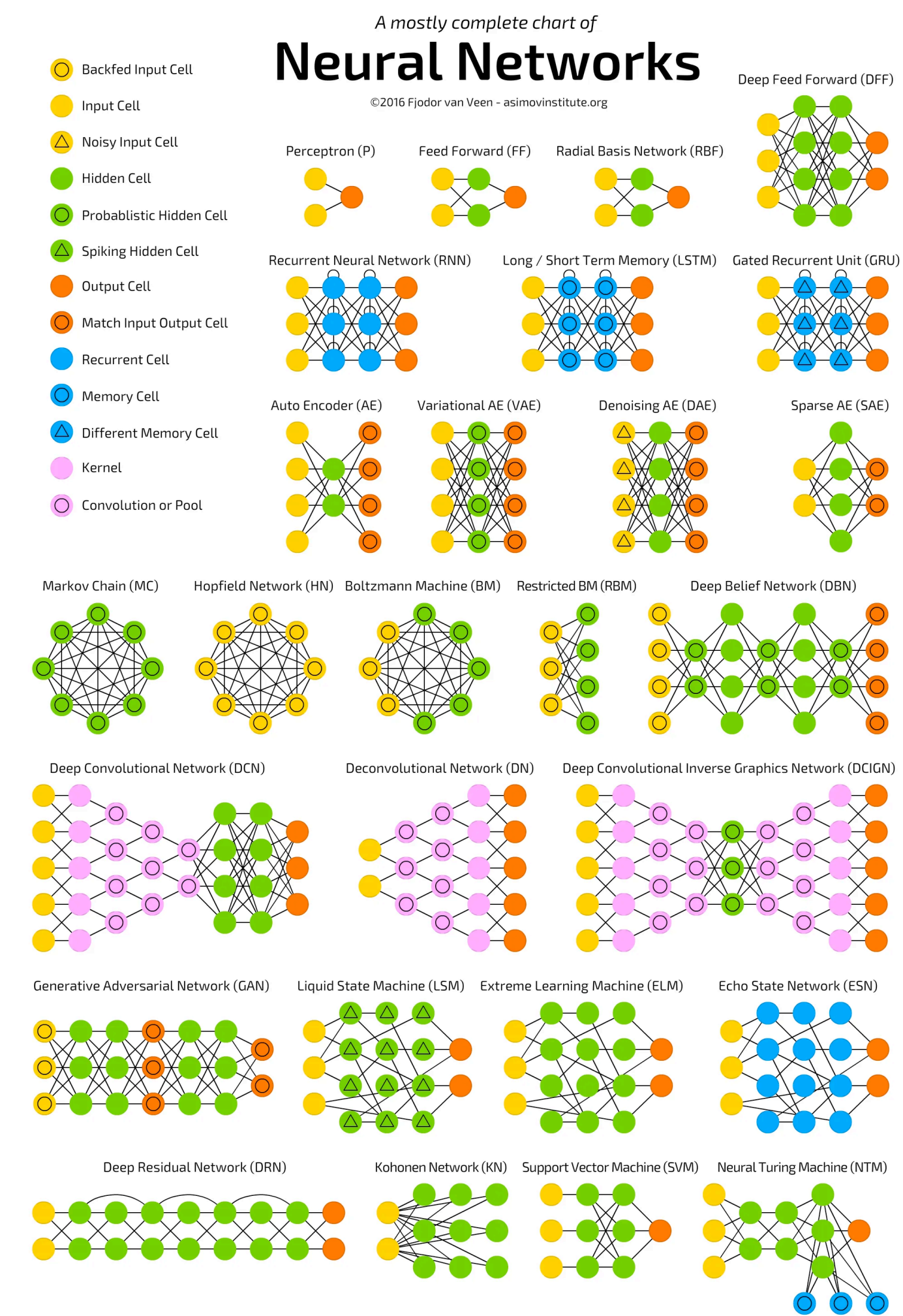
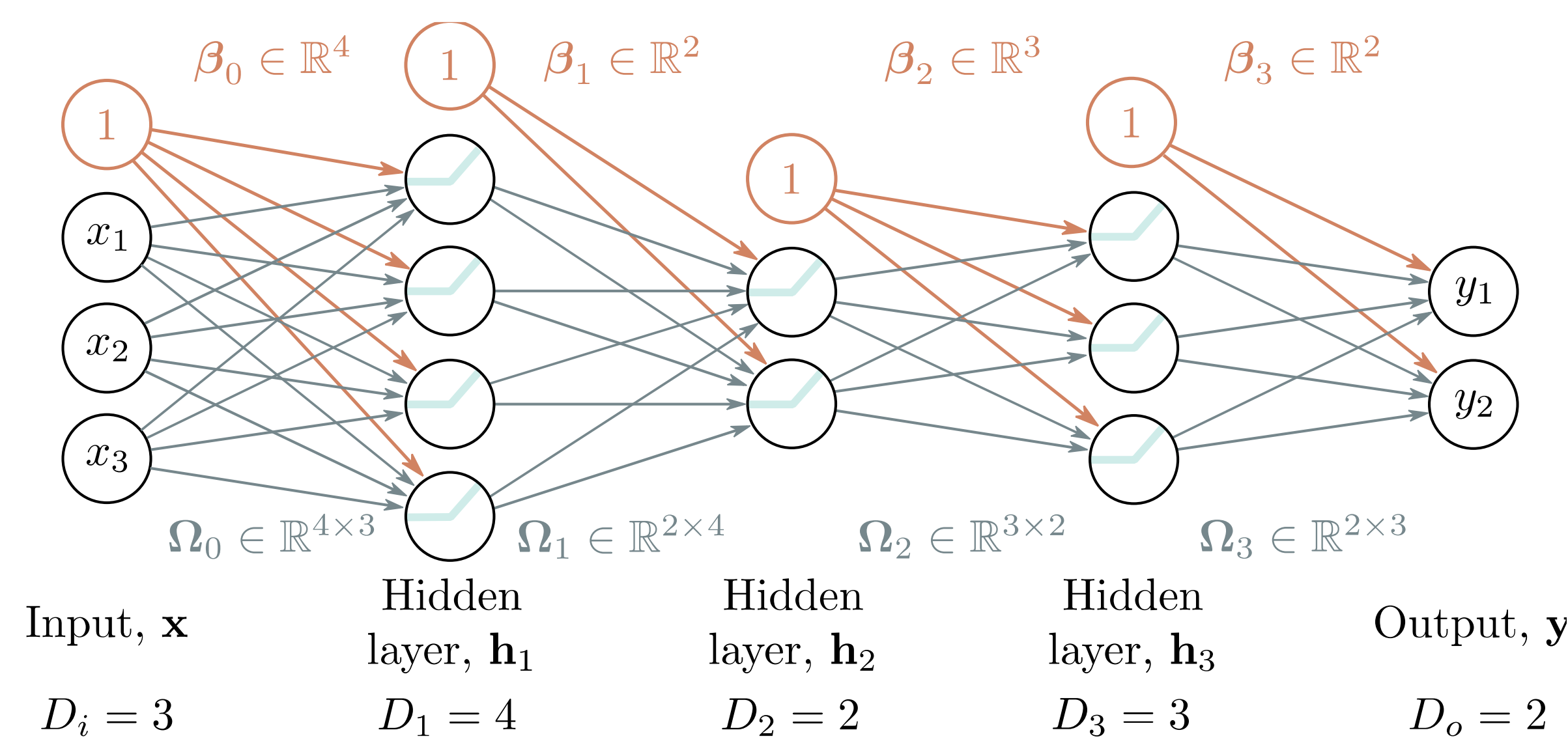


Recurrent NN



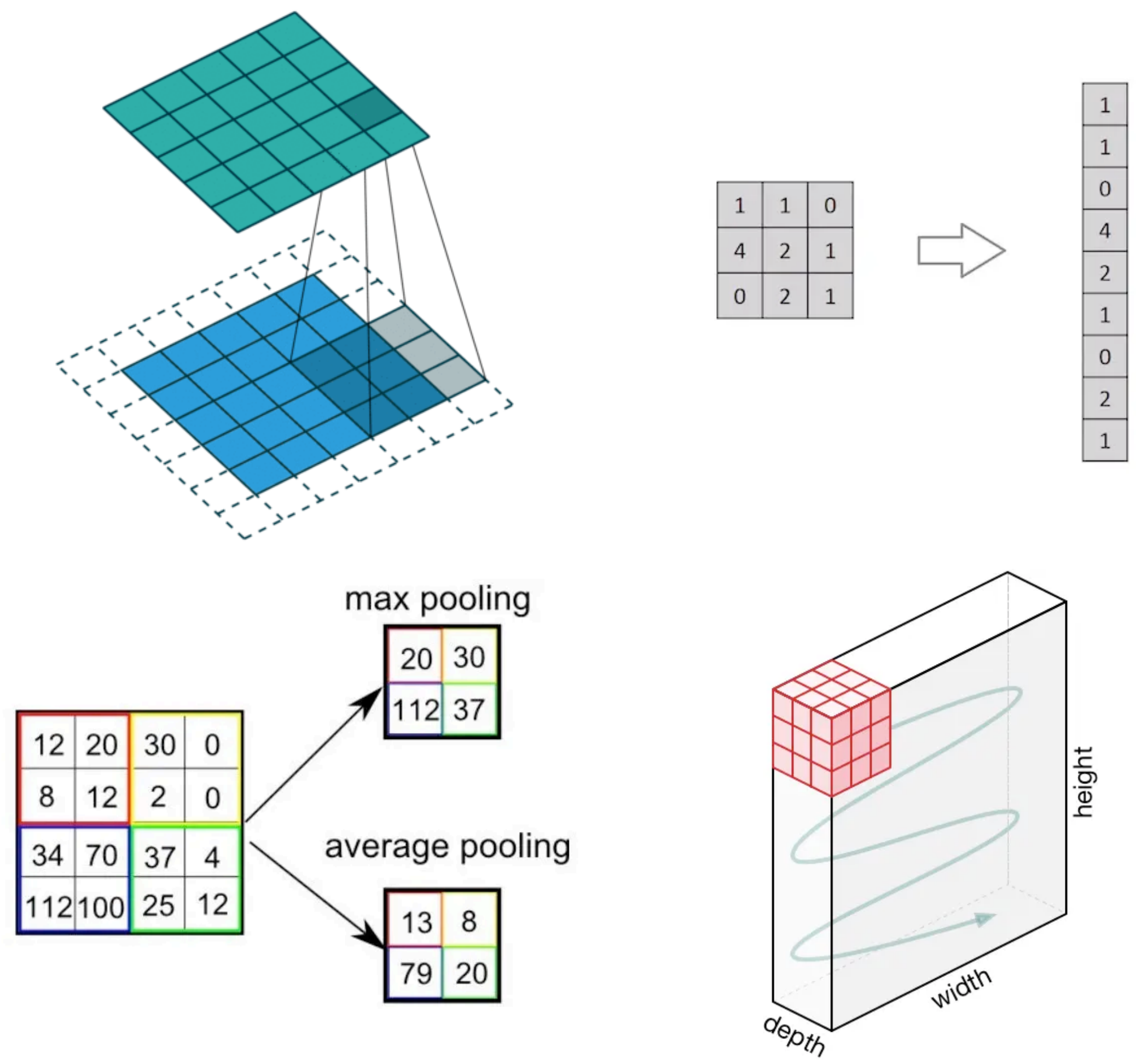
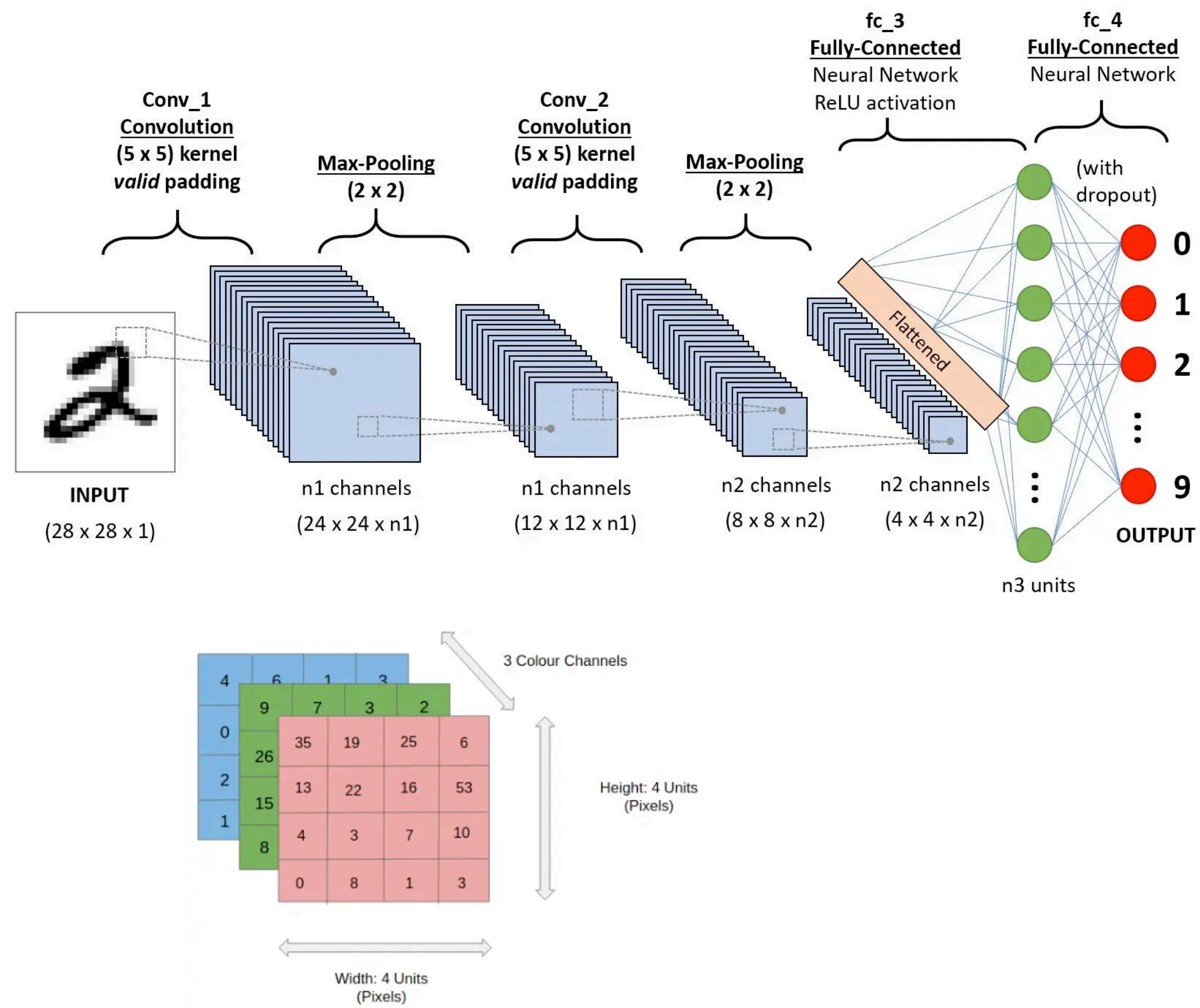
Redes neurais

- **MLP (feedforward neural networks)**
 - Os dados fluem da camada de entrada para a(s) camada(s) oculta(s) e, por fim, para a camada de saída.
 - Possuem pelo menos uma camada oculta entre entrada e saída.



Redes neurais

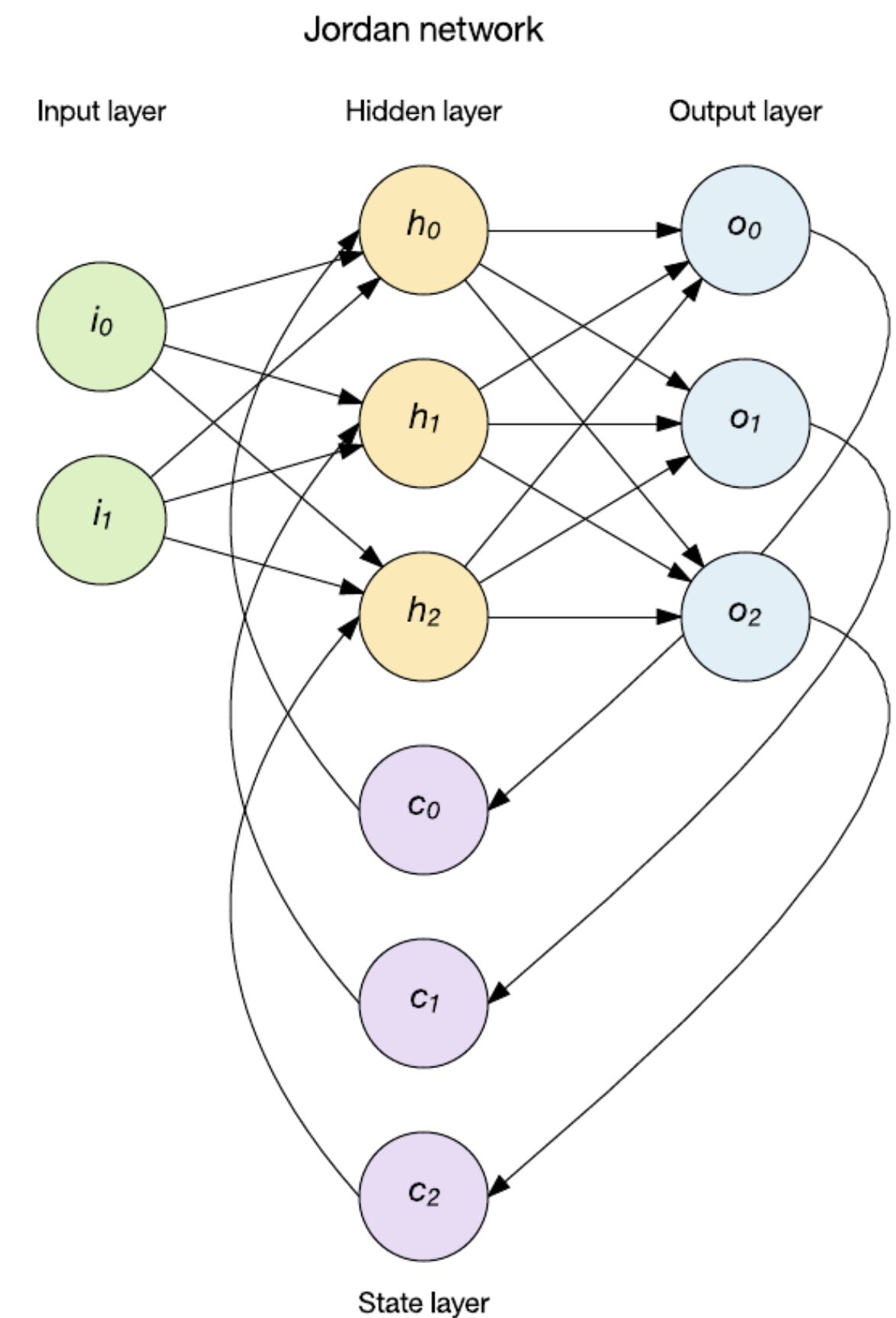
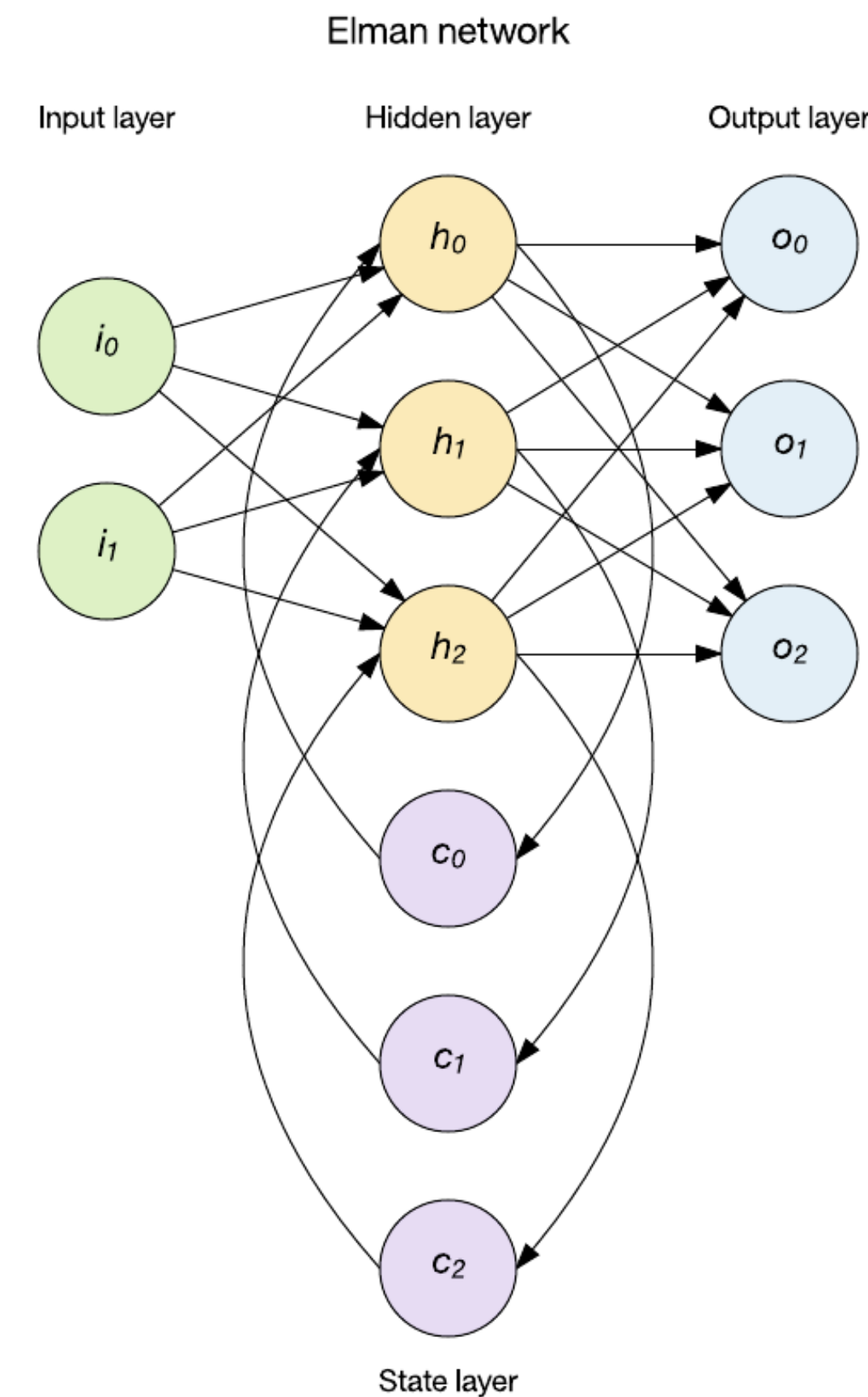
- Convolutional Neural Network



Redes neurais

- **Recurrent Neural Network**

- As redes neurais recorrentes (RNNs) são usadas principalmente para modelar **dados sequenciais e dependências temporais**.
- Diferente das redes neurais tradicionais (feedforward), as RNNs possuem **conexões recorrentes** que permitem que informações passadas influenciem o processamento do presente, tornando-as adequadas para tarefas que envolvem **contexto e memória**.
- **LSTMs** (Long Short-Term Memory) – melhor capacidade de manter memórias longas.



Redes neurais

- **Características das MLPs:**

Arquitetura feedforward

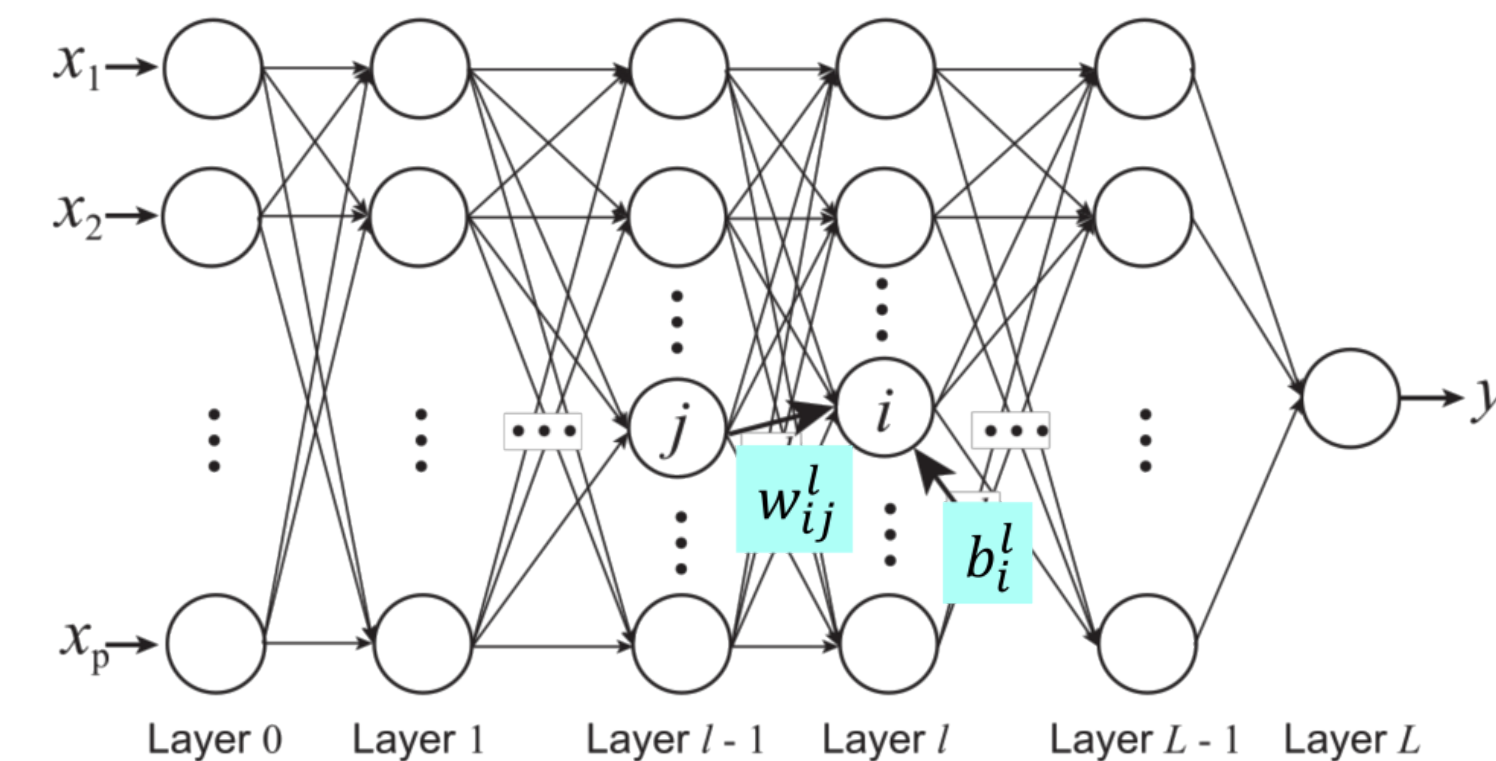
- Os dados fluem da camada de entrada para a(s) camada(s) oculta(s) e, por fim, para a camada de saída.
- Não há conexões recorrentes (como em RNNs).

Múltiplas camadas

- Possuem pelo menos uma camada oculta entre entrada e saída.
- Cada neurônio aplica uma transformação linear seguida de uma função de ativação não linear (como ReLU, sigmoid ou tanh).

Treinamento via retropropagação

- O aprendizado ocorre ajustando os pesos por meio do decréscimo do gradiente e backpropagation.

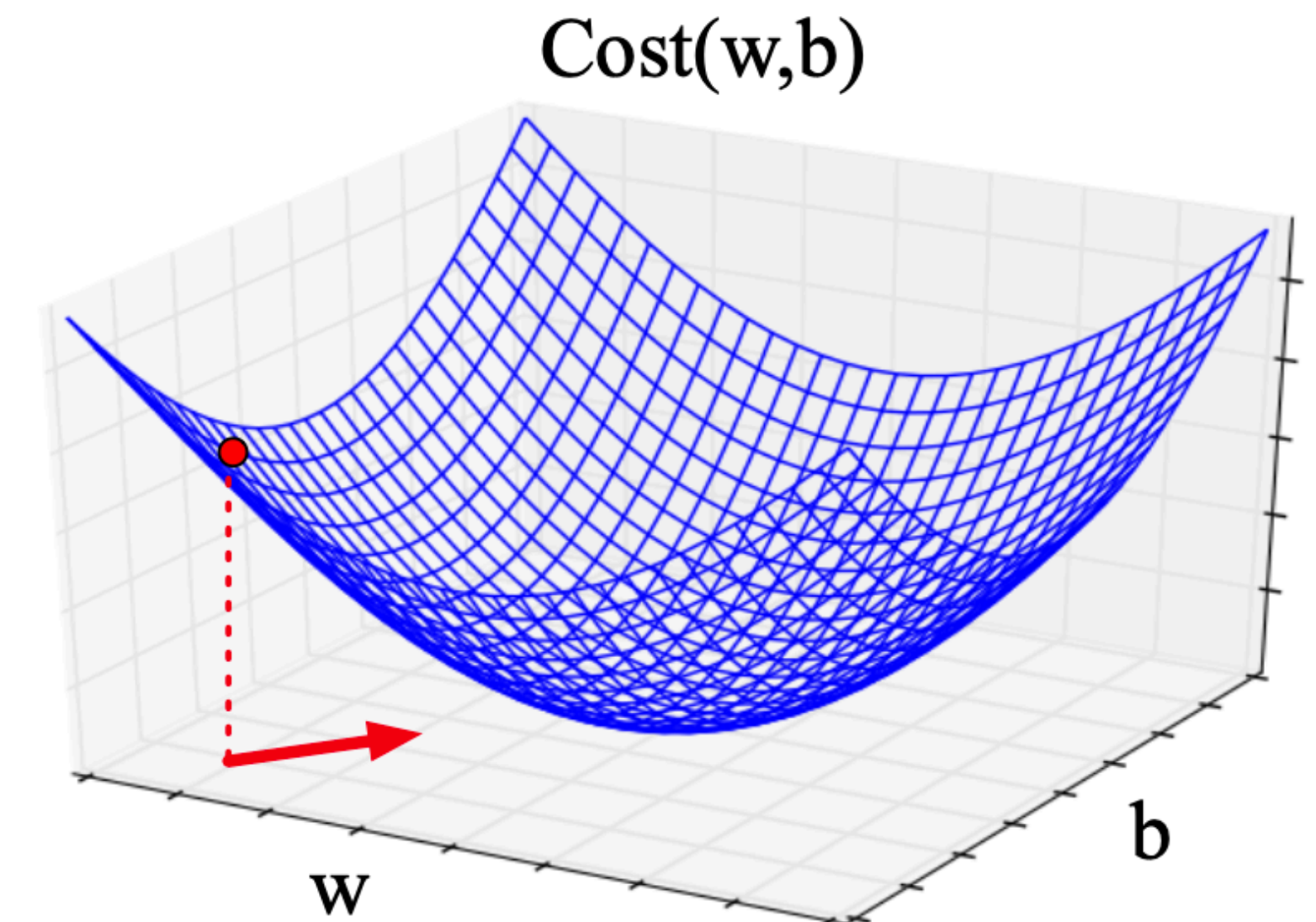
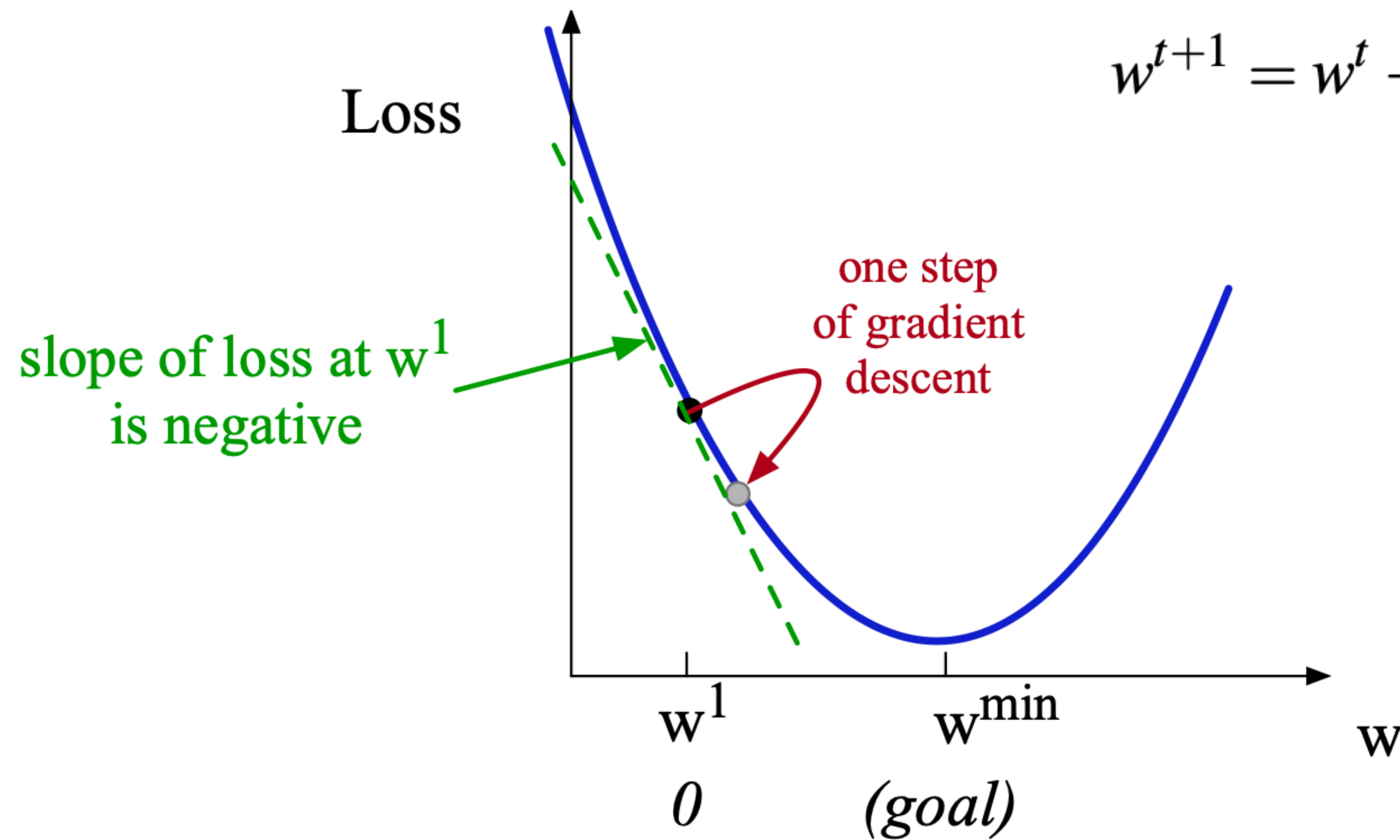


Redes neurais

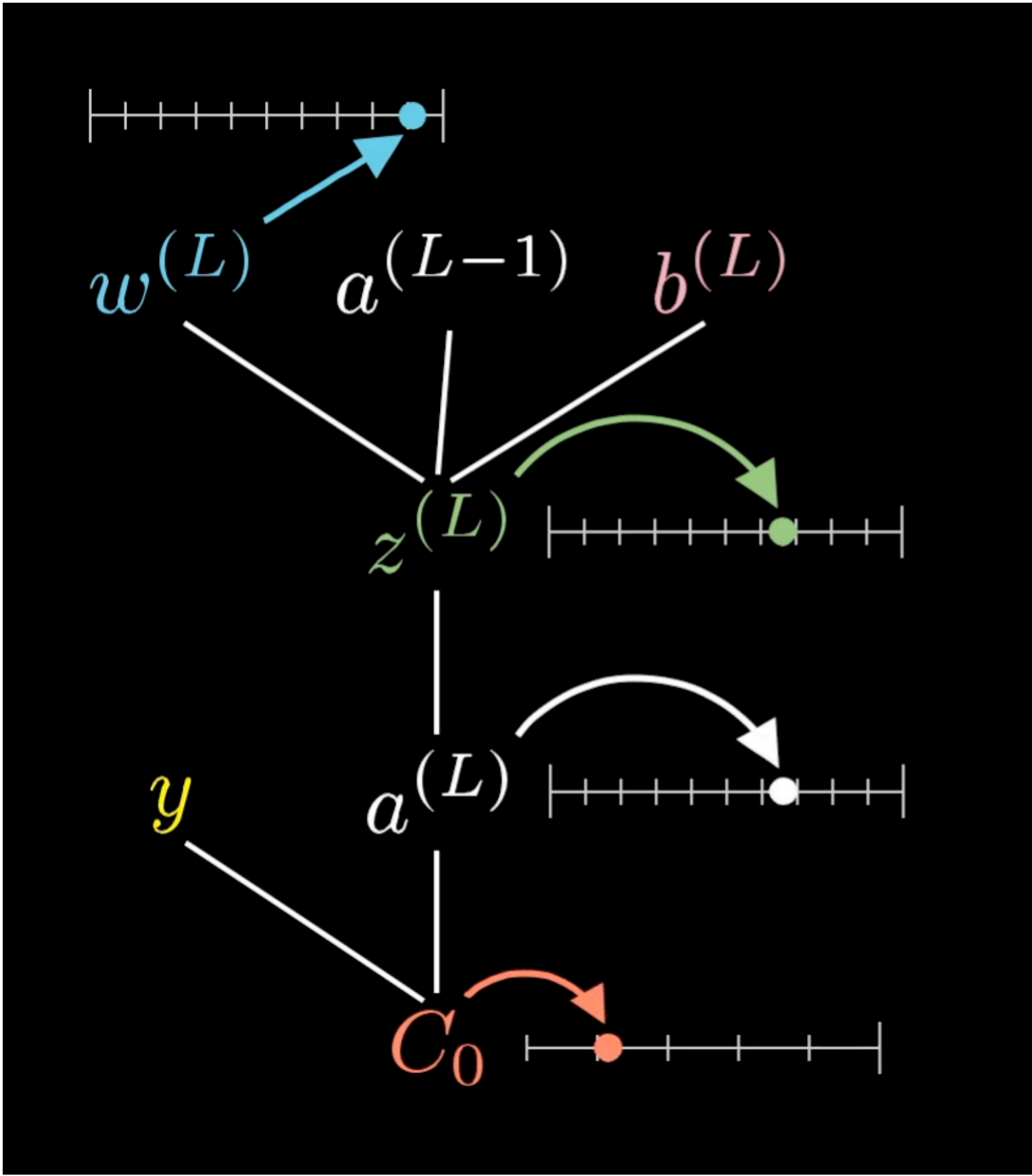
Treinamento: Backpropagation

$$\theta^{t+1} = \theta^t - \eta \nabla L(f(x; \theta), y)$$

$$w^{t+1} = w^t - \eta \frac{d}{dw} L(f(x; w), y)$$



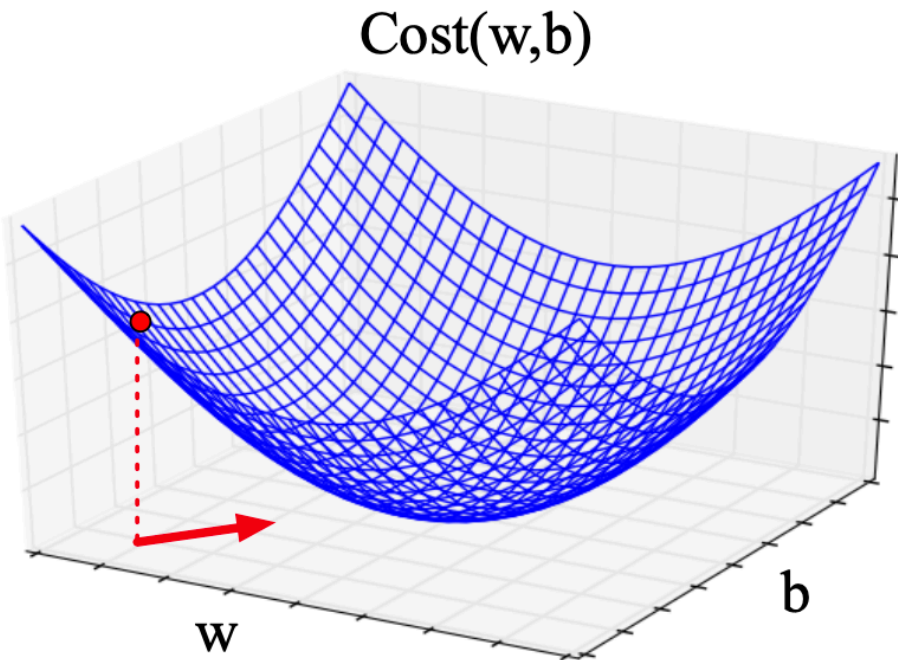
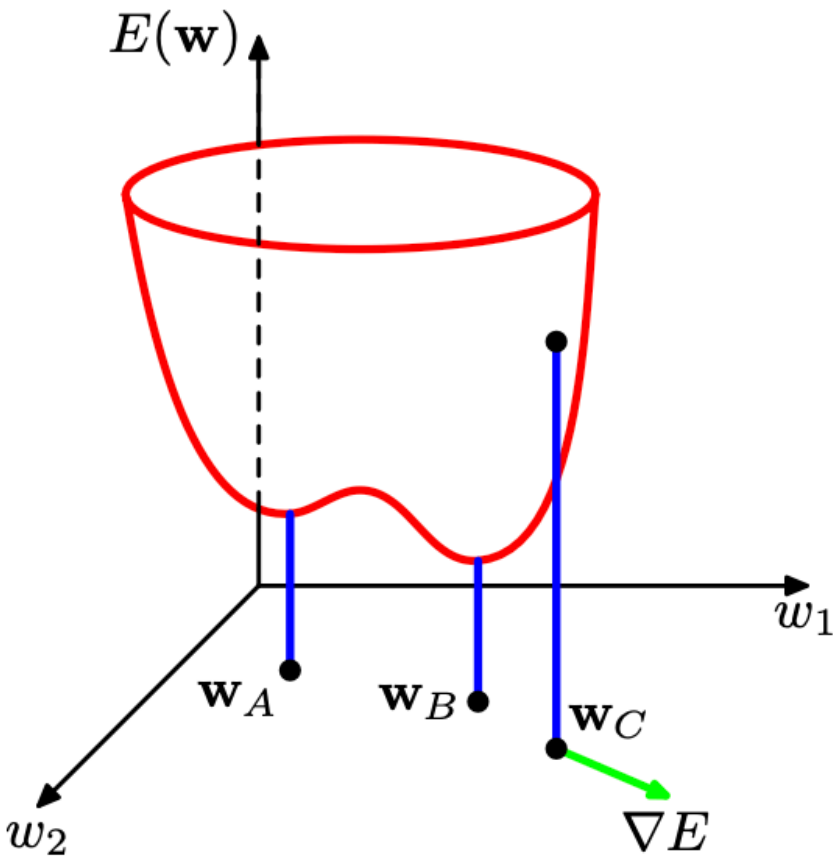
Redes neurais



How much does a nudge to $w^{(L)}$ change $z^{(L)}$? How much does that nudge to $z^{(L)}$ change $a^{(L)}$?

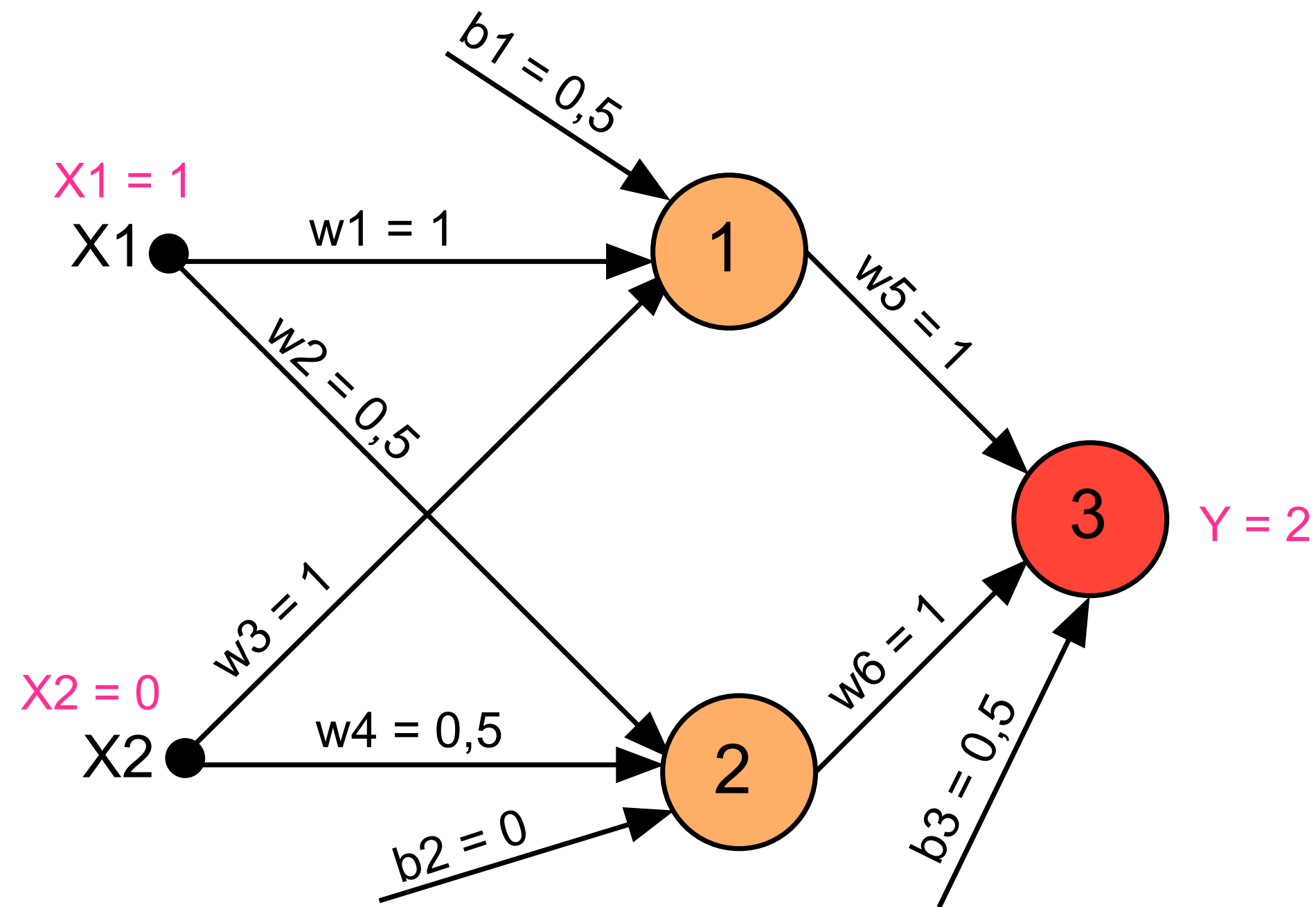
$$\frac{\partial C_0}{\partial w^{(L)}} = \frac{\partial z^{(L)}}{\partial w^{(L)}} \frac{\partial a^{(L)}}{\partial z^{(L)}} \frac{\partial C_0}{\partial a^{(L)}}$$

How much does *that* nudge to $a^{(L)}$ change C_0 ?



<https://www.3blue1brown.com/lessons/backpropagation-calculus>

Redes neurais



X1	X2	Y
1	0	2
2	1	6
...	...	...

Forward

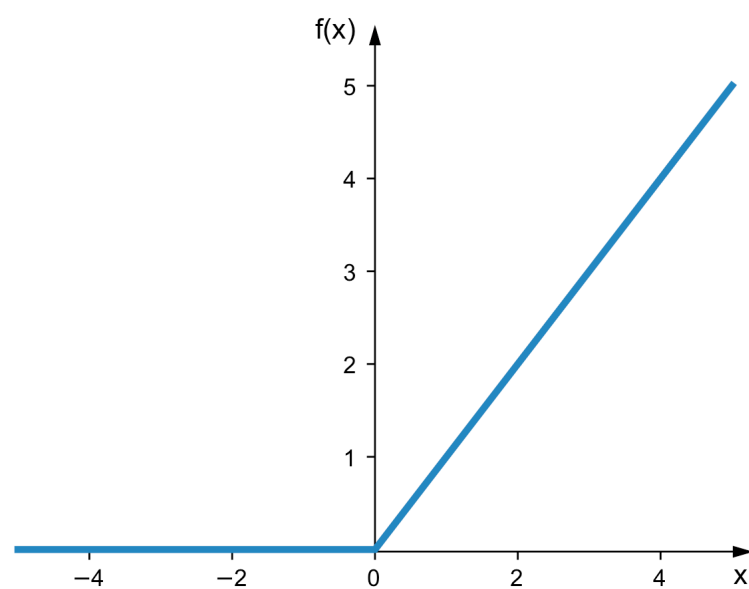
$$h_1 = \text{RELU}(1w_1 + 0w_3 + b_1 = 1,5) = 1,5$$

$$h_2 = \text{RELU}(1w_2 + 0w_4 + b_2 = 0,5) = 0,5$$

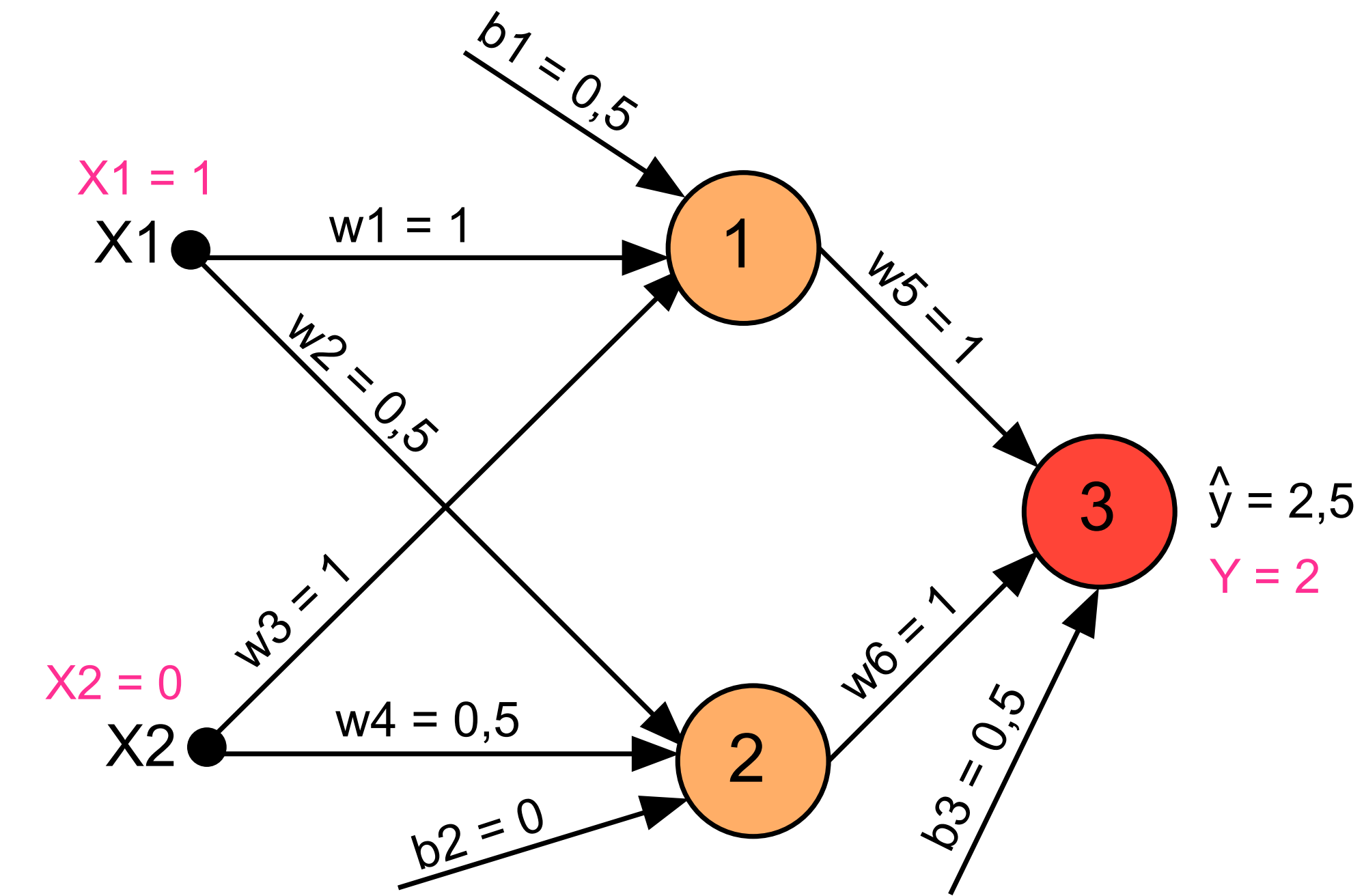


$$h_3 = \hat{y} = h_1w_5 + h_2w_6 + b_3 = 2,5$$

$$L = (\hat{y} - y)^2 = (2,5 - 2)^2 = 0,25$$



Redes neurais



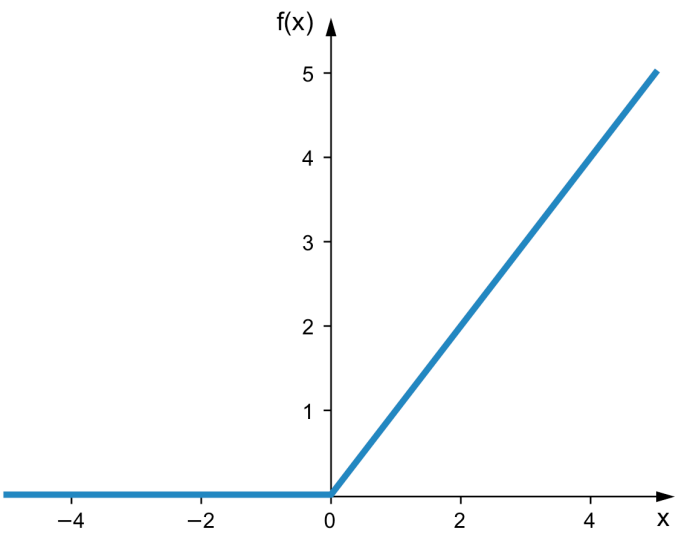
X1	X2	Y
1	0	2
2	1	6
...	...	...

Backward

$$\frac{\partial L}{\partial w_5} = ?$$

$$h_3 = \hat{y} = h_1 w_5 + h_2 w_6 + b_3$$

$$L = (\hat{y} - y)^2 = (h_1 w_5 + h_2 w_6 + b_3 - y)^2$$

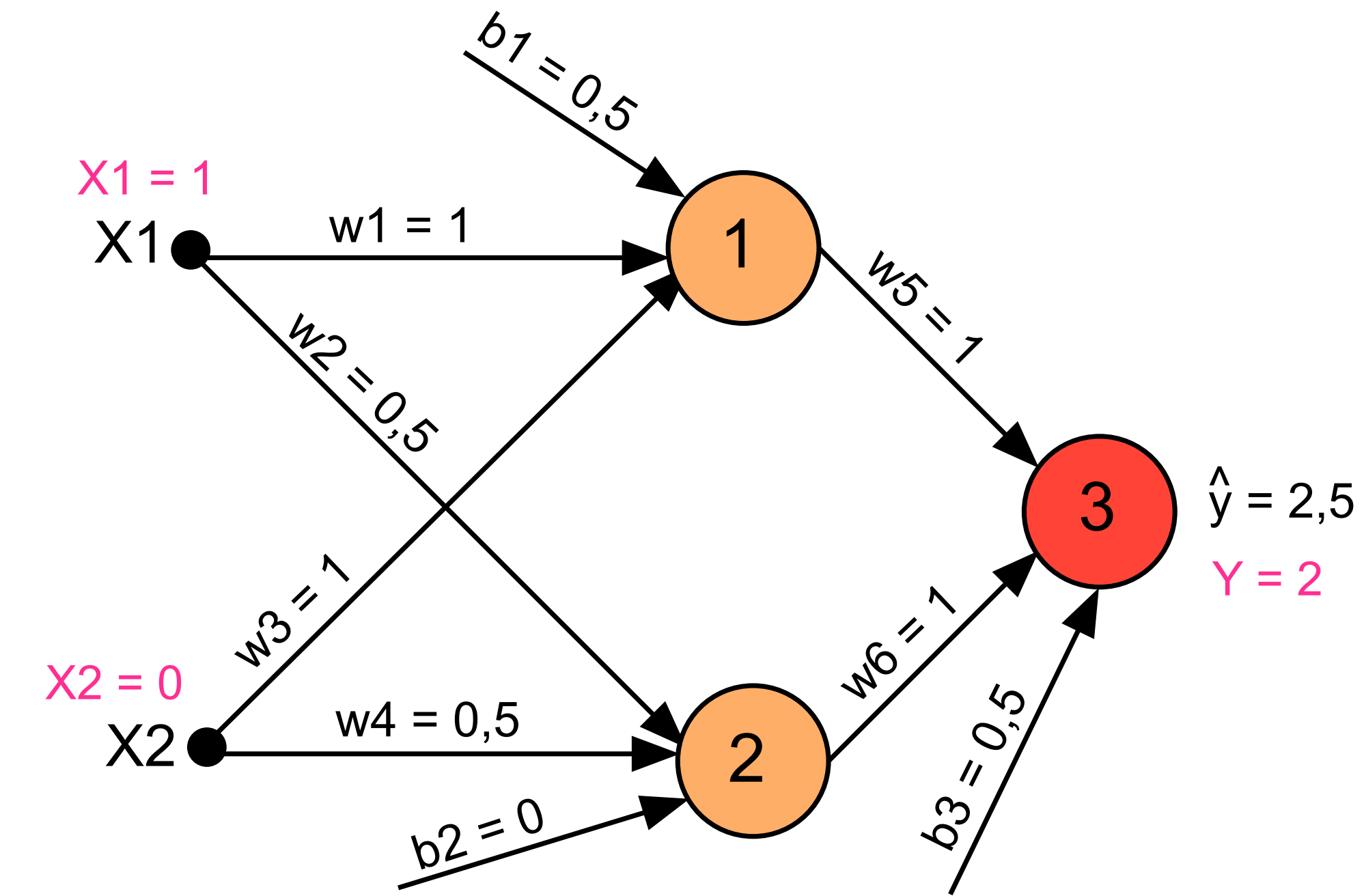


$$\hat{y} = 2,5 \quad y = 2$$

$$\frac{\partial L}{\partial w_5} = 2h_1 (h_1 w_5 + h_2 w_6 + b_3 - y) = 2 \times 1,5 \times (2,5 - 2) = 1,5$$

$$\frac{\partial L}{\partial w_5} = 1,5$$

Redes neurais

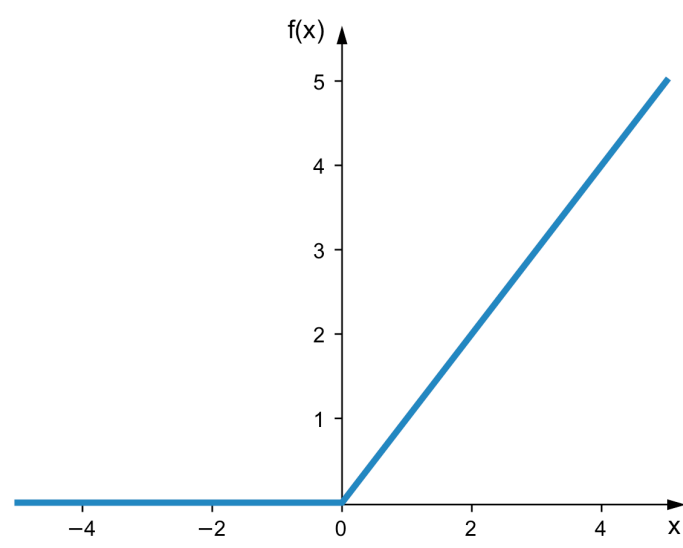


X1	X2	Y
1	0	2
2	1	6
...	...	...

Backward

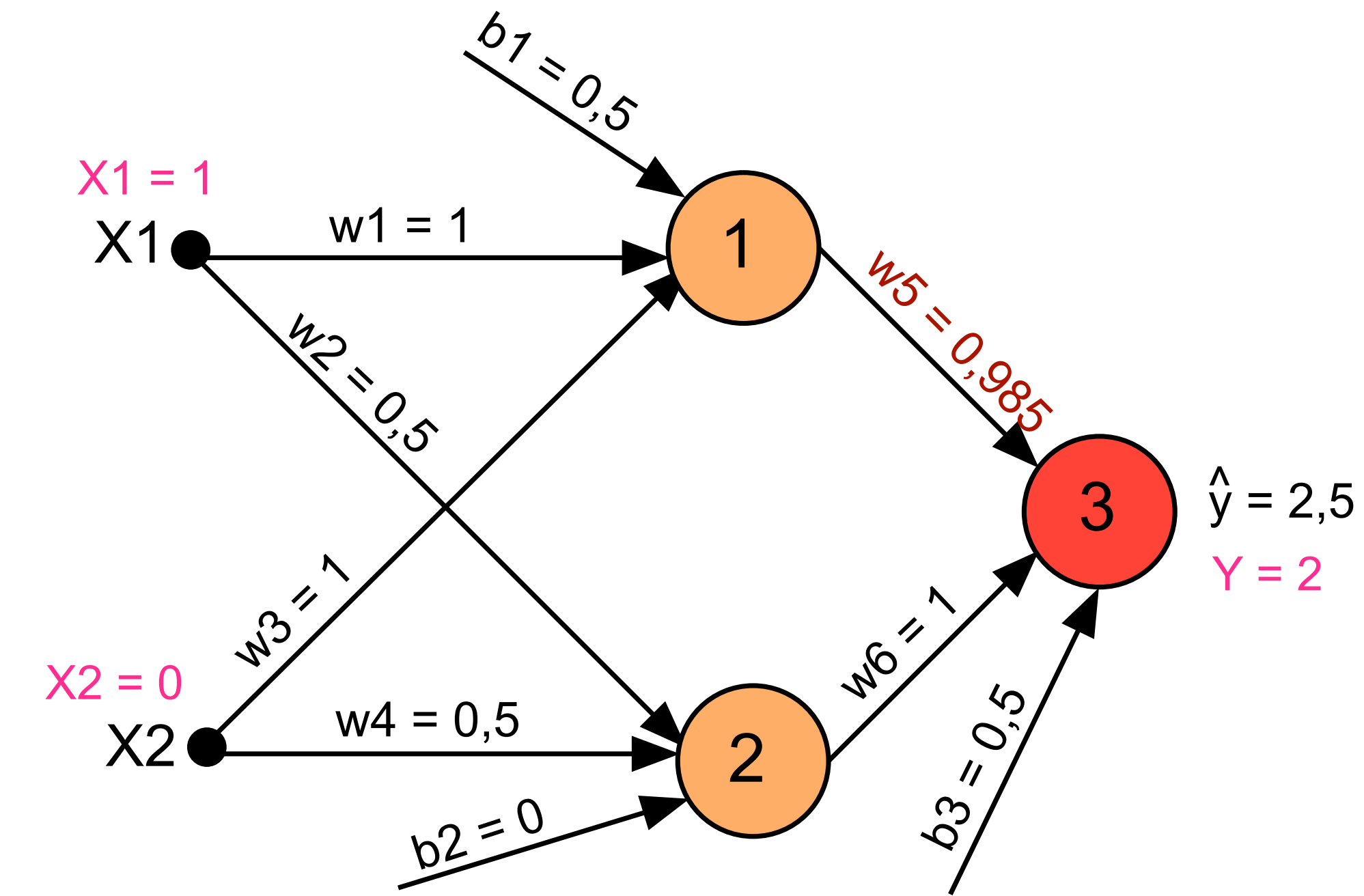
$$\frac{\partial L}{\partial w_5} = 1,5$$

$$w_5 = w_5 - \eta \frac{\partial L}{\partial w_5} = 1 - 0,01 \times 1,5 = 0,985$$



$$\hat{y} = 2,5 \quad y = 2$$

Redes neurais

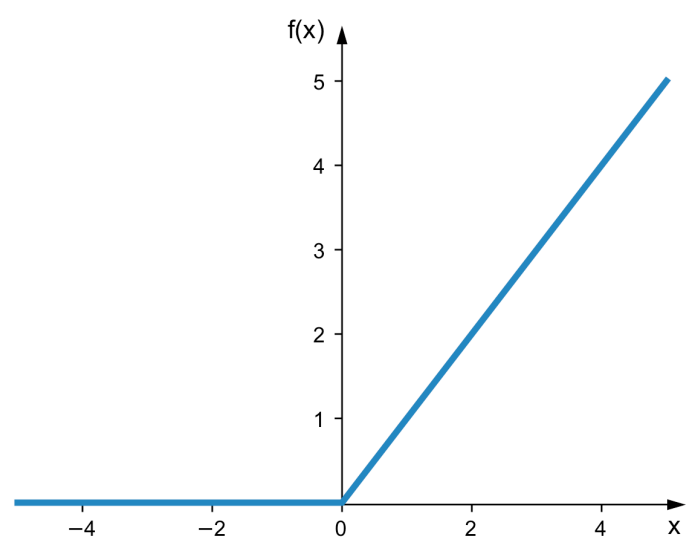


Backward

$$\frac{\partial L}{\partial w_5} = 1,5$$

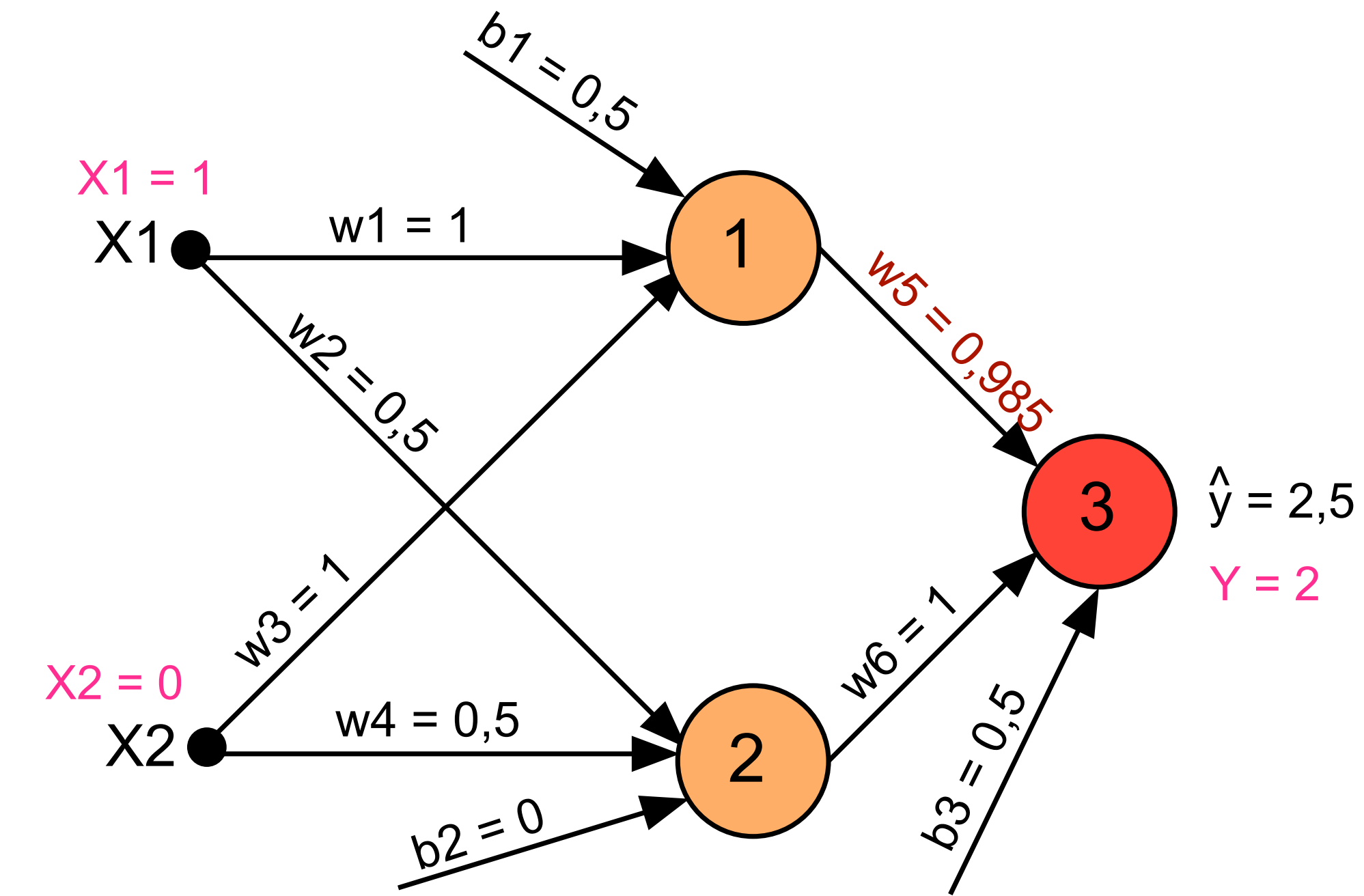
$$w_5 = w_5 - \eta \frac{\partial L}{\partial w_5} = 1 - 0,01 \times 1,5 = 0,985$$

X1	X2	Y
1	0	2
2	1	6
...	...	...



$$\hat{y} = 2,5 \quad y = 2$$

Redes neurais



X1	X2	Y
1	0	2
2	1	6
...	...	...

Backward

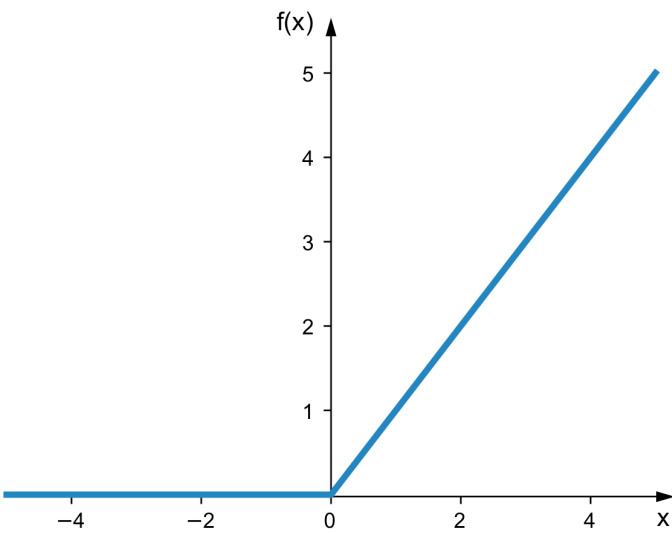
$$\frac{\partial L}{\partial b_3} = ?$$

$$L = (\hat{y} - y)^2 = (h_1 w_5 + h_2 w_6 + b_3 - y)^2$$

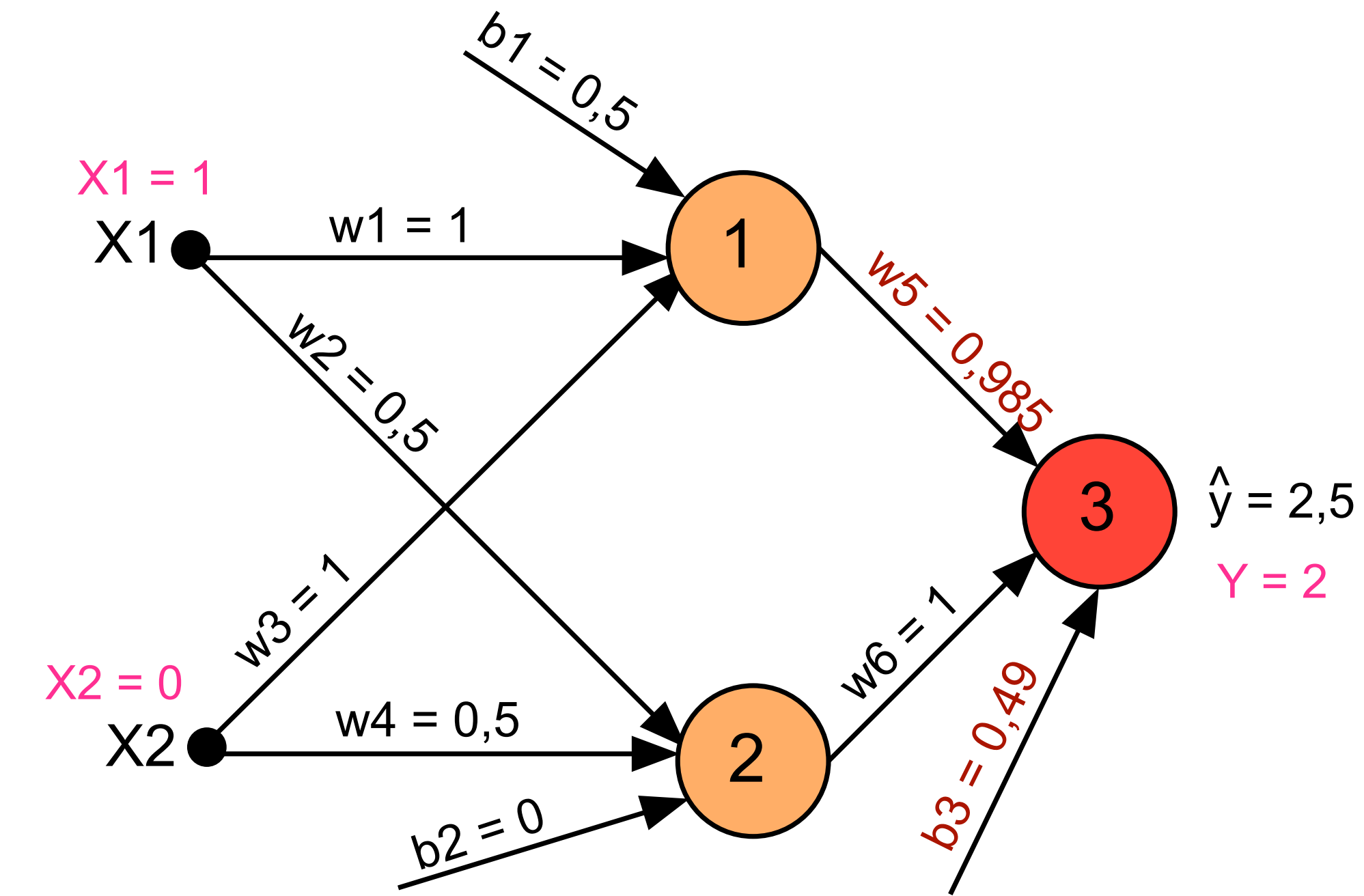
$$\frac{\partial L}{\partial b_3} = 2(1)(h_1 w_5 + h_2 w_6 + b_3 - y) = 2(1)(2,5 - 2) = 1$$

$$\frac{\partial L}{\partial b_3} = 1$$

$$\hat{y} = 2,5 \quad y = 2$$



Redes neurais

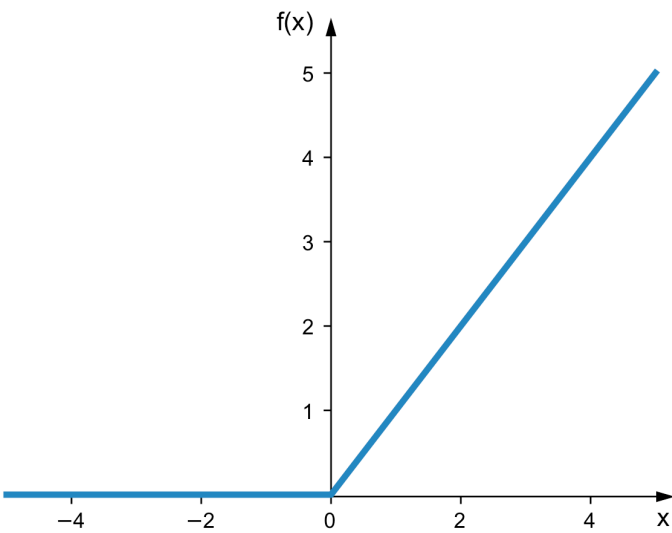


X1	X2	Y
1	0	2
2	1	6
...	...	...

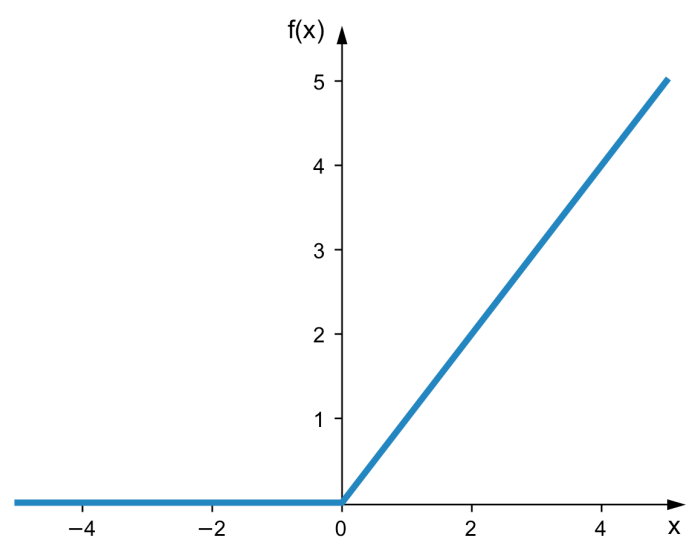
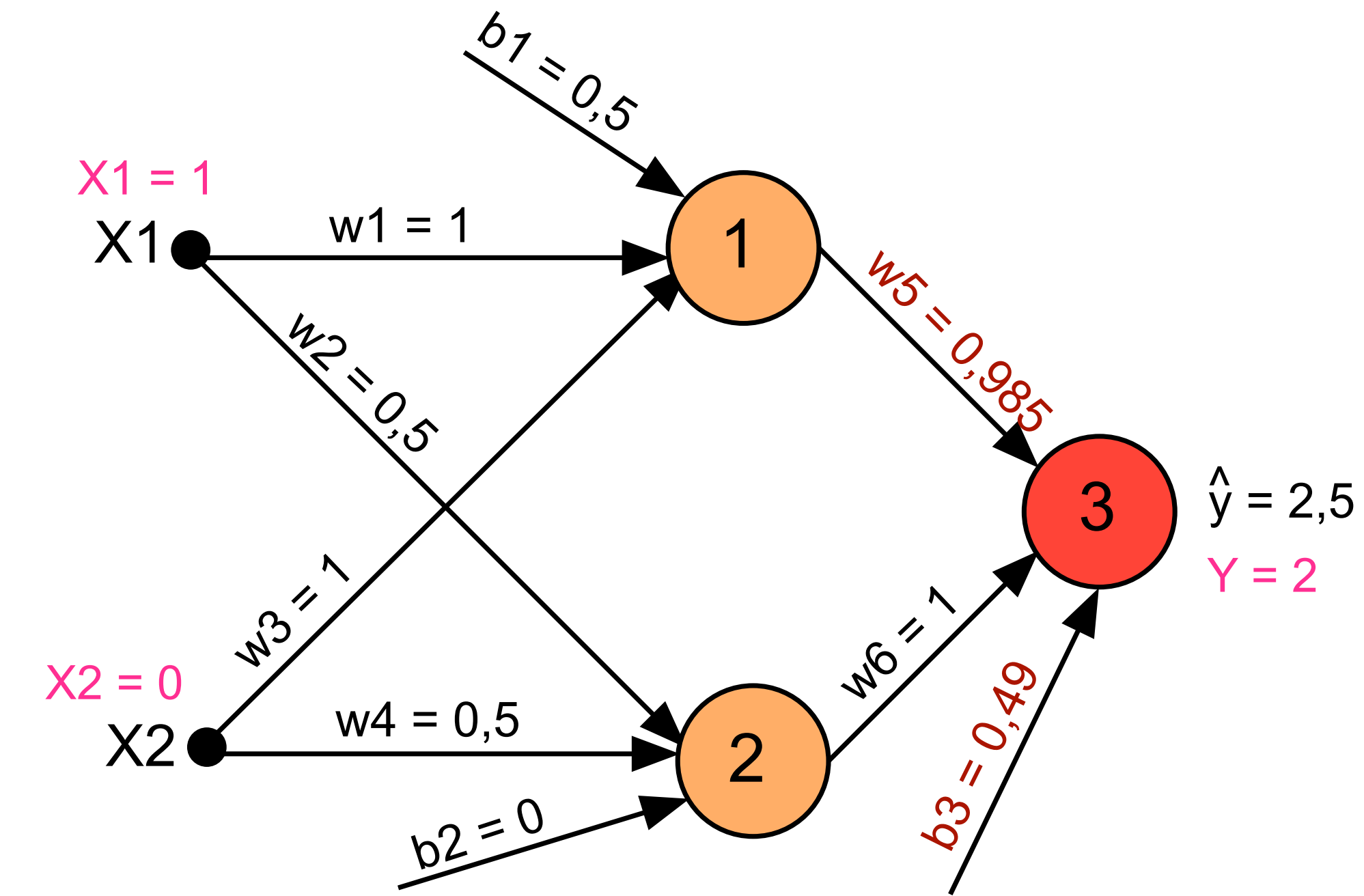
Backward

$$\frac{\partial L}{\partial b_3} = 1$$

$$b_3 = b_3 - \eta \frac{\partial L}{\partial b_3} = 0,5 - 0,01 \times 1 = 0,49$$



Redes neurais



Backward

$$\frac{\partial L}{\partial w_1} = ?$$

$$\hat{y} = 2,5 \quad y = 2$$
$$h_1 = \text{RELU}(1w_1 + 0w_3 + b_1)$$

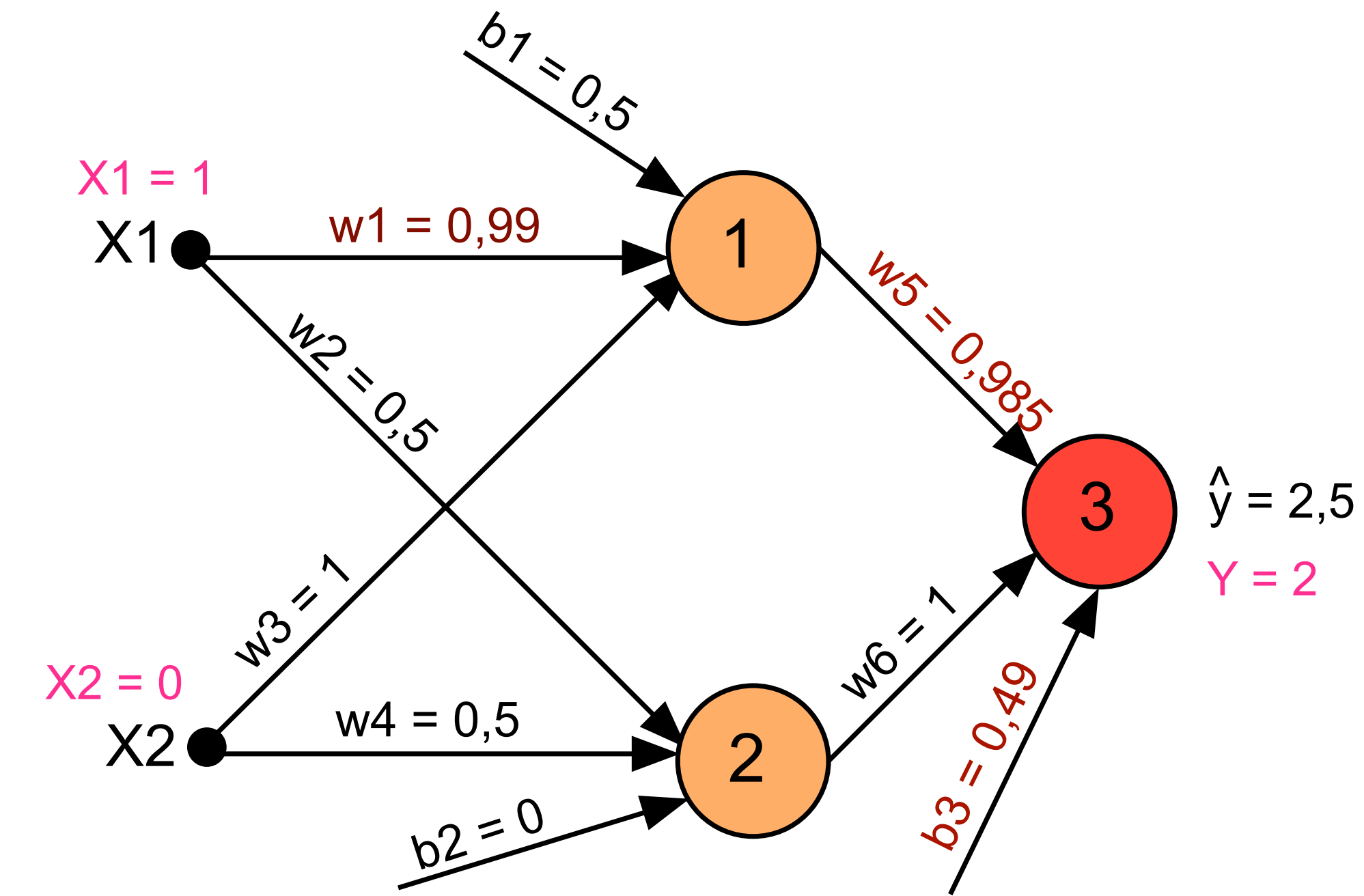
$$L = (h_1w_5 + h_2w_6 + b_3 - y)^2$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1} = 1$$

$$\frac{\partial L}{\partial h_1} = 2w_5(h_1w_5 + h_2w_6 + b_3 - y) = 2 \times 1 \times 0,5 = 1$$

$$\frac{\partial h_1}{\partial w_1} = \frac{\partial}{\partial w_1}(\text{RELU}(1w_1 + 0w_3 + b_1)) = 1$$

Redes neurais

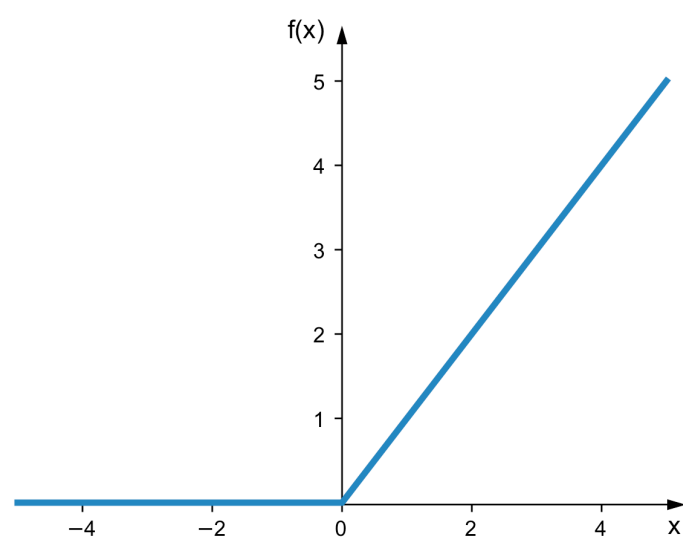


Backward

$$\frac{\partial L}{\partial w_1} = 1$$

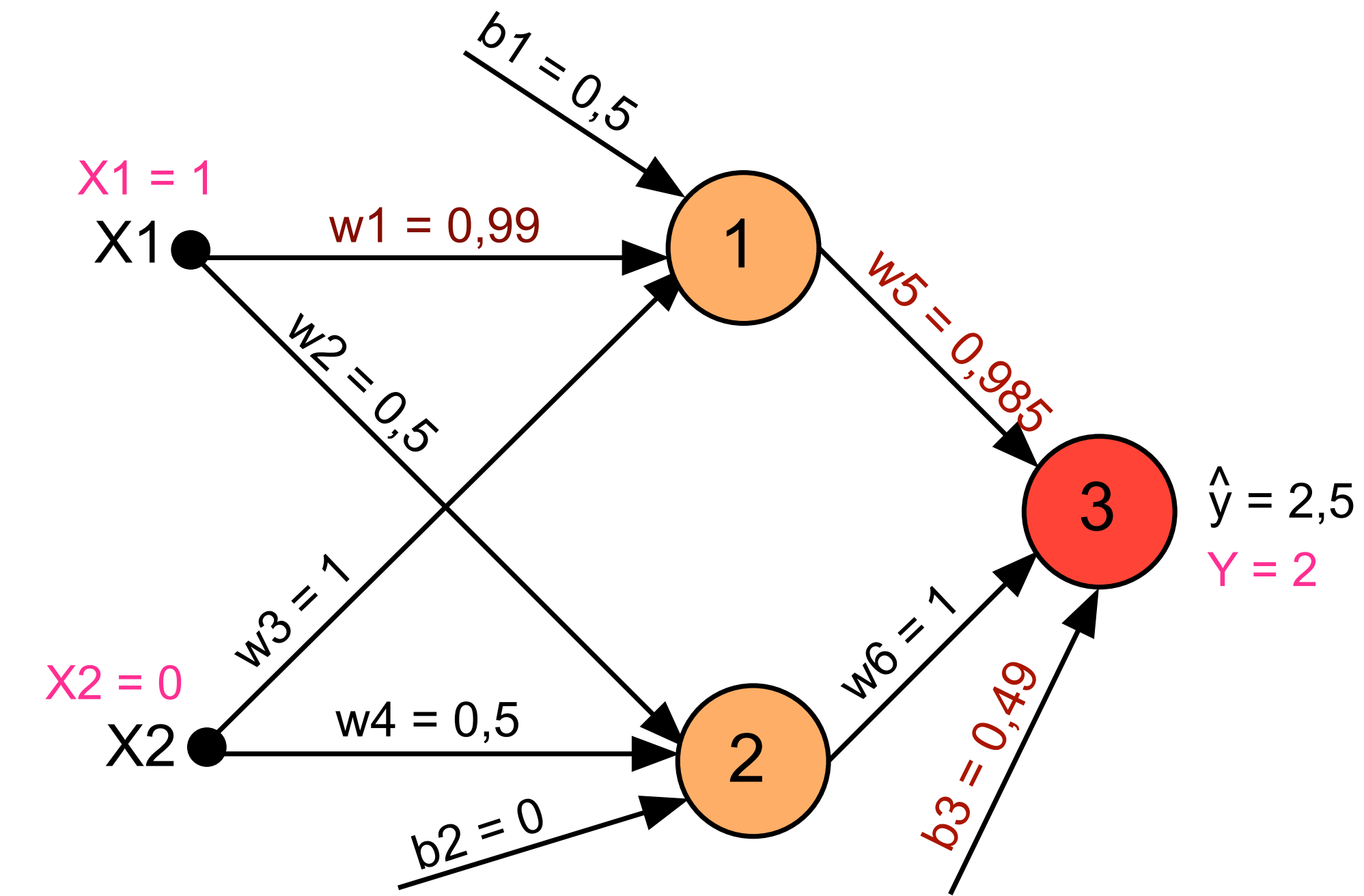
$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1} = 1 - 0,01 \times 1 = 0,99$$

$$\hat{y} = 2,5 \quad y = 2$$
$$h_1 = \text{RELU}(1w_1 + 0w_3 + b_1)$$



X1	X2	Y
1	0	2
2	1	6
...	...	...

Redes neurais



Backward

$$\hat{y} = 2,5 \quad y = 2$$
$$h_1 = \text{RELU}(1w_1 + 0w_3 + b_1)$$

$$\frac{\partial L}{\partial b_1} = ?$$

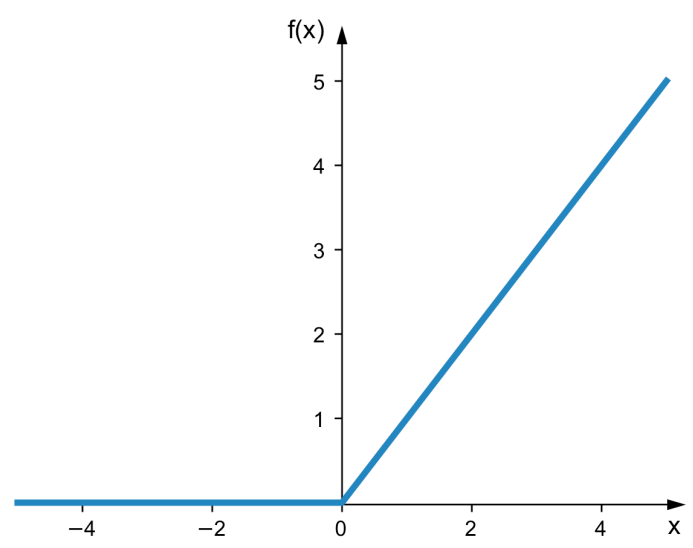
$$L = (h_1w_5 + h_2w_6 + b_3 - y)^2$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial b_1} = 1$$

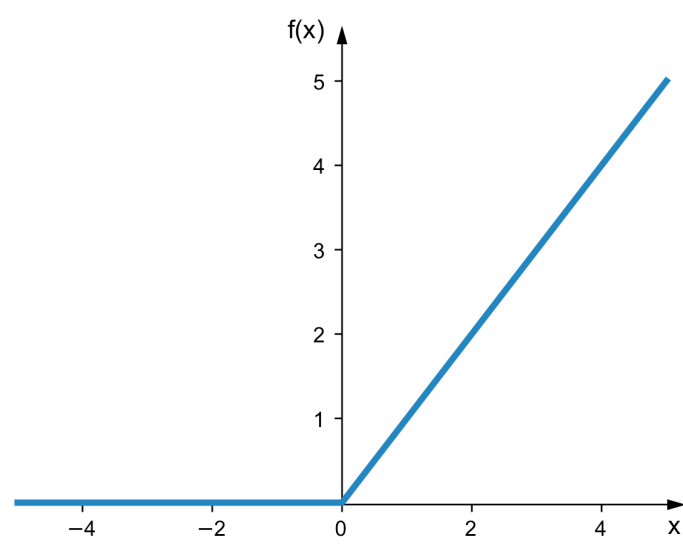
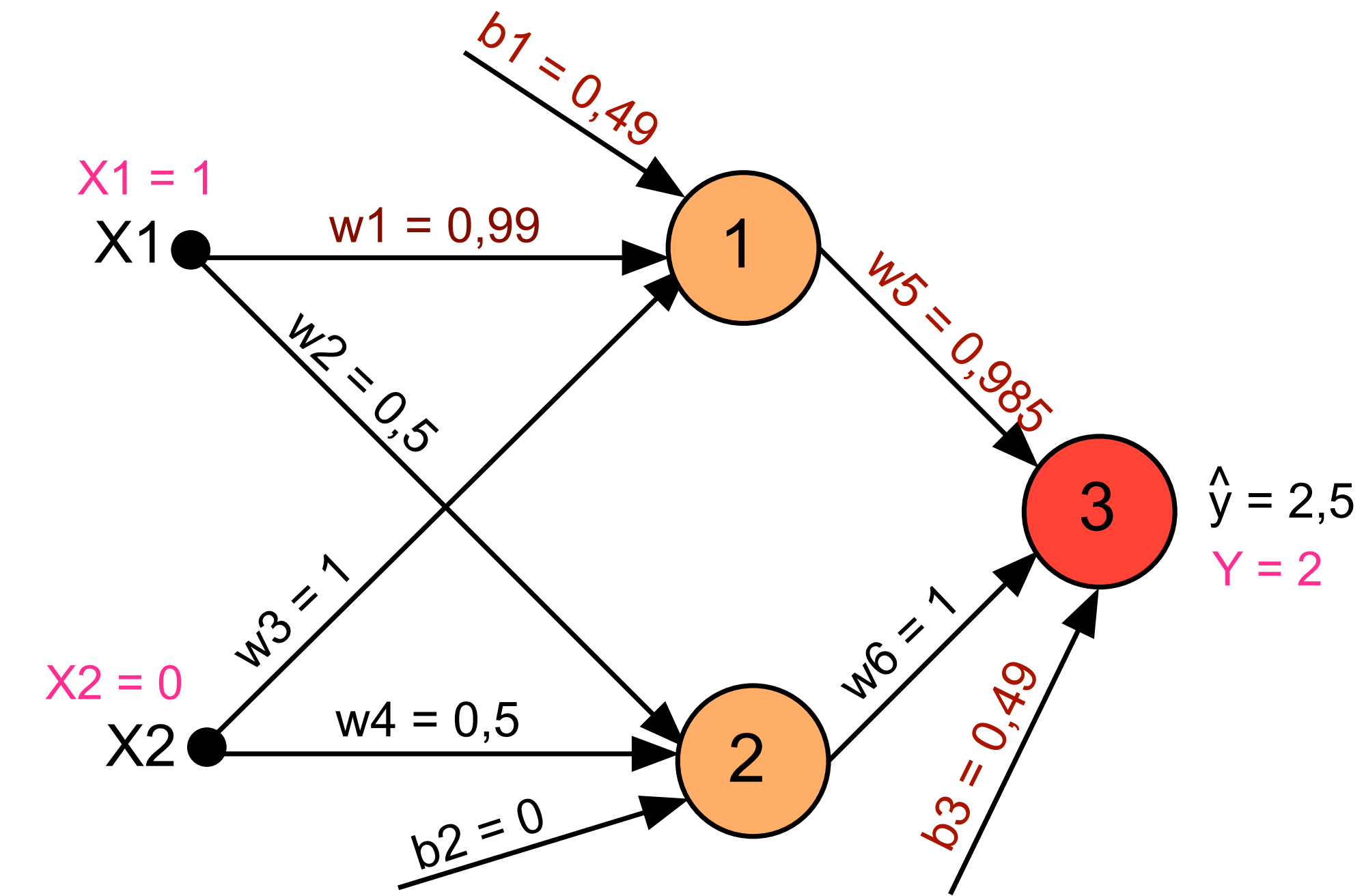
$$\frac{\partial L}{\partial h_1} = 2w_5(h_1w_5 + h_2w_6 + b_3 - y) = 2 \times 1 \times 0,5 = 1$$

$$\frac{\partial h_1}{\partial b_1} = \frac{\partial}{\partial b_1}(\text{RELU}(1w_1 + 0w_3 + b_1)) = 1$$

$$\frac{\partial L}{\partial b_1} = 1$$



Redes neurais



Backward

$$\hat{y} = 2,5 \quad y = 2$$

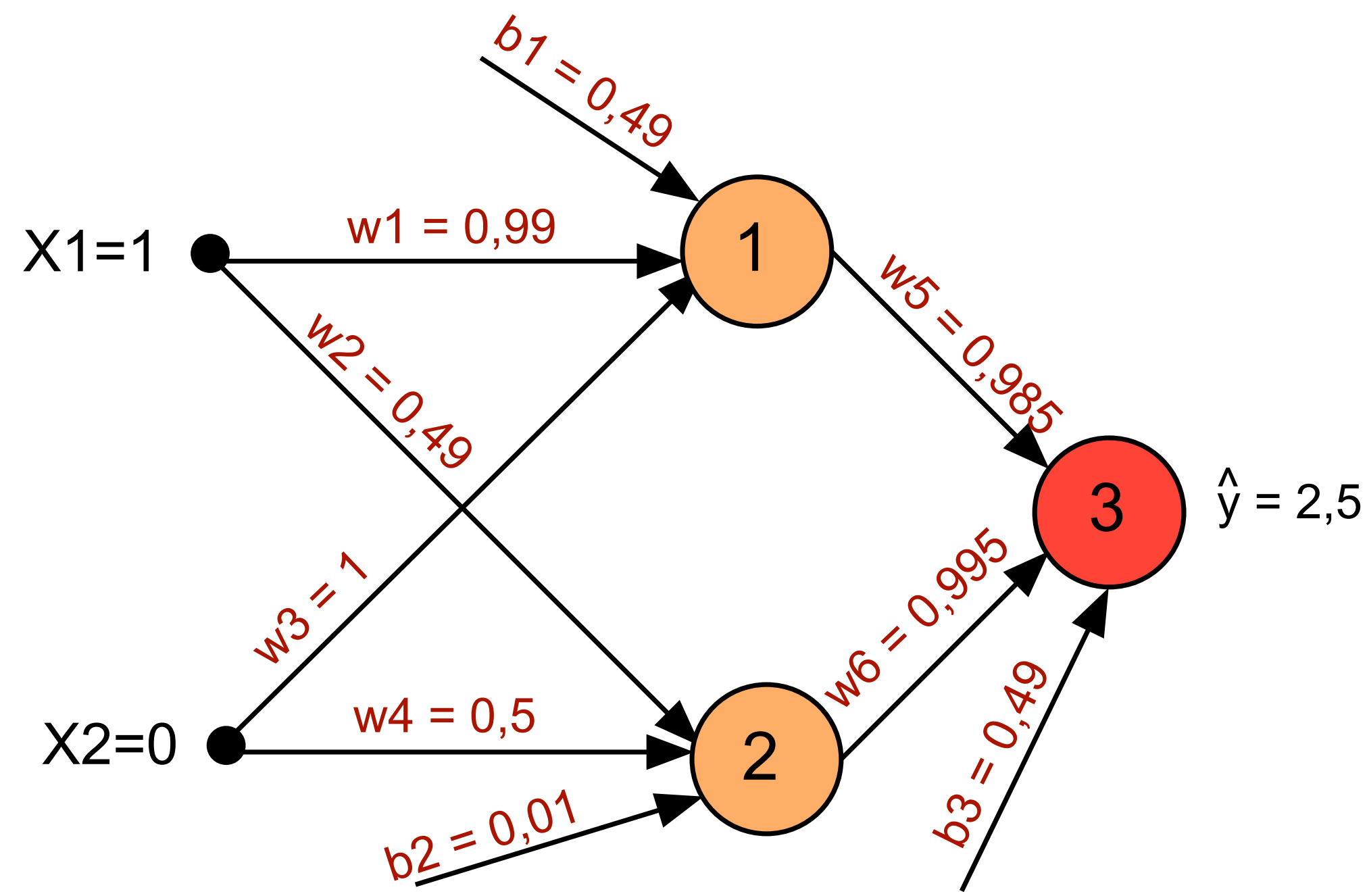
$$h_1 = \text{RELU}(1w_1 + 0w_3 + b_1)$$

$$\frac{\partial L}{\partial b_1} = 1$$

$$b_1 = b_1 - \eta \frac{\partial L}{\partial b_1} = 0,5 - 0,01 \times 1 = 0,49$$

X1	X2	Y
1	0	2
2	1	6
...	...	...

Redes neurais



X1	X2	Y
1	0	2
2	1	6
...	...	...

Redes neurais

Uma época ocorre quando o modelo **vê todos os exemplos de treinamento uma vez** e atualiza seus pesos com base nesses dados.

Suponha que você tenha:

- 1.000 amostras de treinamento
- Treinando uma rede neural

Se o modelo processar todas as 1.000 amostras uma vez, isso corresponde a: 1 época

Se repetir o processo novamente: 2 épocas

E assim por diante.

Redes neurais

Relação com batch (lote)

Na prática, os dados não são enviados de uma vez, mas em **lotes (batches)**.

Exemplo:

- 1.000 amostras
- batch size = 100

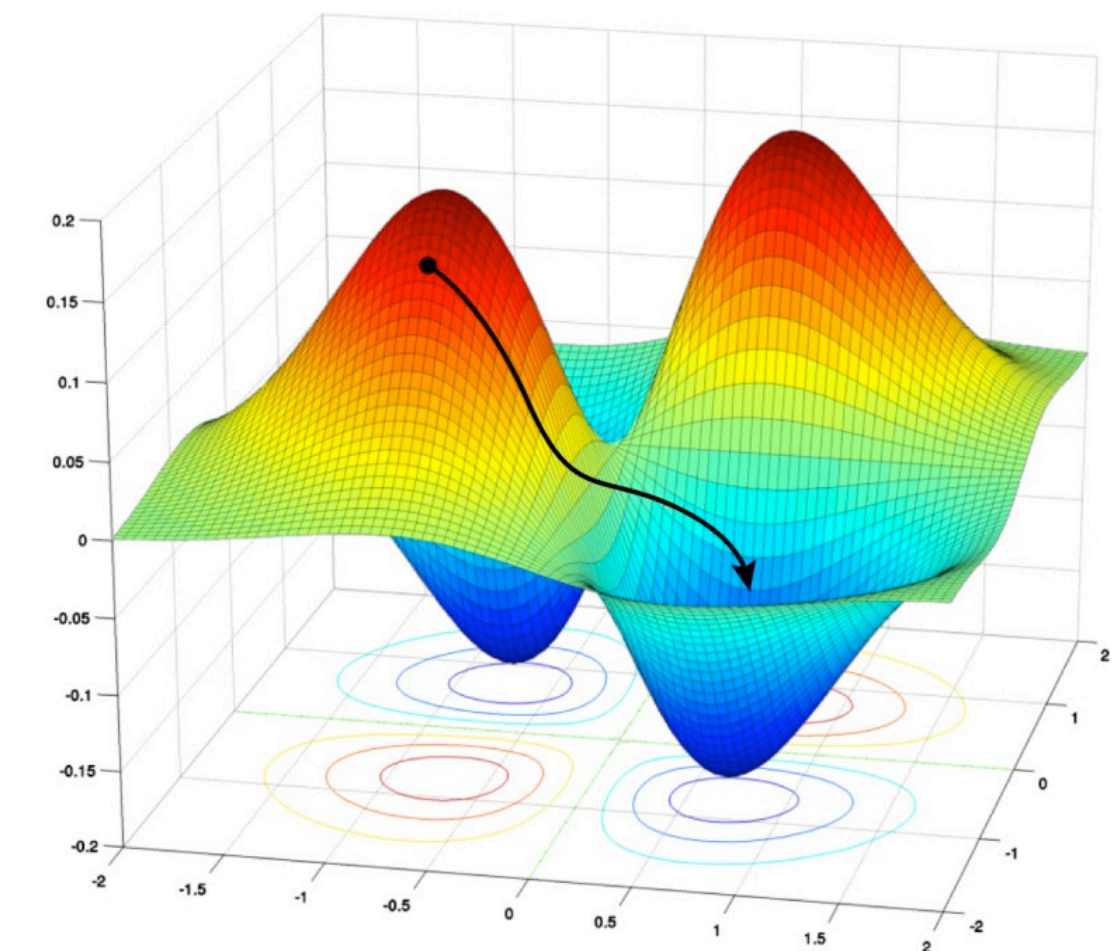
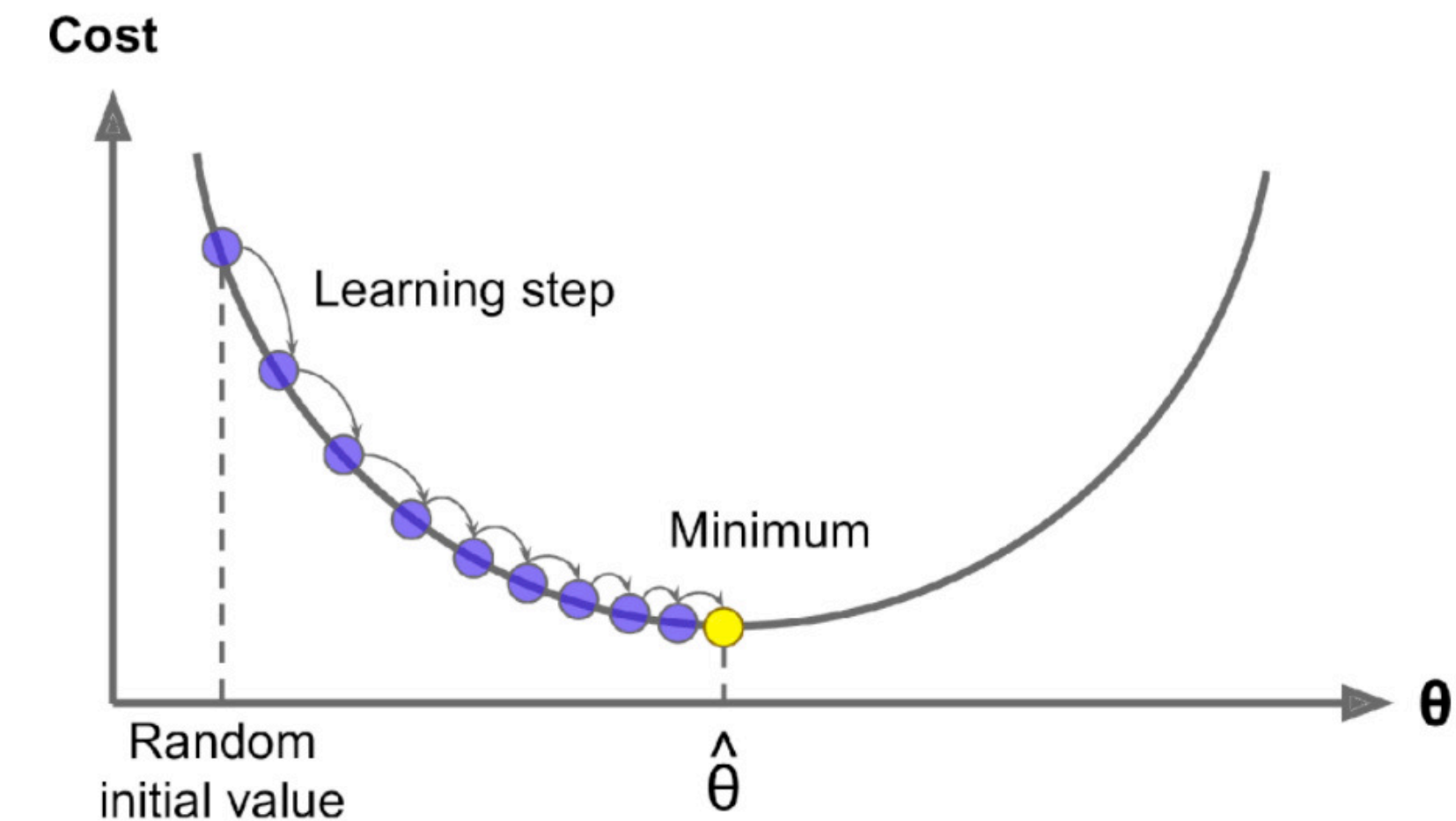
Então:

- 1 época = 10 batches
- Cada batch atualiza os pesos uma vez
- Após os 10 batches → completou 1 época

Redes neurais

1. Batch Gradient Descent (GD ou BGD)

- Calcula o gradiente usando **todos os exemplos** do conjunto de dados antes de atualizar os parâmetros.
- Vantagens: Convergência estável e direção precisa.
- Desvantagens: Alto custo computacional em datasets grandes.



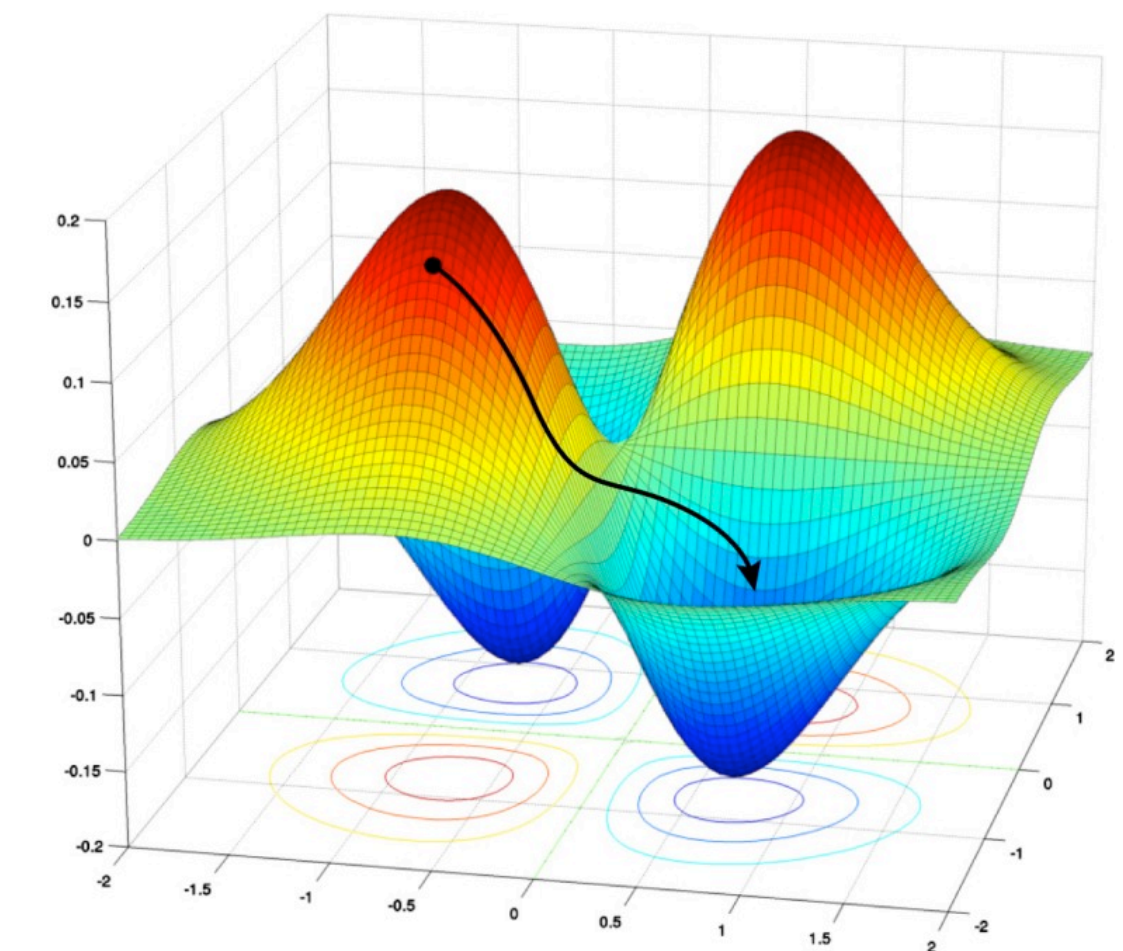
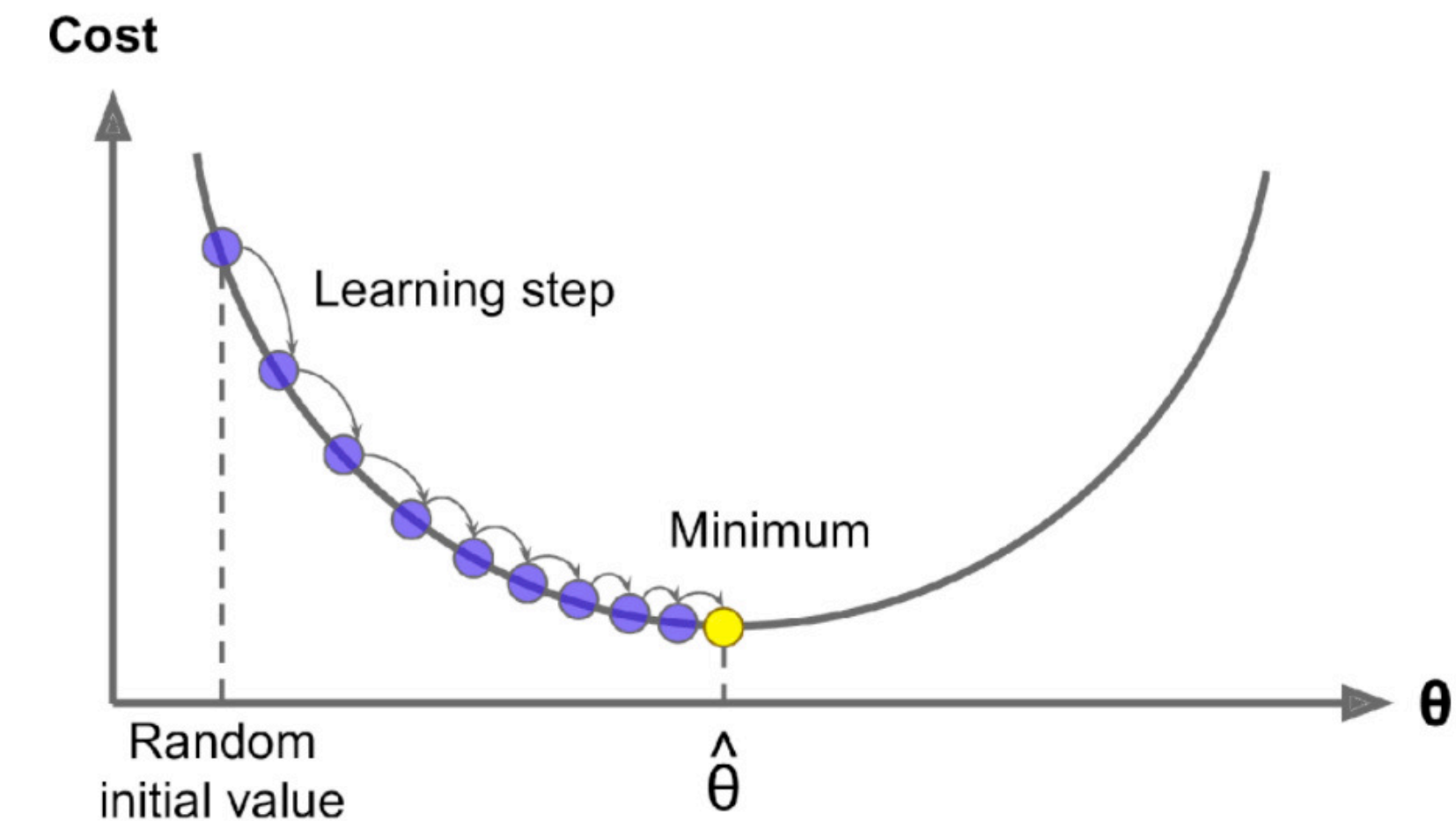
Redes neurais

1. Batch Gradient Descent (GD ou BGD)

- Calcula o gradiente usando **todos os exemplos** do conjunto de dados antes de atualizar os parâmetros.
- Vantagens: Convergência estável e direção precisa.
- Desvantagens: Alto custo computacional em datasets grandes.

2. Stochastic Gradient Descent (SGD)

- Atualiza os parâmetros após **cada amostra** do dataset, tornando o processo mais ruidoso. Ou seja, Pega 1 exemplo, calcula o erro, ajusta o modelo e repete.
- Vantagens: Pode escapar de mínimos locais e convergir mais rapidamente em certos casos.
- Desvantagens: Oscilações na convergência e maior variabilidade nos updates.



Redes neurais

1. Batch Gradient Descent (GD ou BGD)

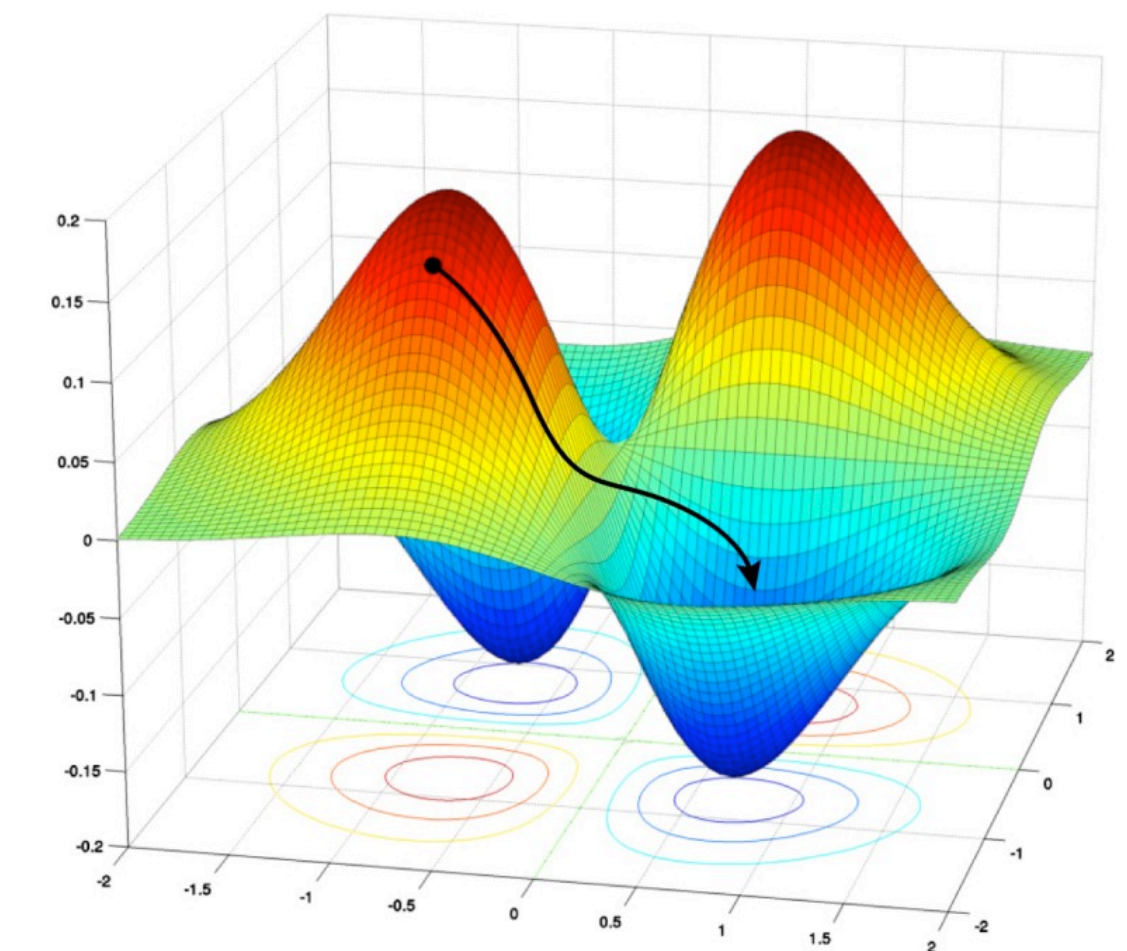
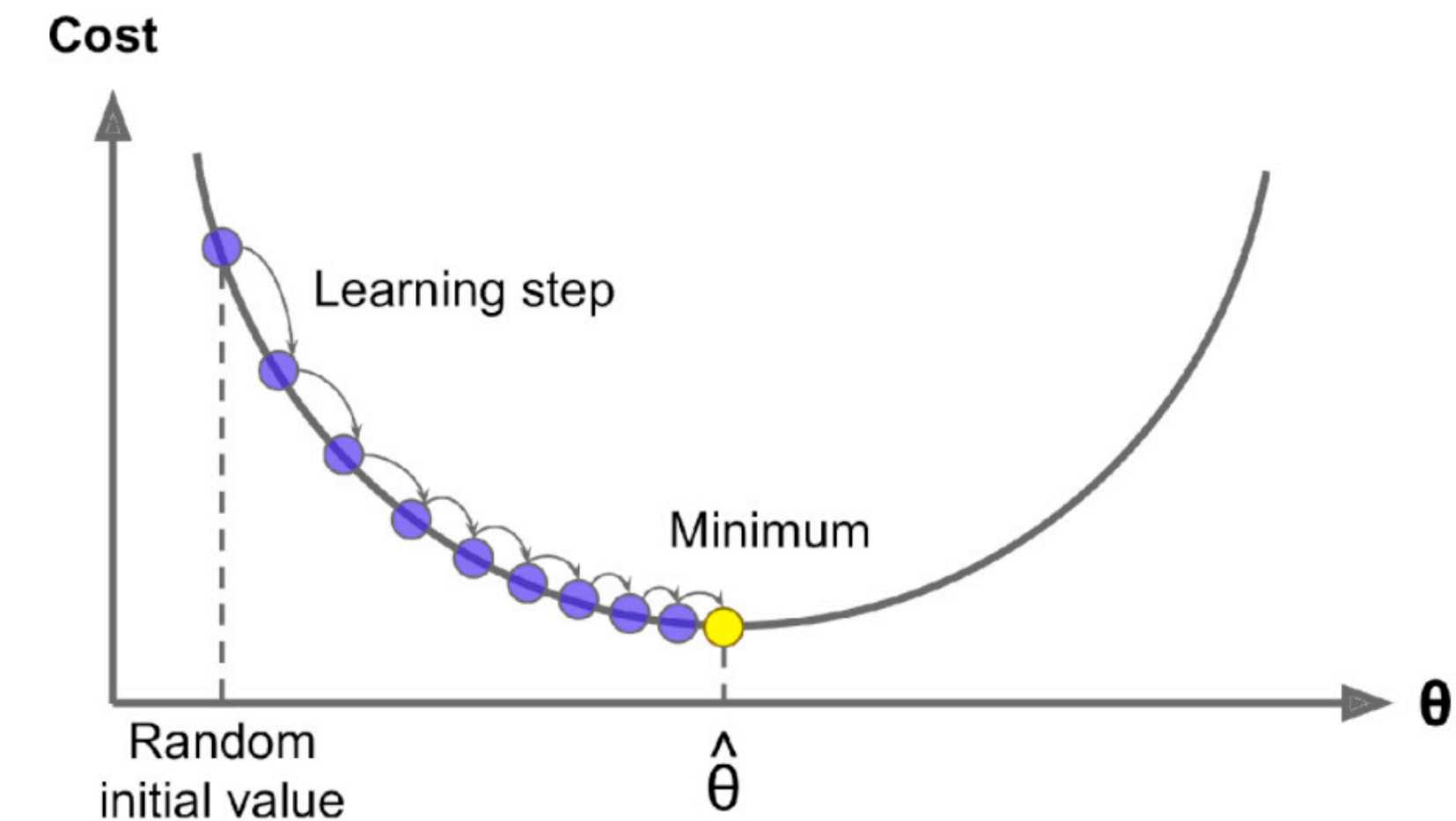
- Calcula o gradiente usando **todos os exemplos** do conjunto de dados antes de atualizar os parâmetros.
- Vantagens: Convergência estável e direção precisa.
- Desvantagens: Alto custo computacional em datasets grandes.

2. Stochastic Gradient Descent (SGD)

- Atualiza os parâmetros após **cada amostra** do dataset, tornando o processo mais ruidoso. Ou seja, Pega 1 exemplo, calcula o erro, ajusta o modelo e repete.
- Vantagens: Pode escapar de mínimos locais e convergir mais rapidamente em certos casos.
- Desvantagens: Oscilações na convergência e maior variabilidade nos updates.

3. Mini-Batch Gradient Descent (MBGD)

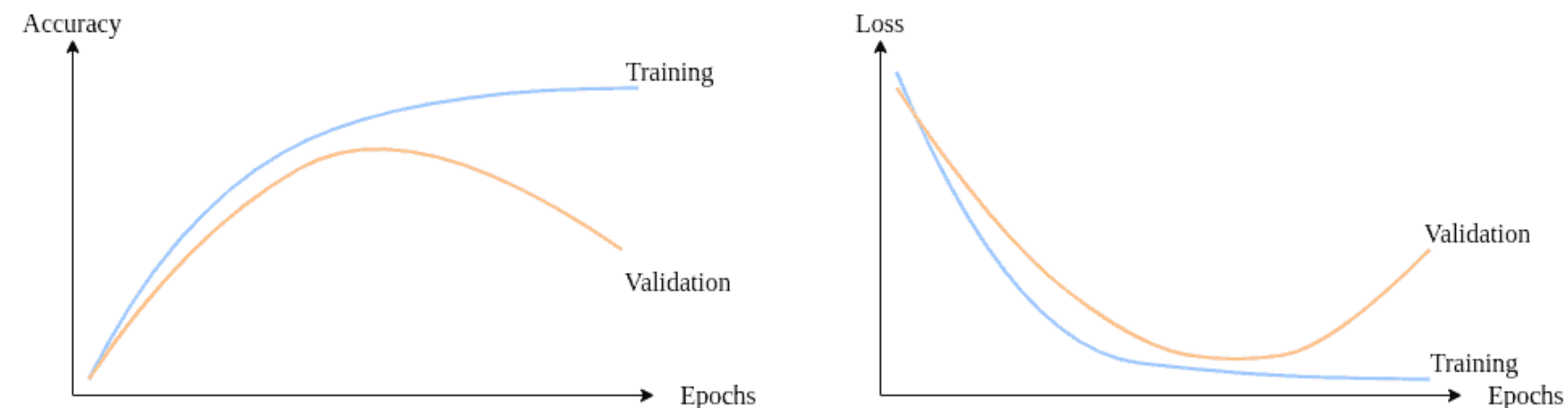
- Atualiza os parâmetros usando um **pequeno lote (mini-batch) de exemplos**.
- Vantagens: Combina estabilidade do BGD com eficiência do SGD.
- Desvantagens: Escolher um bom tamanho de mini-batch pode ser um desafio.



Redes neurais

Por que várias épocas?

- Uma única época normalmente não é suficiente para aprender bem. A rede precisa ver os dados várias vezes para ajustar os pesos.
- Poucas épocas → subajuste (underfitting)
- Muitas épocas → pode ocorrer sobreajuste (overfitting)
- Por isso, escolhe-se um número adequado de épocas (ex.: 50, 100, 500), muitas vezes com early stopping.



Redes neurais

Por que várias épocas?

- Uma rede neural aprende por meio do ajuste gradual de seus pesos internos, e esse processo não ocorre de uma única vez.
- No início do treinamento, os pesos são inicializados de forma aleatória, o que leva a previsões imprecisas e a um erro elevado.
- A cada vez que o modelo processa um conjunto de dados, calcula-se esse erro e os pesos são ajustados levemente na direção que tende a reduzi-lo.
- Esses ajustes são pequenos e progressivos, pois mudanças bruscas poderiam desestabilizar o processo de aprendizagem.

Redes neurais

Por que várias épocas?

- Assim, uma única passagem pelos dados não é suficiente para levar o modelo a um bom desempenho: é necessário repetir o processo diversas vezes para que os pesos se aproximem gradualmente de valores adequados.
- Cada nova época de treinamento funciona como um refinamento do conhecimento da rede. Na primeira passagem pelos dados, o modelo começa a captar padrões mais evidentes e relações gerais entre as variáveis.
- Com a repetição das épocas, esses padrões são ajustados e refinados, permitindo que a rede corrija erros e desenvolva representações internas mais precisas.
- Esse processo contínuo de revisão e aprimoramento é essencial para que o modelo alcance uma boa capacidade de generalização, isto é, para que consiga realizar previsões adequadas também em dados que não foram usados no treinamento.

Redes neurais

Por que várias épocas?

- Do ponto de vista matemático, a função de erro que a rede tenta minimizar é altamente complexa e não linear.
- Ela depende de milhares ou até milhões de parâmetros e apresenta uma paisagem de otimização repleta de vales e picos.
- Não existe uma fórmula direta que forneça os melhores valores dos pesos; em vez disso, utiliza-se um procedimento iterativo de otimização, como o gradiente descendente.
- A cada época, o modelo recalcula os gradientes e ajusta os pesos um pouco mais na direção do mínimo da função de perda.
- Esse processo iterativo exige múltiplas passagens pelos dados para que a solução encontrada seja suficientemente boa.

Redes neurais

Por que várias épocas?

- Uma analogia intuitiva pode ser feita com o estudo para uma prova.
- Ler o material apenas uma vez costuma ser insuficiente para compreendê-lo plenamente.
- Ao revisitar o conteúdo várias vezes, o estudante identifica erros, reforça conceitos e aprofunda sua compreensão.
- De modo semelhante, a rede neural precisa revisar repetidamente os dados de treinamento para consolidar padrões e melhorar seu desempenho.
- Cada época corresponde a uma nova oportunidade de aprendizado e ajuste.

Redes neurais

Por que várias épocas?

- No entanto, **treinar por muitas épocas** também pode trazer problemas. Se a rede continuar ajustando seus pesos indefinidamente com base nos mesmos dados, ela pode acabar **memorizando detalhes específicos do conjunto de treinamento**, perdendo a capacidade de generalizar para novos dados. Esse fenômeno é conhecido como **sobreajuste (overfitting)**. Por outro lado, **poucas épocas** podem levar ao **subajuste (underfitting)**, quando o modelo ainda não aprendeu o suficiente. Por isso, busca-se um equilíbrio, frequentemente utilizando técnicas como validação e interrupção antecipada (early stopping) para determinar o número adequado de épocas.

Redes neurais

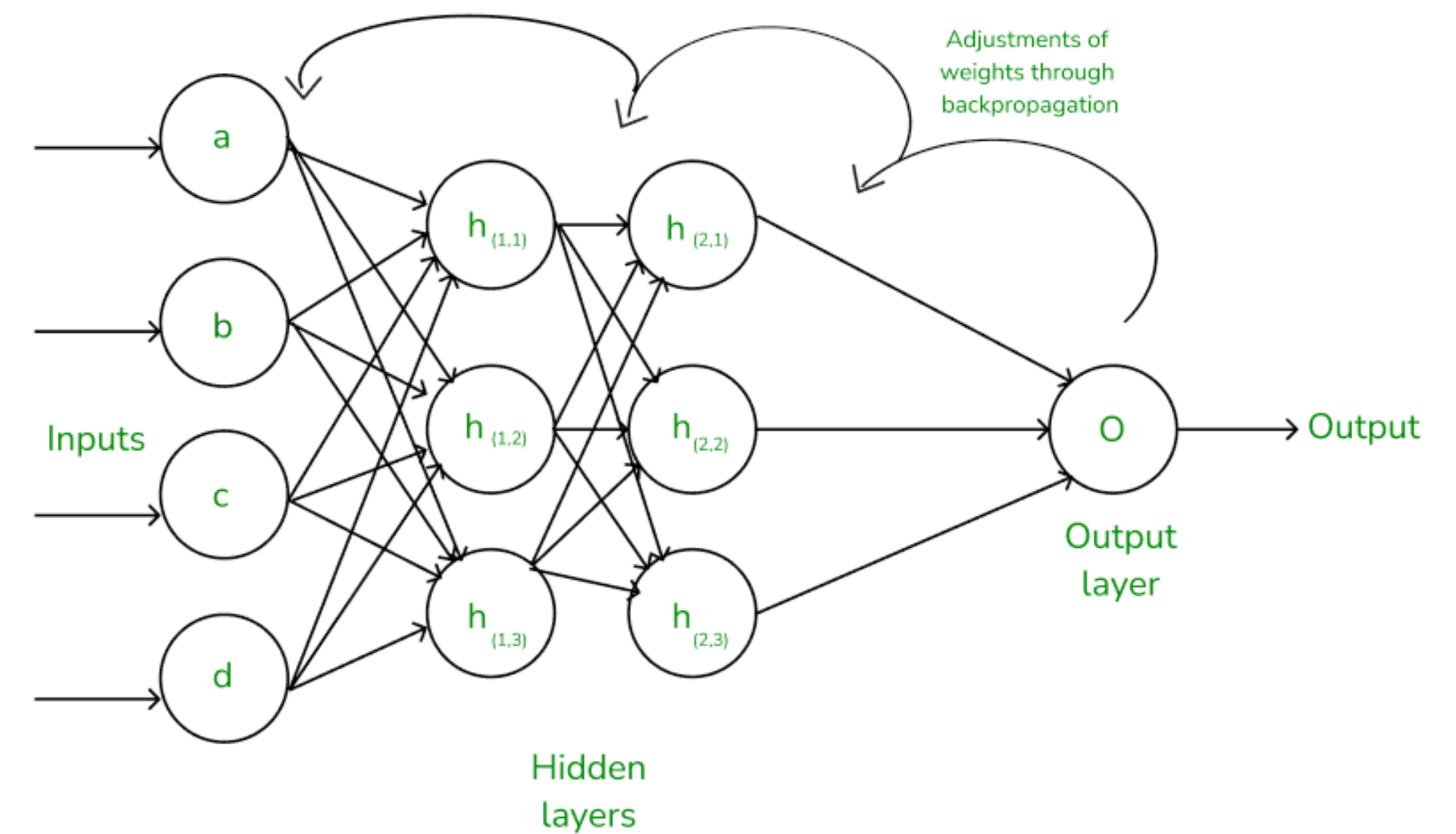
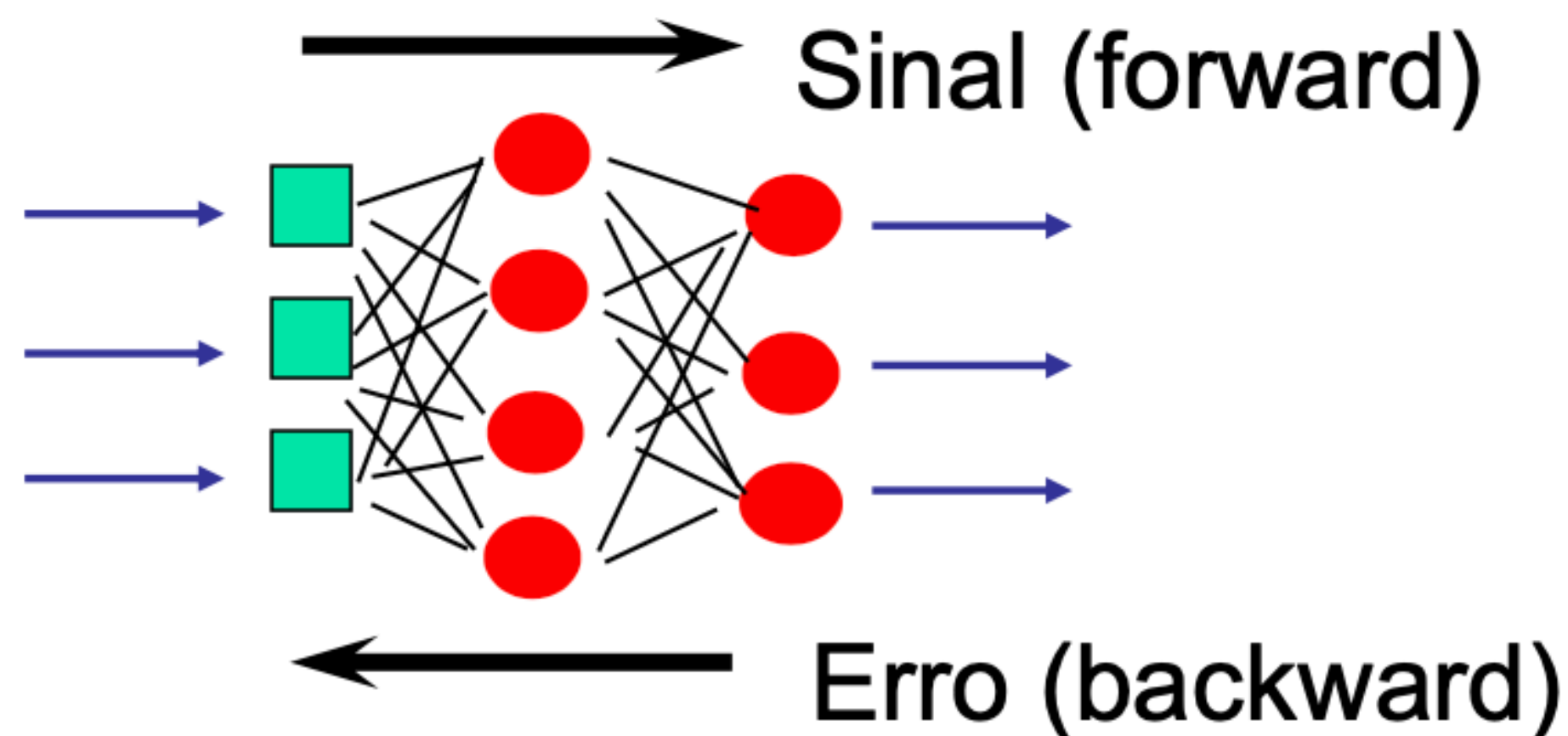
Por que várias épocas?

- Em uma visão mais profunda, o treinamento de uma rede neural pode ser entendido como a **resolução de um problema de otimização em alta dimensão**. O objetivo é encontrar o conjunto de parâmetros que minimiza a função de perda. Como essa minimização só pode ser realizada por métodos iterativos, cada época representa uma nova aproximação da solução. Sem repetição, o processo de otimização não converge e os pesos não se estabilizam. Dessa forma, ver os dados várias vezes não é redundante, mas parte essencial do mecanismo de aprendizagem das redes neurais.

Redes neurais

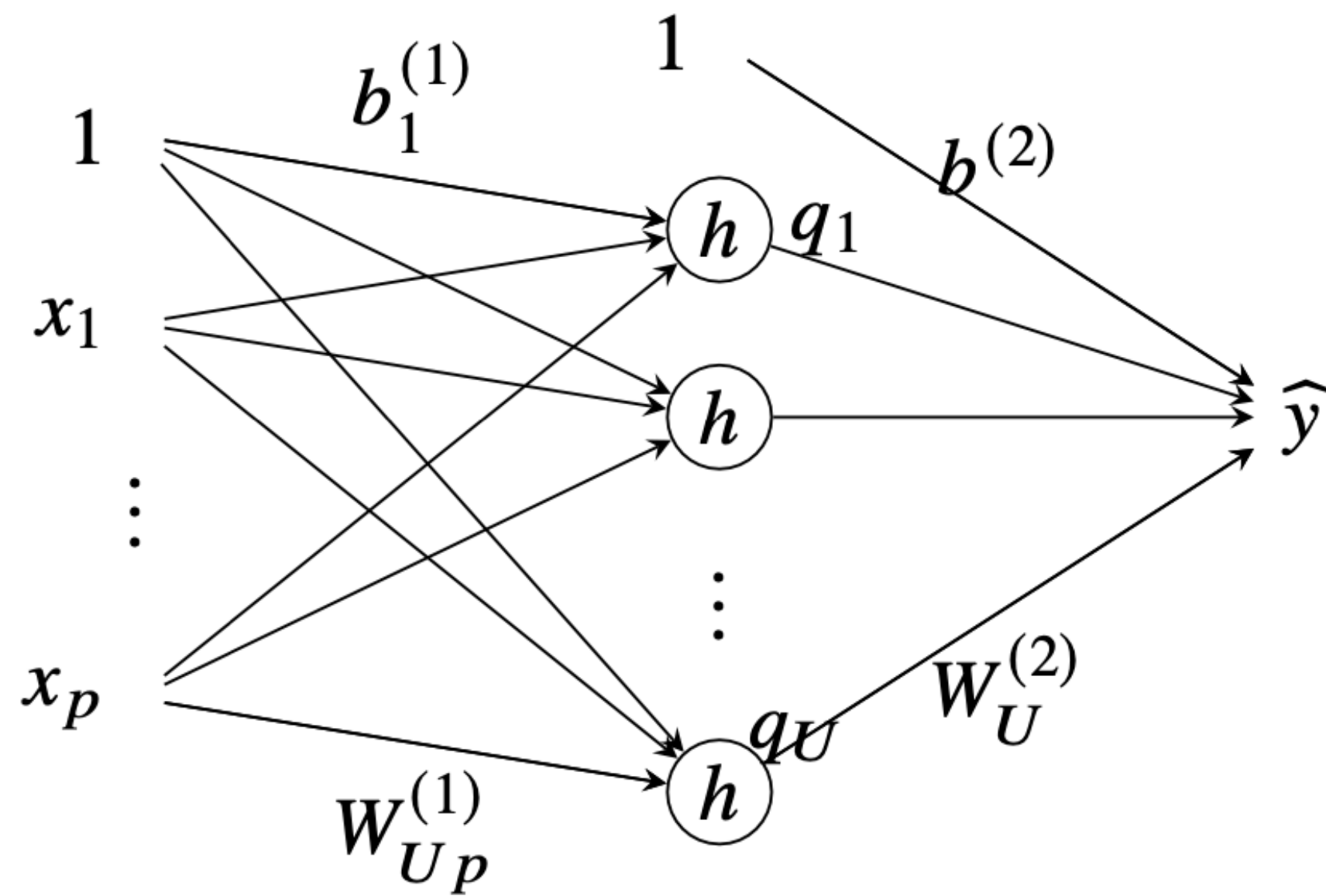
- **Treinamento**

- Treina a rede com pares entrada-saída
 - Cada vetor de entrada é associado a uma saída desejada.
- Treinamento em duas fases, cada uma percorrendo a rede em um sentido:
 - Fase forward
 - Fase backward



Redes neurais

Input variables Hidden units Output



$$q_1 = h \left(W_{11}^{(1)} x_1 + W_{12}^{(1)} x_2 + \cdots + W_{1p}^{(1)} x_p + b_1^{(1)} \right),$$

$$q_2 = h \left(W_{21}^{(1)} x_1 + W_{22}^{(1)} x_2 + \cdots + W_{2p}^{(1)} x_p + b_2^{(1)} \right),$$

$\vdots$

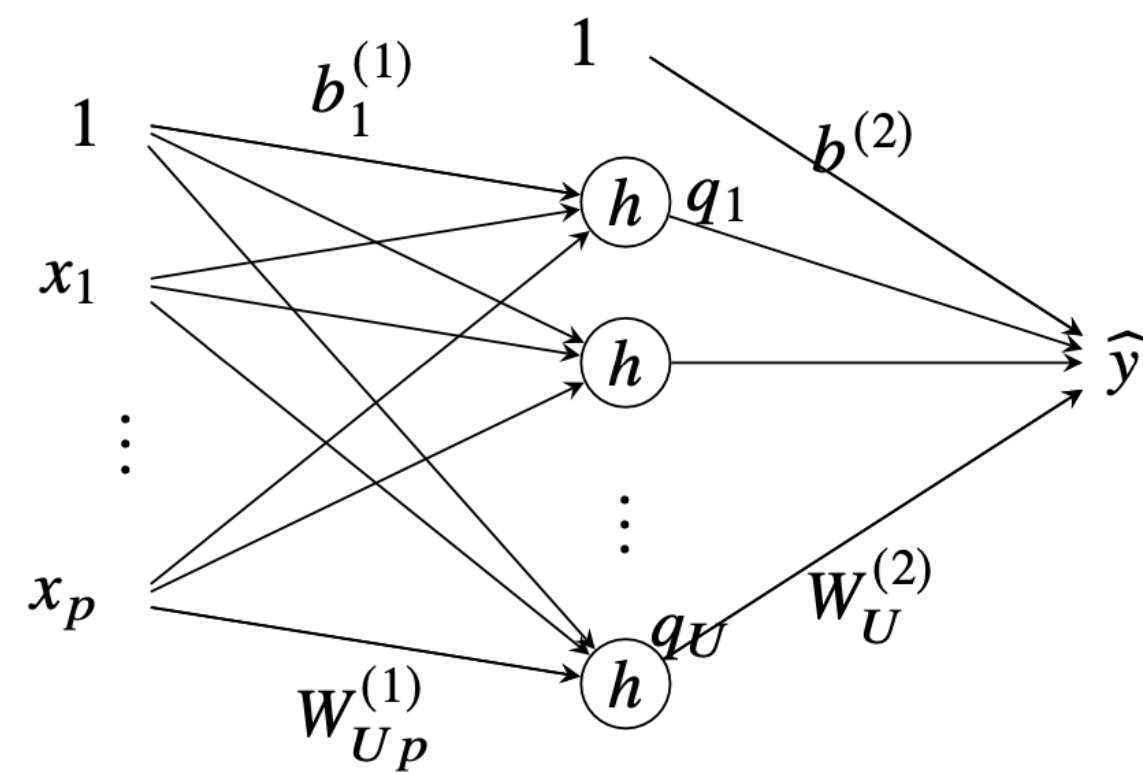
$$q_U = h \left(W_{U1}^{(1)} x_1 + W_{U2}^{(1)} x_2 + \cdots + W_{Up}^{(1)} x_p + b_U^{(1)} \right),$$

$$\hat{y} = W_1^{(2)} q_1 + W_2^{(2)} q_2 + \cdots + W_U^{(2)} q_U + b^{(2)}.$$

$$q_k = h \left(W_{k1} x_1 + W_{k2} x_2 + \cdots + W_{kp} x_p + b_k \right), \quad k = 1, \dots, U.$$

Redes neurais

Input variables Hidden units Output



$$q_1 = h \left(W_{11}^{(1)} x_1 + W_{12}^{(1)} x_2 + \cdots + W_{1p}^{(1)} x_p + b_1^{(1)} \right),$$

$$q_2 = h \left(W_{21}^{(1)} x_1 + W_{22}^{(1)} x_2 + \cdots + W_{2p}^{(1)} x_p + b_2^{(1)} \right),$$

$\vdots$

$$q_U = h \left(W_{U1}^{(1)} x_1 + W_{U2}^{(1)} x_2 + \cdots + W_{Up}^{(1)} x_p + b_U^{(1)} \right),$$

$$\hat{y} = W_1^{(2)} q_1 + W_2^{(2)} q_2 + \cdots + W_U^{(2)} q_U + b^{(2)}.$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} W_{11}^{(1)} & \cdots & W_{1p}^{(1)} \\ \vdots & & \vdots \\ W_{U1}^{(1)} & \cdots & W_{Up}^{(1)} \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} b_1^{(1)} \\ \vdots \\ b_U^{(1)} \end{bmatrix},$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} W_1^{(2)} & \cdots & W_U^{(2)} \end{bmatrix}, \quad \mathbf{b}^{(2)} = [b^{(2)}].$$

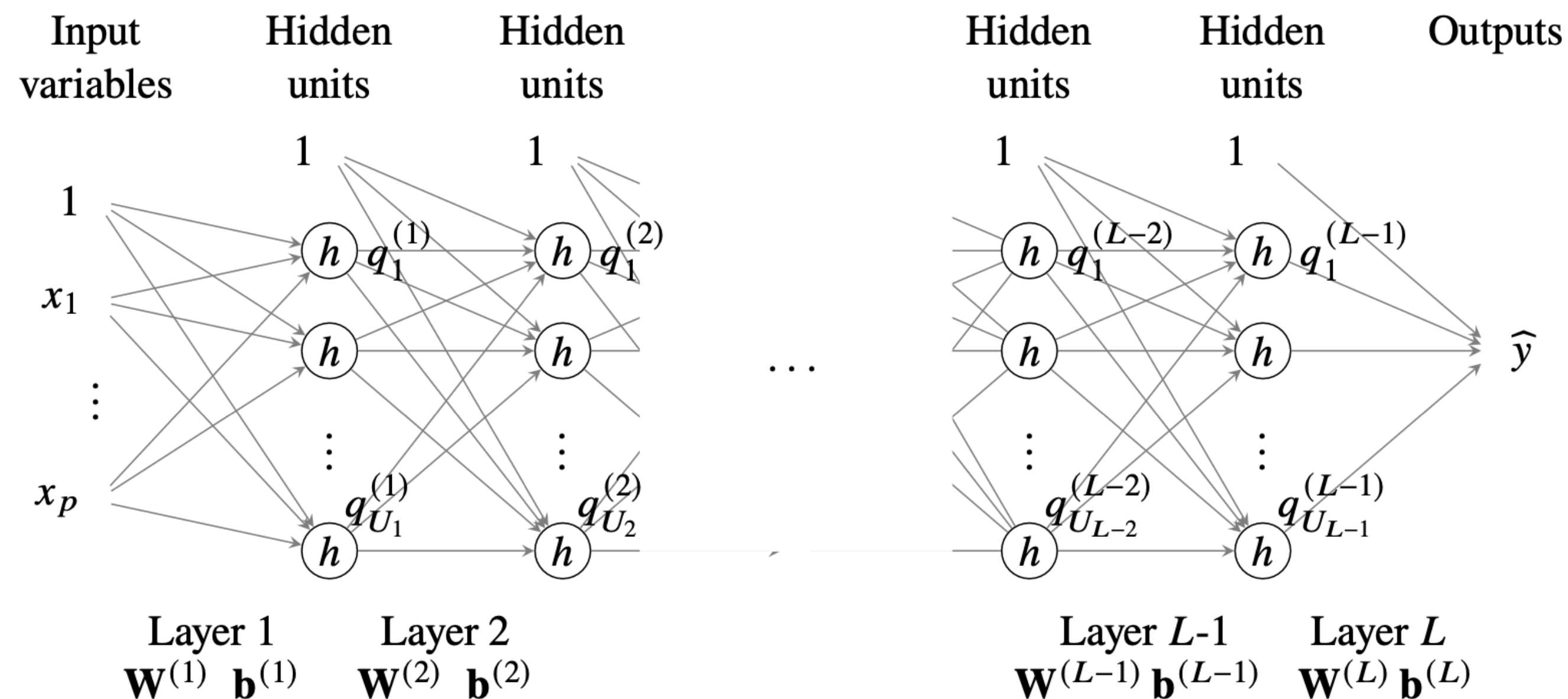
$$\mathbf{q} = h(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}),$$

$$\hat{y} = \mathbf{W}^{(2)} \mathbf{q} + \mathbf{b}^{(2)},$$

$$\boldsymbol{\theta} = [\text{vec}(\mathbf{W}^{(1)})^\top \quad \mathbf{b}^{(1)\top} \quad \text{vec}(\mathbf{W}^{(2)})^\top \quad \mathbf{b}^{(2)\top}]^\top,$$

Redes neurais

Deep neural networks



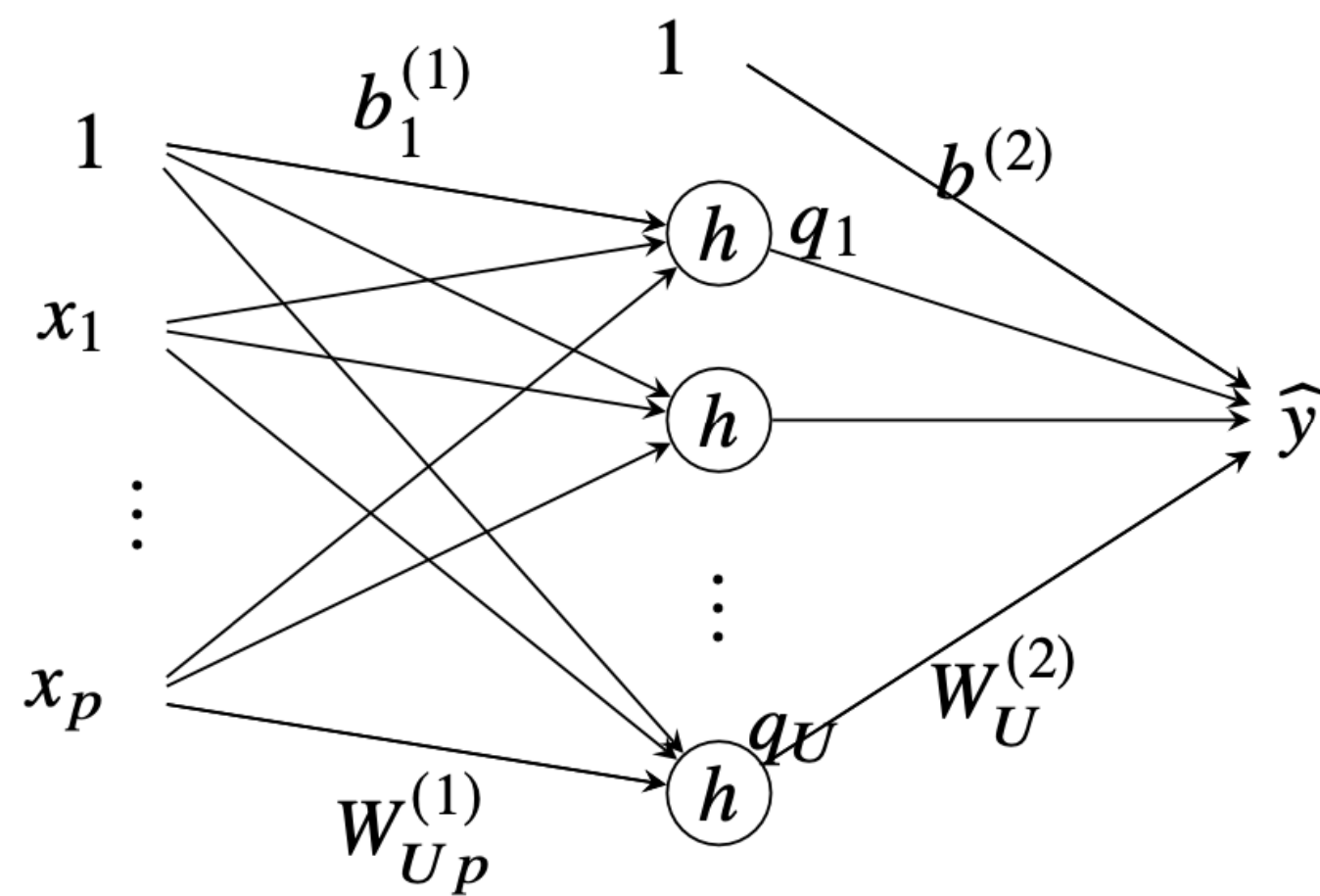
$$\begin{aligned} \mathbf{q}^{(1)} &= h(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}), \\ \mathbf{q}^{(2)} &= h(\mathbf{W}^{(2)} \mathbf{q}^{(1)} + \mathbf{b}^{(2)}), \\ &\vdots \\ \mathbf{q}^{(L-1)} &= h(\mathbf{W}^{(L-1)} \mathbf{q}^{(L-2)} + \mathbf{b}^{(L-1)}), \\ \hat{y} &= \mathbf{W}^{(L)} \mathbf{q}^{(L-1)} + \mathbf{b}^{(L)}. \end{aligned}$$

$$\mathbf{q}^{(l)} = h(\mathbf{W}^{(l)} \mathbf{q}^{(l-1)} + \mathbf{b}^{(l)}).$$

Redes neurais

Notação vetorial

Input variables Hidden units Output



$$\mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}, \quad \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_n^\top \end{bmatrix}, \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_n \end{bmatrix}, \quad \text{and} \quad \mathbf{Q} = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_n^\top \end{bmatrix}.$$

$$\mathbf{Q} = h \left(\mathbf{XW}^{(1)\top} + \mathbf{b}^{(1)\top} \right),$$
$$\hat{\mathbf{y}} = \mathbf{QW}^{(2)\top} + \mathbf{b}^{(2)\top},$$

$$\mathbf{q}_i^\top = h(\mathbf{x}_i^\top \mathbf{W}^{(1)\top} + \mathbf{b}^{(1)\top}),$$
$$\hat{y}_i = \mathbf{q}_i^\top \mathbf{W}^{(2)\top} + \mathbf{b}^{(2)\top}, \quad i = 1, \dots, n.$$

Redes neurais

Treinamento

$$\boldsymbol{\theta} = [\text{vec}(\mathbf{W}^{(1)})^\top \quad \mathbf{b}^{(1)\top} \quad \text{vec}(\mathbf{W}^{(2)})^\top \quad \mathbf{b}^{(2)\top}]^\top,$$

$$\hat{\theta} = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n L(\mathbf{x}_i, y_i, \theta)$$

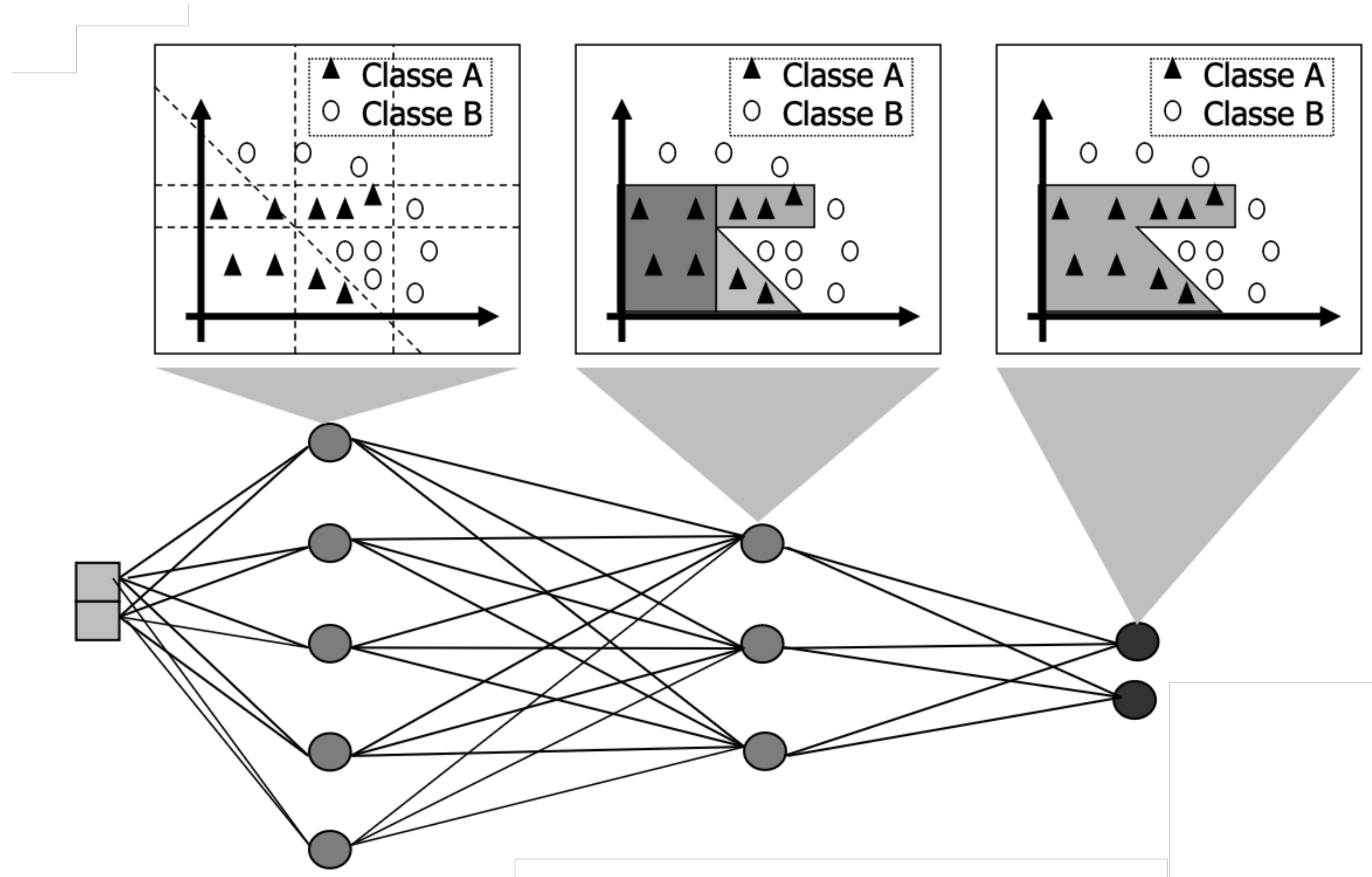
Regressão: $L(\mathbf{x}, y, \theta) = (y - f(\mathbf{x}; \theta))^2$

Classificação: $L(\mathbf{x}, y, \theta) = -y \log f(\mathbf{x}; \theta) - (1 - y) \log(1 - f(\mathbf{x}; \theta))$

Cross entropy.

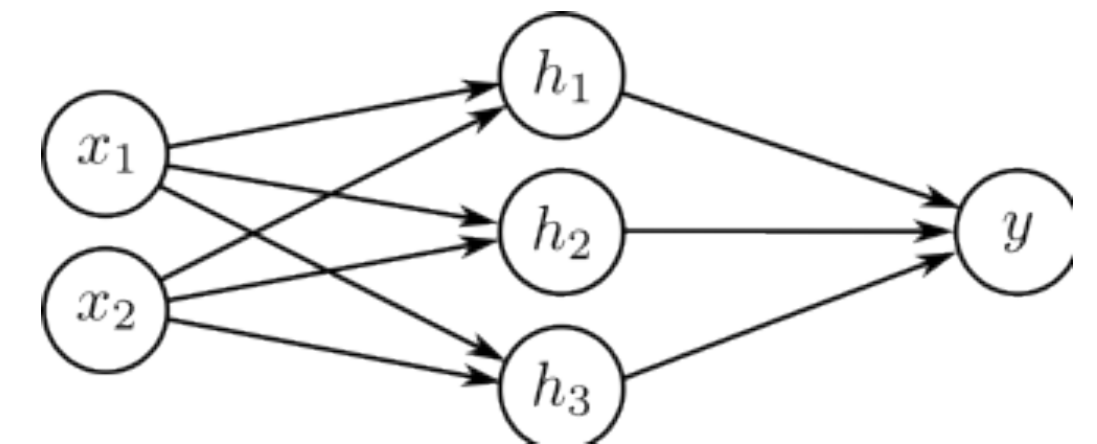
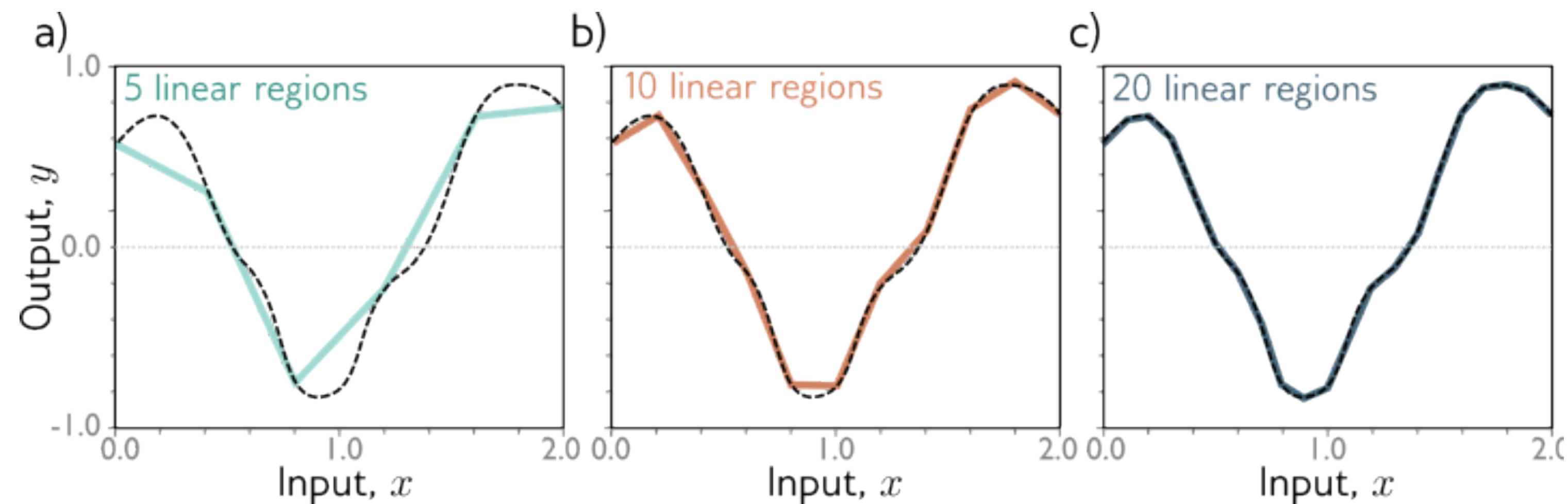
Outras funções de perda podem ser usadas!

Redes neurais



Redes neurais

Universal approximation theorem



“Uma prova formal de que, com unidades ocultas suficientes, uma rede neural pode descrever qualquer função contínua em um subconjunto compacto do $\mathbb{R}^D$ com precisão arbitrária”.

Hiperparâmetros comuns

- **Hiperparâmetros de arquitetura**

- Número de camadas (profundidade): Define quantas camadas ocultas a rede terá.
- Número de neurônios por camada: Afeta a capacidade de representação da rede.
- Função de ativação: Ex: ReLU, tanh, sigmoid, LeakyReLU. Influencia a não-linearidade da rede.

- **Hiperparâmetros de otimização**

- Taxa de aprendizado (learning rate): Controla o tamanho do passo durante a atualização dos pesos.
- Algoritmo de otimização: Ex: SGD, Adam, RMSprop, Adagrad. Cada um tem características específicas.
- Tamanho do batch (batch size): Quantos exemplos são usados por vez na atualização dos pesos.
- Número de épocas (epochs): Quantas vezes o conjunto de dados é passado pela rede durante o treinamento.

Hiperparâmetros comuns

- **Hiperparâmetros de regularização**
 - Dropout rate: Probabilidade de desligar neurônios durante o treinamento para evitar overfitting.
 - L1 / L2 regularization (penalidades nos pesos): Adicionam termos ao custo para limitar o crescimento dos pesos.
 - Early stopping: Parar o treinamento quando a validação não melhora após certo número de épocas.

Redes neurais

- **Vantagens:**

- Capacidade de Modelar Relações Complexas.
- Podem se adaptar a novos dados sem a necessidade de reprogramação explícita, permitindo aprendizado contínuo.
- Operações de redes neurais podem ser facilmente paralelizadas e aceleradas por hardware especializado, como GPUs, permitindo processamento eficiente de grandes volumes de dados.
- Diferentemente de modelos estatísticos tradicionais, as redes neurais podem trabalhar diretamente com imagens, áudio, texto e séries temporais sem exigir engenharia manual intensiva de recursos.
- São extremamente eficazes em reconhecimento de padrões, visão computacional, processamento de linguagem natural e reconhecimento de fala.
- Redes bem treinadas podem ser robustas a ruído nos dados, conseguindo manter um desempenho razoável mesmo quando os dados de entrada apresentam variações ou imperfeições.

Redes neurais

- **Desvantagens:**

- Redes neurais, especialmente as profundas, exigem grandes volumes de dados para treinamento eficaz.
- Treinar redes neurais pode ser extremamente exigente em termos de poder computacional, demandando GPUs para processamento eficiente.
- As redes neurais funcionam como "caixas-pretas", dificultando a interpretação dos resultados e a explicação das decisões tomadas pelo modelo.
- Dependendo da arquitetura e do volume de dados, o treinamento pode levar horas, dias ou até semanas.
- O desempenho das redes neurais depende fortemente da escolha da arquitetura (número de camadas, tipo de ativação, número de neurônios, etc.) e dos hiperparâmetros (taxa de aprendizado, regularização, etc.), o que exige experimentação intensiva e otimização.
- O treinamento de redes neurais é baseado em otimização numérica, que pode levar a mínimos locais ou problemas de estabilidade. Isso significa que nem sempre há garantia de que o modelo atingirá uma solução ótima.

Leitura adicional

- **Machine Learning - A First Course for Engineers and Scientists**

Andreas Lindholm, Niklas Wahlström,
Fredrik Lindsten, Thomas B. Schön

<http://smlbook.org/book/sml-book-draft-latest.pdf>

- **Understanding Deep Learning**, Simon Prince.

<https://udlbook.github.io/udlbook/>

